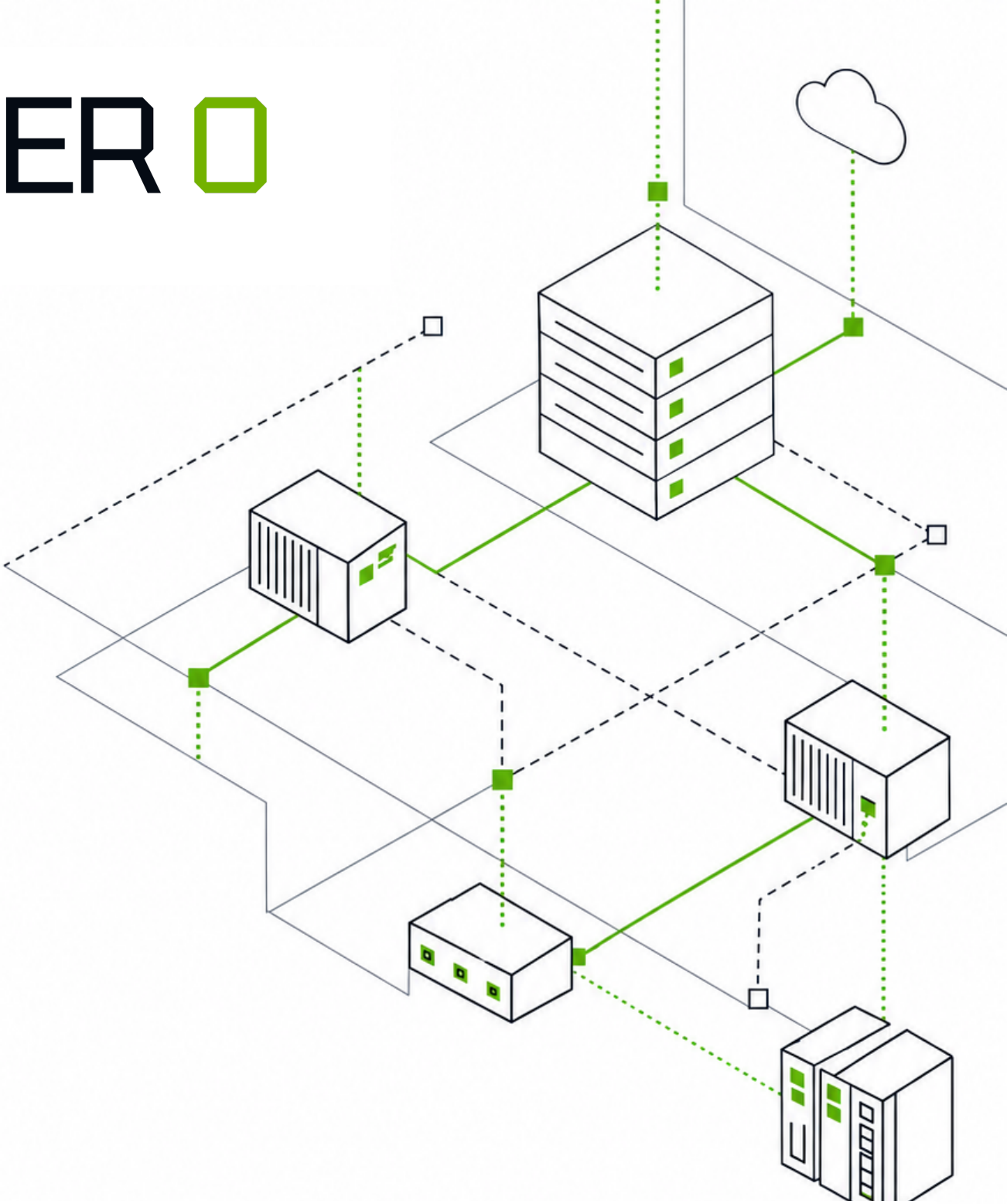
TIER 0



Tier0 기술 가이드

목차

시작하기

1. 최적의 버전 선택
2. 설치
3. Tier0에서 Demo Factory 체험

TIER0 사용

1. 공장 데이터 기반
2. 데이터 기반 준비
3. 센터에서 엣지까지 데이터 관리
4. 공장 데이터 작업
5. 앱 구축하기
6. Digital Twin 구축하기
7. 데이터 분석
8. Agents로 Tier0 운영하기

베스트 프랙티스

1. 공장 데이터 모델링
2. 산업 프로토콜 연결
3. 보안 및 복원력
4. 분석 앱 구축
5. Agent로 데이터 워크플로 구축하기
6. Digital Twin으로 데이터 표시하기

참조

1. API 레퍼런스
2. CLI 명령 참조
3. 내장 Builder Skills
4. 사용자 관리
5. Tier0 Cloud 네트워크 및 데이터 보안 안내
6. ISA-95 설비 계층
7. 표준 배포 계획
8. 고가용성 배포 계획
9. SLA 및 High Availability 경계
10. 표준 Port 목록
11. 운영 Runbook

부록

1. 전체 OpenAPI 레퍼런스

시작하기

최적의 버전 선택

Tier0 Edge, Cloud, Enterprise의 포지셔닝, 요금제, 기능 매트릭스를 비교합니다.

Tier0는 세 가지 에디션으로 제공되는 하나의 platform입니다. namespace model, flow, CLI는 동일하며, 차이는 누가 운영하는지와 사용할 수 있는 platform 범위입니다.

■ OPEN SOURCE

Edge

한 대의 machine에서 운영하는 UNS foundation입니다.

> PoC, 기술 평가, open source를 직접 운영하는 team에 적합합니다.

Apache-2.0

FREE

UNS core, SourceFlows/EventFlows, history storage를 포함한 single-machine Docker deployment입니다.

GitHub에서 clone ->

■ MANAGED SAAS

Cloud

대부분의 team이 여기서 시작합니다

운영을 대신 맡기는 full platform입니다.

> 첫날부터 apps, notebooks, launchpad를 사용할 수 있으며 직접 운영할 infrastructure가 없습니다.

Builder

$199/SEAT/MO

App generation만 포함합니다.

Growth ★ $20,000/YR

최대 5개 edge. 몇 개의 app으로 빠르게 시작하려는 single factory에 적합합니다.

Scale $38,000/YR

최대 10개 edge. 여러 factory와 많은 app을 운영하는 사용자에게 적합합니다.

7일 평가판 시작 ->

■ PRIVATE DEPLOYMENT

Enterprise

고객 환경에 맞춰 운영하는 full platform입니다.

> data sovereignty, scale, governance, enterprise-grade oversight가 필요한 경우에 적합합니다.

Base $10,000/YR

Unified data foundation과 소수의 single-purpose app을 제공합니다.

Plus $20,000/YR

Single Instance. single-factory data integration과 여러 use case의 app에 적합합니다.

Cluster ★ $39,900+/YR

Multi-Instance. 여러 factory, 많은 app, 중앙 집중식 private-cloud management에 적합합니다.

team에 문의 ->

Add-ons: 추가 edge node는 $2,000 /edge/year, 추가 instance는 $10,000 /instance/year입니다. 가격은 참고용이며, 공식 기준은 tier0.app/pricing입니다.

ⓘ 용어 설명
Cloud plans에서 edge는 cloud Tier0와 통신하는 connection node입니다. Edge Tier0일 수도 있고 gateway 또는 industrial PC일 수도 있습니다.

Capability Matrix

Capability	Edge	Cloud	Enterprise
UNS / Data Modeling	✓ single-machine UNS	✓ Growth / Scale	✓ Base / Plus / Cluster
Industrial protocols	✓ MQTT	✓ Growth / Scale: MQTT, REST, i3X, OPC UA	✓ Base: MQTT; Plus / Cluster: MQTT, REST, i3X, OPC UA
UNS Agent	×	✓ Growth / Scale	×
Notebook (Advanced Analysis)	×	✓ Growth / Scale	✓ Plus / Cluster
Vision	×	✓ Scale	✓ Plus / Cluster
Anchor	×	✓ Scale	✓ Cluster
App Builder + Template Library	×	✓ Builder / Growth / Scale	×
Launchpad / My Apps	×	✓ Builder / Growth / Scale	✓ Base / Plus / Cluster
Audit / app & system logs	×	✓ Growth / Scale	✓ Plus / Cluster; SIEM in Cluster
HA / multi-instance / governance	×	×	✓ Cluster

Capability	Edge	Cloud	Enterprise
Operations	사용자가 직접 운영	FREEZONEX 운영	사용자 운영, support 제공

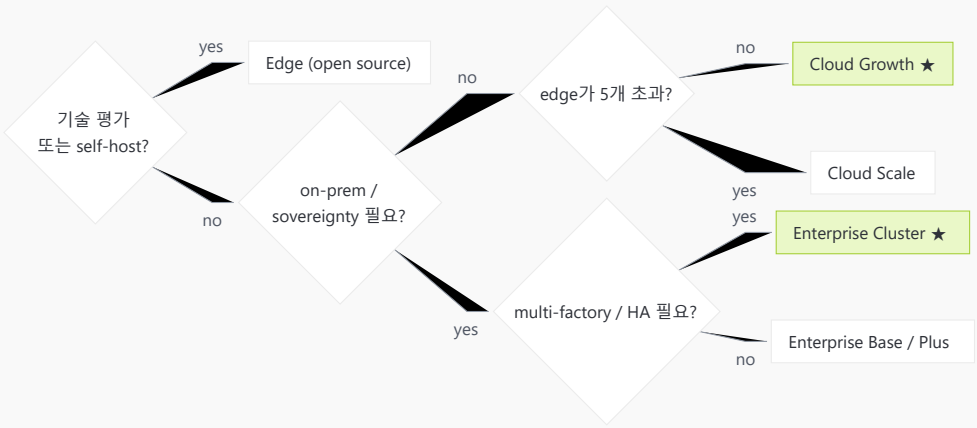
Edge 하드웨어 요구사항

> **Edge를 사용하려면**
> Edge는 기술 평가 목적에 적합하며 운영 경험이 필요합니다. 사용 전에 환경이 아래 hardware requirements를 만족하는지 확인하세요.

	Minimum	Recommended
CPU	4 cores	8 cores
Memory	8 GB	16 GB
Disk	100 GB (1000 IOPS)	1 TB
OS	Ubuntu 24.04, Windows 10/11 (Docker)	-

Decision Tree

아직 어떤 option을 선택할지 모르겠다면 아래 path를 따라 결정하세요.



“edge” 단위란?

Cloud plans에서 *edge*는 장비 가까이에서 데이터를 수집하고 namespace로 publish하는 connection point입니다. 일반적으로 collection flows를 실행하는 gateway 또는 industrial PC입니다. 대략 site 또는 독립 network segment마다 하나로 계산합니다. 위의 open-source distribution인 **Edge edition**과 혼동하지 마세요.

다음 단계

- UNS에서 앱 구축 - Edge와 Enterprise에서 container를 배포합니다.
- 설치 - 7일 Cloud 평가판은 full platform입니다.

시작하기

설치

Tier0 설치 옵션을 선택하고 시작합니다.

Tier0는 세 가지 edition으로 사용할 수 있습니다.

- **Edge** - 단일 machine 또는 Docker에서 직접 실행합니다.
- **Cloud** - 운영 인프라가 필요 없는 managed SaaS입니다.
- **Enterprise** - enterprise environment를 위한 private deployment입니다.

어떤 edition을 선택해야 할지 확실하지 않다면 최적의 버전 선택을 먼저 확인하세요.

Tier0 설치

탭: CLOUD

Tier0 Cloud는 설치가 필요하지 않습니다. signup 후 workspace를 만들면 됩니다.

1. Register Tier0에서 계정을 만듭니다.
2. email confirmation을 완료하고 login page로 돌아갑니다.
3. **Login** button을 클릭해 계정에 로그인합니다.
4. 기본 정보를 입력하고 **Continue**를 클릭합니다.
5. **Create a New Workspace**를 선택합니다.
6. workspace를 만들고 Cloud 환경을 사용합니다.

탭: ENTERPRISE

요구사항

- 단일 노드 배포
  - 최소: 4C / 8G / 100G
  - 권장: 8C / 16G / 500G
- Standard Deployment
  - 최소: 4 VMs

    - VM resource 권장:
        - App: 8C / 32G / 100G
        - Database: 4C / 8G / 500G
        - Witness: 2C / 4G / 40G
- High Availability Deployment
    - 최소: 6 VMs
    - VM resource 권장:
        - App (Swarm): 8C / 32G / 100G x 3
        - Database: 4C / 8G / 500G x 2
        - Witness: 2C / 4G / 40G

> 구체적인 **requirements**는 **deployment scenario**에 따라 달라질 수 있습니다
> Enterprise hardware requirements는 deployment scenario에 따라 달라질 수 있습니다.

> 참고
> 자세한 배포 정보는 표준 배포 계획과 고가용성 배포 계획을 확인하세요.

설치

1. package를 control host에 복사하고 검증합니다.

```
sha256sum -c tier0-enterprise-offline-standard-
YYYYmmddTHHMMSS.tar.gz.sha256
```

2. package를 압축 해제합니다.

```
tar -xzf tier0-enterprise-offline-standard-YYYYmmddTHHMMSS.tar.gz
cd tier0-enterprise-offline-standard-YYYYmmddTHHMMSS
```

3. deployment detail에 맞게 `install.env`를 수정한 뒤 deployment를 시작합니다.

```
vi install.env
bash install.sh
```

유용한 운영 **command**

- Enterprise 설치 전 deployment configuration을 점검합니다.

```
./install.sh --check-only
```

- 실패한 deployment 후 모든 node를 정리합니다.

```
./full-cleanup.sh --env install.env --yes
```

> 이 command는 VM, operating system, installation package, Docker software는 유지하지만 Tier0 runtime data와 database backup을 삭제합니다.

- 설치 후 deployment report가 생성됩니다.

```
runtime/install-reports/<timestamp>/deployment-report.md
```

> report에 **FAIL** 항목이 있으면 project를 공유하기 전에 먼저 support에 문의하세요.

탭: EDGE

요구사항

- 최소 4-core CPU, 8 GB RAM, 100 GB disk
- Docker와 Git이 설치된 Ubuntu 24.04 또는 Windows 10/11

설치

1. GitHub에서 Tier0-Edge repository를 서버에 clone하고 project로 이동합니다.

```
git clone https://github.com/FREEZONEX/Tier0-Edge
cd Tier0-Edge/deploy
```

2. 배포 전에 `.env.default`에서 필요한 environment parameter를 수정합니다.

```
```

```
cp .env.default .env
vi .env
:wq!
```

ⓘ 설정 가능한 **parameters**

Field	Type	Description	Example
VOLUMES_PATH	string	project data 저장 경로	Linux에서는 `/srv/tier0/data`, Windows Bash에서는 `D:/tier0/data` 또는 `/d/tier0/data`
ENTRANCE_DOMAIN/ENTRANCE_PORT	string	Tier0 access endpoint(IP/domain + port)	`192.168.1.10` / `8088`
LANGUAGE	enum	zh-CN / en-US	`en-US`
ADMIN_INIT_PASSWORD	string	admin user initial password	안전한 private password
COMPOSE_PROJECT_NAME/PORT_OFFSET	string	같은 host에서 실행되는 여러 instance를 격리	`edge_lab` / `100`

3 install script를 실행하고 installation status를 확인합니다.

```
bash bin/install.sh
bash bin/compose.sh ps # container status 확인
bash bin/compose.sh logs -f backend # backend container log 확인
curl -fsS http://127.0.0.1:8088/readyz # backend service readiness 확인
```

> ⚠ 주의
> `deploy/.env.runtime`는 수정하지 마세요. 생성된 secrets와 resolved runtime values가 포함되어 있습니다.

4. 설정한 endpoint와 기본 admin account로 Tier0에 접속합니다.
   - Username: `tier0`
   - Password: `.env`의 `ADMIN_INIT_PASSWORD` 값. 설정하지 않았다면 `tier0`.

다음 단계

- Demo Factory 체험 - 설치하지 않고 Tier0 workflow를 확인합니다.
- 최적의 버전 선택 - requirement에 맞는 edition을 선택합니다.

시작하기

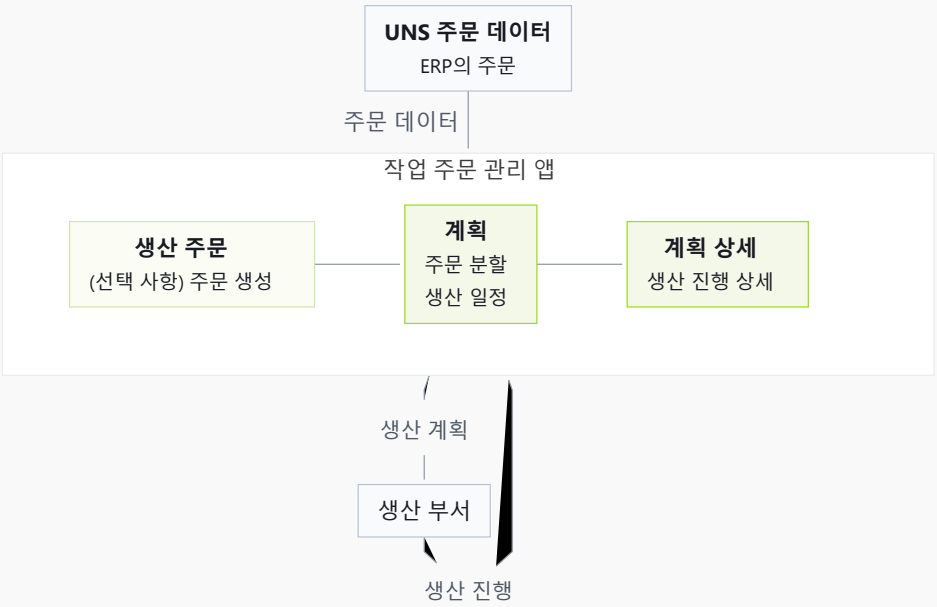
Tier0에서 Demo Factory 체험

Cloud 평가판 workspace에서 namespace, flows, notebooks, apps가 포함된 사전 구성된 실행 가능한 공장을 살펴봅니다.

Tier0를 가장 빠르게 이해하는 방법은 이미 작동하는 공장을 살펴보는 것입니다.

비즈니스 배경

Tier0의 공장은 ERP에서 주문을 받고, 주문을 분할하고, 생산 일정을 만들고, 그 일정을 생산 부서로 보내며, 작업이 진행되는 동안 생산 진행 데이터를 받습니다.



프로세스 이해

1. Tier0에서 **UNS**로 이동해 ERP에서 온 주문의 세부 정보를 확인합니다.
   - `DemoFactory/ERP/ProductionOrders/State/UpsertProductionOrder` : 주문.
   - `DemoFactory/ERP/ProductionOrders/State/OrderList` : 현재 주문 목록 스냅샷.

> 🚀 데이터는 어떻게 **UNS**로 수집되나요?
> **Flows** > **Source Flow** > **DemoFactory-Flow**로 이동해 데이터 수집 세부 정보를 확인합니다.

2. (선택 사항) **Launchpad**로 이동해 **Work Order Management** 애플리케이션을 열고 **Production Orders** 페이지에서 주문을 생성합니다.
3. **Work Order Management**에서 **Plans** 페이지를 사용해 주문을 분할하고, 분할된 작업 주문의 생산 계획을 예약합니다.
4. 계획을 생산 부서로 보내고 **UNS**의 다음 topics에서 계획 세부 정보를 확인합니다.
   - `DemoFactory/ERP/WorkOrderPlan/Metric/SplitCount` : 분할 후 작업 주문 수.
   - `DemoFactory/ERP/WorkOrderPlan/State/PlanStatus` : 현재 계획 상태.
   - `DemoFactory/ERP/WorkOrderPlan/State/WorkOrderList` : 생산 계획 일정 이후의 작업 주문 목록.

> ⓘ 참고
> 애플리케이션은 plans와 workorders를 **UNS**에 직접 다시 씁니다.

5. 애플리케이션의 **Plan Details** 페이지에서 생산 진행 데이터를 확인합니다.

> 🚀 상세 데이터는 어디에서 오나요?
> 생산 진행 데이터는 생산 부서에서 오며, **Source Flow**의 **DemoFactory-Flow**가 이를 수집해 **UNS**에 게시하고 **Plan Details** 페이지에 표시합니다.
>
> - `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/State/CurrentWorkOrder`: 진행 중인 주문.
> - `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/State/WorkOrderStatus`: 현재 주문의 실행 상태.
> - `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Target_Qty`: 현재 주문의 목표 생산량.

- `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Produced_Qty`: 현재 주문에서 완료된 제품 수량.
- `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Defect_Qty`: 현재 주문의 불량 수량.
- `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Completion_Rate`: 현재 주문의 완료율.

다음 단계

- 가장 적합한 버전 선택 - 각 버전을 비교하고 가장 적합한 버전을 선택합니다
- **UNS 개념** - Unified Namespace의 데이터 모델링을 이해합니다

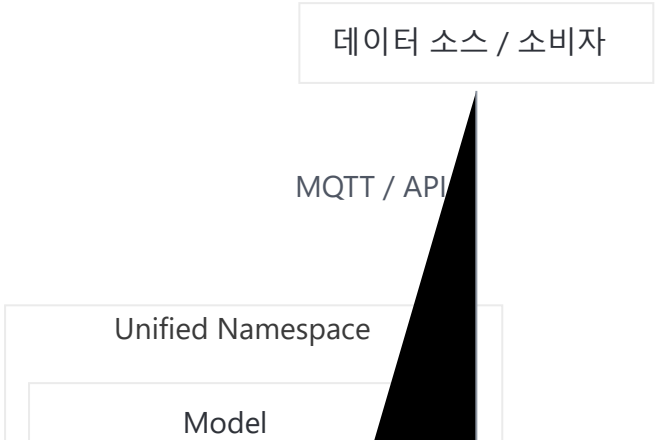
Tier0 사용

공장 데이터 기반

Unified Namespace model: Metric, State, Action, 그리고 UNS가 공장 데이터를 저장하는 방식.

Tier0에서 공장 데이터는 UNS(Unified Namespace) 방법론을 기반으로 명확한 트리 구조의 모델로 구성되어 통합 데이터 기반을 형성합니다.

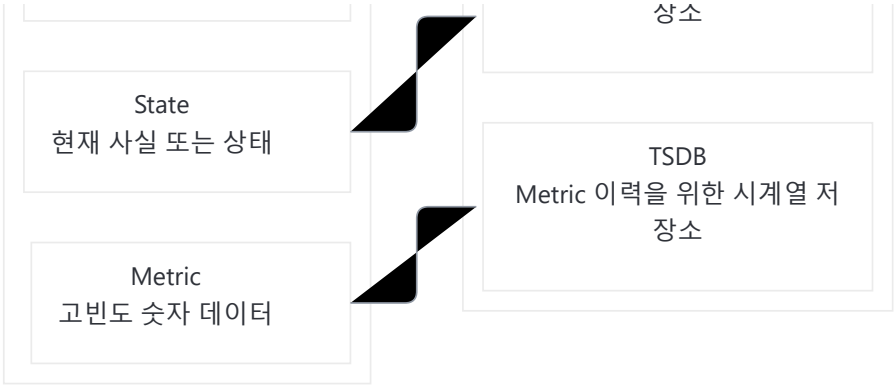
- Broker는 model topic을 통해 데이터를 수신하고 전송합니다.





- Model은 **Metric**, **State**, **Action** 세 가지 type으로 분류됩니다.
- 데이터는 model type에 따라 서로 다른 database에 저장됩니다.
  - **Action/State**: JSONB (PostgreSQL)
  - **Metric**: real-time data(TSDB)





Model

> 🚀 **model을 만들 때 어떤 guide를 따라야 하나요?**
> 데이터가 무엇을 나타내는지에 따라 modeling 방법을 선택합니다.
>
> - **Physical Data**: machine에서 나오는 real-time data입니다. 보통 **ISA-95** standard를 기반으로 하며 location hierarchy에 크게 의존합니다.
> - **Business Data**: 생산 요구사항, schedule, task, dispatching과 관련된 데이터입니다. 보통 **business domain**과 **object** 중심으로 구성합니다.

Metric, State, Action

산업 데이터는 collection frequency와 목적이 서로 다르기 때문에 Tier0는 UNS model을 세 가지 고정 type으로 구분합니다.

Type	Why	Example
`METRIC`	지속적으로 수집되고 time-series storage가 필요한 고빈도 데이터를 위해 설계되었습니다.	machine temperature / pressure / voltage
`STATE`	더 낮은 빈도로 변경되며 object의 현재 condition 또	machine operational status

Type	Why	Example
	는 fact를 설명하는 데이터를 위해 설계되었습니다.	
`ACTION`	외부 system 또는 operator가 실행해야 하는 command를 기록하는 저빈도 command data를 위해 설계되었습니다.	start batch, stop machine

Path/Topic, Payload

각 UNS model은 path, topic, payload로 구성됩니다. 이들은 함께 business context를 담고 산업 데이터를 전달하는 **tree** 구조를 이룹니다.

- **Path/Topic**: path는 data ownership을 나타내고 topic은 message payload를 담습니다. 둘의 조합이 ***model address***입니다.
- **Payload**: topic이 보유한 message payload입니다. JSON **key-value** 형식이며 ***model data***를 정의합니다.

예를 들어 UNS의 model은 아래 tree처럼 보입니다.

탭: PHYSICAL DATA

```
Factory_A
└── Site_01
    └── SMT_Line_1
        └── Metric
            └── Machine_001
```

- **Path**: `Factory_A/Site_01/SMT_Line_1/Metric`
- **Topic**: `/Machine_001` .
- **Payload**

```
"temperature": 85,
```

```
"vibration": 2.8,
```

탭: BUSINESS DATA

```
Factory_A
└── ERP
    └── ProductionOrders
        └── State
            └── OrderList
```

- **Path**: `Factory_A/ERP/ProductionOrders/State`
- **Topic**: `/OrderList`.
- **Payload**

```
"orders": "0123569876",
"updated_at": "2026-7-13",
```

> **model을 만들려면**
> 데이터 모델 구축 방법을 확인하세요.

Broker

Tier0에서 broker는 data model을 기반으로 MQTT 또는 OpenAPI를 통해 데이터를 수신하고 전송하는 platform component입니다.

- **By MQTT**: model tree의 각 node 전체 path가 message payload를 전달하는 topic address로 동작합니다.
- **By OpenAPI**: HTTP request를 통해 data model에 대한 CRUD command를 수행합니다.

> **API 참조**
> API 세부 정보는 **API 참조**를 확인하세요.

Storage

Tier0는 PostgreSQL을 사용해 데이터를 저장합니다. data type마다 변경 빈도와 storage requirement가 다릅니다.

Data Type	Storage	Description
`METRIC`	Time-series database	temperature, pressure 같은 고빈도 measurement에 최적화되어 있습니다. 각 topic에는 기본 field `timestamp`와 `quality`가 포함됩니다.
`STATE`	PostgreSQL	machine 또는 system state를 유연한 `JSONB` format으로 저장합니다.
`ACTION`	PostgreSQL	command 또는 action payload를 유연한 `JSONB` format으로 저장합니다.

다음 단계

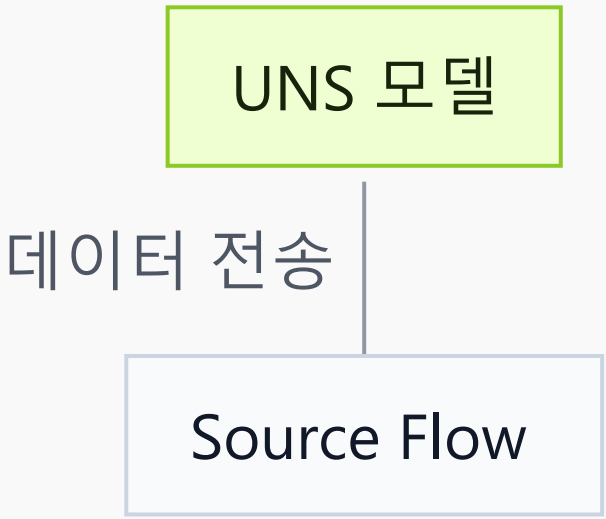
- **UNS**에 데이터 연결 - plant를 UNS로 model화하고 데이터를 model에 연결하는 방법입니다.
- **UNS** 데이터 작업 - MQTT와 API로 UNS model을 운영하는 방법입니다.

TIER0 사용

데이터 기반 준비

namespace 모델을 만들고 Source Flow로 데이터를 가져옵니다. 모든 에디션에서 사용할 수 있습니다.

먼저 **UNS**에서 모델을 만든 다음 **Source Flow**를 통해 데이터를 모델에 연결하여, 이후 사용을 위한 데이터 기반을 구성합니다.



데이터 모델 만들기

다음 기준에 따라 데이터 계층을 트리 맵으로 정의합니다.

- 데이터가 발생하는 물리적 위치(ISA-95 참고);
- 데이터가 따르는 비즈니스 로직.

모델을 수동으로 만들기

1. 모델 구조를 설계하고, topic을 추가하기 전에 path를 순서대로 추가합니다.
2. 데이터 특성에 따라 **Metric**, **State**, **Action** 중에서 **Topic Type**을 설정합니다.
3. **Mock Data**를 선택해 모델로 시뮬레이션 데이터를 보내고, **Enable History**를 켜서 데이터를 데이터베이스에 저장합니다.

   > ⓘ 참고
   > 시뮬레이션 데이터를 모델로 보내기 위한 flow가 **Flows**에 생성됩니다.

4. **Auto Parsing** 또는 **Pre-defined**로 올바른 데이터 타입의 필드를 topic에 추가합니다.
   - **Auto Parsing**: JSON 텍스트에서 필드를 일괄 자동 변환합니다.
   - **Pre-defined**: topic 필드를 하나씩 수동으로 설정합니다.

모델 가져오기

> 팁
> ChatGPT 같은 LLM을 사용해 모델 가져오기를 도울 수 있습니다.

1. **Import**의 템플릿 JSON을 AI에 보내고 비슷한 prompt를 사용합니다.

   ```
   템플릿을 기반으로 xx 공장의 xx 용도에 사용할 UNS 모델을 생성하고, xx 장비와 데이터 소스를 포함하세요.
   ```

2. 생성된 결과를 UNS로 가져옵니다.

데이터를 UNS에 연결하기

데이터 플로우 특징

- Node-RED 기반입니다.

- 모든 flow는 `mqtt out` 노드로 끝납니다. 이 노드는 MQTT client로 동작하여 UNS broker에 데이터를 publish합니다.
- UNS broker는 flow와 같은 이름의 `mqtt out` 및 `mqtt in` 노드에 내장되어 있습니다.
- UNS 모델을 topic으로 사용하면 데이터가 **UNS**의 해당 모델로 바로 들어갑니다.

단계

1. 데이터 소스 유형에 맞는 노드를 사용해 flow를 만들고, `mqtt out` 노드로 끝냅니다.

   > 🚀 필요한 노드가 없는 경우
   > 요구사항에 맞게 Node-RED 커뮤니티에서 노드를 설치할 수 있습니다. 커뮤니티 노드는 보통 `node-red-contrib`로 시작합니다.

2. `mqtt out` 노드의 **Server**가 UNS broker로 설정되어 있고, topic이 **UNS**의 모델인지 확인합니다.

Flow 버전 관리

데이터 flow의 버전을 저장하면 이전 버전으로 되돌리거나, 히스토리 버전을 기반으로 새 flow를 시작할 수 있습니다.

> ⚠ 주의
> Edge edition에서는 버전 롤백이 지원되지 않습니다.

다음

- 공장 데이터로 앱 구축하기 - UNS 데이터로 산업용 애플리케이션을 구축합니다.
- 데이터 분석 - Marimo Notebook과 Python으로 UNS 데이터를 분석합니다.

TIER0 사용

센터에서 엣지까지 데이터 관리

중앙 노드에서 연결된 엣지 노드의 데이터를 원활하게 관리합니다.

실제 공장에서는 네트워크 대역폭 사용량을 줄이고 데이터 보안을 강화하기 위해 엣지 디바이스에서 데이터를 수집하고 중앙 노드에서 통합 관리하는 경우가 많습니다. Tier0는 중앙 노드에서 엣지 노드의 데이터를 관리할 수 있는 Fleet Management 솔루션을 제공합니다.

엣지 노드를 중앙 노드에 연결하기

1. 먼저 중앙 노드에서 Fleet 노드를 생성합니다.

   > ⓘ 참고
   > 연결할 수 있는 엣지 노드 수는 라이선스에 따라 달라집니다.

2. 중앙 노드에서 생성된 토큰을 사용하여 각 엣지 노드를 연결합니다.

엣지와 센터 간 데이터 동기화

- UNS 데이터는 엣지 노드에서 중앙 노드로만 선택하여 동기화할 수 있습니다.
- 중앙 노드에서 엣지 노드에 사용자를 할당하여 노드를 관리할 수 있습니다.

다음

- 공장 데이터 모델링 — UNS에서 공장 데이터를 모델링하는 방법을 알아봅니다.
- 데이터 기반 준비 — namespace를 모델링하고 소스 데이터를 연결합니다.

TIER0 사용

공장 데이터 작업

MQTT와 API로 UNS 데이터를 조작하고 API 레퍼런스를 확인합니다.

MQTT와 API를 통해 UNS 데이터를 조작할 수 있습니다.

MQTT

MQTT client는 Tier0에서 생성한 credential로 UNS broker에 연결할 수 있습니다.

> ⓘ 참고
> **MQTTX**, **MQTT Explorer**, **Node-RED** 같은 MQTT client를 사용해 broker에 연결하고 message를 publish/subscribe할 수 있습니다.

MQTT Credential 생성

Tier0의 **Edge** 탭에서 새 credential을 만들고 기록해 둡니다.

> ⓘ **Tier0 Edge version** 예외
> Tier0 Edge version은 외부 연결에 credential이 필요하지 않습니다.

MQTT Client 연결

> 팁
> - UNS broker는 다음 주소로 TCP와 WSS를 모두 지원합니다.
>   - mqtt://host:1883
>   - mqtts://host:8883
>   - ws://host:8083/mqtt
>   - wss://host:8084/mqtt
> - 아래 연결 예시는 TLS 없는 TCP를 기준으로 합니다.

탭: MQTTX

1. MQTTX를 설치하고 엽니다.
2. Tier0 UNS broker 정보와 생성된 credential로 connection을 추가합니다.

```
{
  "Host": "mqtt.tier0.dev",
  "Port": 1883,
  "Client ID": "<generated client ID>",
  "Username": "<generated username>",
  "Password": "<generated password>"
}
```

탭: MQTT EXPLORER

1. MQTT Explorer를 설치하고 엽니다.
2. broker 정보와 생성된 credential로 새 connection을 추가합니다.

```
{
  "Protocol": "mqtt://",
  "Host": "mqtt.tier0.dev",
  "Port": 1883,
  "Username": "<generated username>",
  "Password": "<generated password>"
}
```

3. 연결하기 전에 **ADVANCED**에서 client ID를 credential ID로 변경합니다.

탭: NODE-RED

- **Event Flow** 사용 - Tier0 안의 Node-RED 기반 data processing module입니다.
  1. **Event Flow**에서 flow를 만들고 `mqtt in` node로 시작합니다.
  2. node에서 **Server**를 UNS broker(flow와 같은 이름)로 선택하고 **Topic**을 UNS model로 설정합니다.
- 독립 **Node-RED Instance** 사용
  1. Node-RED instance를 설치하고 엽니다.
  2. `mqtt in` node에서 아래 정보로 server를 추가합니다.

```
{
  {/* under Connection */}
  "Server": "mqtt://mqtt.tier0.dev",
  "Port": 1883,
  "Client ID": "<generated clientId>",
  {/* under Security */}
  "Username": "<generated username>",
  "Password": "<generated password>"
}
```

Connection 테스트

1. client status가 정상인지 확인합니다.
2. UNS model topic을 복사하고 client를 통해 message를 publish합니다.

API

REST API를 사용해 UNS 데이터를 조작합니다.

> 🚀 **API 사용 방법**
> OpenAPI base path는 `/openapi/v1`입니다. 예: `https://your-workspace.tier0.app/openapi/v1`. API key는 `x-api-key` header로 전달합니다.

API Key 가져오기

API keys는 **Settings** > **API Keys**에 있습니다. account type에 따라 허용되는 key type이 다릅니다.

- **Service Key**: **Admin**과 **Owner**만 만들 수 있습니다.
- **Personal Key**: 누구나 만들 수 있으며 permission scope는 account와 같습니다.

> ⚠ **Key permission vs Account permission**
> Account permission이 key permission보다 우선합니다. key permission을 **Full Access**로 설정해도 account가 접근할 수 없는 데이터는 이 key로도 접근할 수 없습니다.

예시

- `/uns/create`

  UNS path 또는 topic을 생성합니다. `namespace`를 사용해 node tree를 정의합니다.

탭: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/create`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
  },
  body: JSON.stringify({
    namespace: [
      {
        name: 'DemoFactory',
        type: 'PATH',
        children: [
          {
            name: 'Site_01',
            type: 'PATH',
            children: [
              {
                name: 'Production',
                type: 'PATH',
                children: [
                  {
                    name: 'Produced_Qty',
                    type: 'TOPIC',
                    topicType: 'METRIC',
                    displayName: 'Produced Quantity',
                    description: 'Produced quantity from the production
line.',
                    enableHistory: true,
                    fields: [
                      { name: 'value', type: 'DOUBLE', unit: 'pcs' },
                    ],
                  },
                ],
              },
            ],
          },
```

```
        ],
      },
    ],
  }),
});

const result = await response.json();
console.log(result.data.results);
```

탭: PYTHON

```
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]

response = requests.post(
    f"{base_url}/uns/create",
    headers={"x-api-key": api_key},
    json={
        "namespace": [
            {
                "name": "DemoFactory",
                "type": "PATH",
                "children": [
                    {
                        "name": "Site_01",
                        "type": "PATH",
                        "children": [
                            {
                                "name": "Production",
                                "type": "PATH",
                                "children": [
                                    {
                                        "name": "Produced_Qty",
                                        "type": "TOPIC",
                                        "topicType": "METRIC",
                                        "displayName": "Produced
Quantity",
                                        "description": "Produced quantity
from the production line.",
                                        "enableHistory": True,
                                        "fields": [
                                            {"name": "value", "type":
"DOUBLE", "unit": "pcs"},
                                        ],
```

```
                                    },
                                ],
                            },
                        ],
                    },
                ],
            },
        ],
    },
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"]["results"])
```

탭: CURL

```
curl -X POST "$TIER0_BASE_URL/uns/create" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
    "namespace": [
      {
        "name": "DemoFactory",
        "type": "PATH",
        "children": [
          {
            "name": "Site_01",
            "type": "PATH",
            "children": [
              {
                "name": "Production",
                "type": "PATH",
                "children": [
                  {
                    "name": "Produced_Qty",
                    "type": "TOPIC",
                    "topicType": "METRIC",
                    "displayName": "Produced Quantity",
                    "description": "Produced quantity from the production
line.",
                    "enableHistory": true,
                    "fields": [
                      { "name": "value", "type": "DOUBLE", "unit": "pcs"
}
                    ]
```

```
                  }
                ]
              }
            ]
          }
        ]
      }
    ]
  }'
```

- `/uns/read`

  하나 이상의 UNS topics에 대해 최신 값과 메타데이터를 읽습니다.

탭: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/read`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
  },
  body: JSON.stringify({
    topics: [
      'DemoFactory/Site_01/Production/Produced_Qty',
    ],
    include_leaf_value: true,
    include_metadata: true,
  }),
});

const result = await response.json();
console.log(result.data.results);
```

탭: PYTHON

```
import os
import requests
```

```
base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]

response = requests.post(
    f"{base_url}/uns/read",
    headers={"x-api-key": api_key},
    json={
        "topics": [
            "DemoFactory/Site_01/Production/Produced_Qty",
        ],
        "include_leaf_value": True,
        "include_metadata": True,
    },
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"]["results"])
```

탭: CURL

```
curl -X POST "$TIER0_BASE_URL/uns/read" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
    "topics": [
      "DemoFactory/Site_01/Production/Produced_Qty"
    ],
    "include_leaf_value": true,
    "include_metadata": true
  }'
```

- `/uns/update`

  `path`로 UNS node를 업데이트합니다. `updateMask`를 사용해 변경할 필드를 지정합니다.

탭: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
```

```
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/update`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
  },
  body: JSON.stringify({
    path: 'DemoFactory/Site_01/Production/Produced_Qty',
    displayName: 'Produced Quantity',
    description: 'Updated production output quantity.',
    updateMask: ['displayName', 'description'],
  }),
});

const result = await response.json();
console.log(result.data);
```

탭: PYTHON



```
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]

response = requests.post(
    f"{base_url}/uns/update",
    headers={"x-api-key": api_key},
    json={
        "path": "DemoFactory/Site_01/Production/Produced_Qty",
        "displayName": "Produced Quantity",
        "description": "Updated production output quantity.",
        "updateMask": ["displayName", "description"],
    },
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"])
```

탭: CURL

```
curl -X POST "$TIER0_BASE_URL/uns/update" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
    "path": "DemoFactory/Site_01/Production/Produced_Qty",
    "displayName": "Produced Quantity",
    "description": "Updated production output quantity.",
    "updateMask": ["displayName", "description"]
  }'
```

- `/uns/delete`

  하나 이상의 UNS models를 삭제합니다. `hard_delete: false`를 사용하면 models를 휴지통으로 이동합니다.

탭: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/delete`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
  },
  body: JSON.stringify({
    topics: [
      'DemoFactory/Site_01/Production/Produced_Qty',
    ],
    hard_delete: false,
  }),
});

const result = await response.json();
console.log(result.data);
```

탭: PYTHON

```
```

```python
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]

response = requests.post(
    f"{base_url}/uns/delete",
    headers={"x-api-key": api_key},
    json={
        "topics": [
            "DemoFactory/Site_01/Production/Produced_Qty",
        ],
        "hard_delete": False,
    },
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"])
```

탭: CURL

```bash
curl -X POST "$TIER0_BASE_URL/uns/delete" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
    "topics": [
      "DemoFactory/Site_01/Production/Produced_Qty"
    ],
    "hard_delete": false
  }'
```

API 참조

- Tier0 OpenAPI 참조

> ⚠ **Tier0 Edge의 API key 제한**
> Tier0 Edge에서 발급한 API key는 UNS 및 Flows API에서만 유효합니다.

다음 단계

- **UNS**에서 앱 구축 - UNS 데이터로 산업 애플리케이션을 구축합니다.
- 데이터 분석 - Marimo Notebook과 Python으로 UNS 데이터를 분석합니다.

TIER0 사용

앱 구축하기

AI 대화를 통해 공장 데이터, 접근 제어, 버전 관리, 배포 기능을 갖춘 산업용 앱을 구축합니다.

AI 대화를 통해 맞춤형 산업용 애플리케이션을 구축합니다. 애플리케이션은 공장 데이터를 사용할 수 있으며 역할 기반 접근 제어, 버전 관리, 배포 같은 기능을 기본으로 포함할 수 있습니다.

Tier0 앱 구축하기

애플리케이션을 구축하기 전에 project 를 생성합니다.

- Project 는 애플리케이션을 분류하고 애플리케이션 접근 권한을 관리하는 데 사용됩니다.
- 하나의 project 안에 있는 애플리케이션들은 복잡한 시스템의 관련 기능 모듈처럼 구성될 수 있습니다.
- 각 project 의 데이터는 다른 project 의 데이터와 독립적입니다.

Prompt 작성

Agent 가 요구 사항에 더 잘 맞는 애플리케이션을 생성할 수 있도록 prompt 에는 *최소한* 다음 내용을 포함해야 합니다.

- 애플리케이션 목적: 이 애플리케이션의 목적은 무엇인가요?

  ```
  공장 기술자를 위한 설비 유지보수 애플리케이션을 생성합니다.
  ```

- 애플리케이션 데이터: 애플리케이션은 어떤 데이터를 사용하며, 그 데이터는 어디에 있나요?

  탭: PLATFORM PLAN

  Prompt 에 UNS topic 을 명시하면 애플리케이션은 UNS 에서 데이터를 소비하고 UNS 로 데이터를 게시할 수 있습니다.

```
이 애플리케이션은 xxx UNS model 의 설비 상태, 알람 기록, 유지보수 이력을 사용하고, 업데이트를 xxx UNS topic 으로 게시해야 합니다.
```

탭: BUILDER PLAN

애플리케이션은 UNS 에 연결되지 않습니다. 애플리케이션 사용 중 생성되는 데이터는 데이터베이스에 저장됩니다.

- 애플리케이션 기능: 애플리케이션은 무엇을 해야 하나요?

```
이 애플리케이션은 사용자가 설비 상태를 모니터링하고, 유지보수 요청을 제출하며, 수리 진행 상황을 추적할 수 있어야 합니다.
```

⚠ **Builder** 를 사용하는 경우
Builder plan 은 현재 UNS 를 지원하지 않지만, 시뮬레이션 데이터는 사용할 수 있습니다. 더 구체적인 데이터 통합 요구 사항이 있으면 sales@freezonex.io 로 문의하세요.

애플리케이션을 더 빠르게 구축하기

- 일반적인 산업용 애플리케이션 템플릿은 **Library** 에 제공됩니다. 템플릿을 사용하고 그 위에서 기능을 수정하면 애플리케이션을 훨씬 빠르게 구축할 수 있습니다.
- 내장 **Builder Skills**를 사용해 애플리케이션 구축을 지원할 수 있습니다.
- 대화가 너무 길어지면 `/clear` 로 컨텍스트를 비우고, `/compact` 로 컨텍스트를 압축하세요.

접근 제어

역할 기반 접근 제어는 projects 와 애플리케이션 모두에 적용됩니다.

- Project access levels
  - **Owner**: project 에 대한 전체 접근 권한입니다.
  - **Can Edit**: project 삭제를 제외한 전체 접근 권한입니다.

    - **Can Launch**: **Launchpad** 에서 project 를 보고 허용된 애플리케이션만 사용할 수 있습니다.

- Application roles
    - 명시적으로 정의하지 않은 경우 애플리케이션의 비즈니스 로직에 따라 agent 가 자동 생성합니다. 책임 범위가 일치하면 다른 애플리케이션의 기존 역할을 재사용할 수 있습니다.
    - Project 멤버에게만 할당할 수 있습니다.
    - Prompt 에 역할이 필요 없다고 명시하면 애플리케이션은 모든 project 멤버에게 열립니다.

> 🚀 직접 역할을 지정하려는 경우
> Prompt 에 역할과 권한 범위를 명시하세요. Agent 는 이를 다른 애플리케이션에서 생성된 기존 역할과 비교합니다. 충돌이 있으면 agent 가 확인을 요청합니다.

> ⚠ 역할로 제어되는 애플리케이션에는 애플리케이션 역할 할당이 필수입니다
> **Launchpad** 에서 역할로 제어되는 애플리케이션에 접근하려면 애플리케이션 역할이 반드시 할당되어야 합니다. Workspace/project owner 라도 애플리케이션 역할이 없으면 해당 애플리케이션에 접근할 수 없습니다.

버전 관리

Tier0 에서 애플리케이션은 3가지 상태로 존재합니다.

- **Draft**: 애플리케이션의 최신 상태입니다. 배포하지 않으면 이 버전의 변경 사항은 다른 버전에 영향을 주지 않습니다.
- **Snapshot**: snapshot 생성 시점의 애플리케이션을 저장합니다. Snapshot 버전을 배포하면 현재 **Active** 버전이 교체됩니다.
- **Active**: 현재 온라인으로 실행 중인 애플리케이션 버전입니다.

> 🚀 버전 전환
> - Draft 를 배포하면 active 애플리케이션이 더 새로운 버전으로 업데이트됩니다.
> - Snapshot 을 배포하면 active 애플리케이션이 이전 버전으로 롤백됩니다.

외부로 내보내기

애플리케이션이 준비되면 **Preview** 페이지에서 외부 타사 환경 또는 Tier0 Enterprise 용으로 내보낼 수 있습니다.

- **Download Source Code**: 2차 개발을 위해 애플리케이션 소스 코드를 내보냅니다.
- **Export Bundle**: **Tier0 Enterprise** 에서 로컬로 가져오기 위한 전용 내보내기입니다.

> 🚀 **Snapshot 소스 코드 다운로드**
> Snapshot 을 저장한 후 다운로드하면 해당 snapshot 버전의 소스 코드를 받을 수 있습니다.

다음

- 데이터 분석 - Marimo Notebook 과 Python 으로 UNS 데이터를 분석합니다.
- 베스트 프랙티스: 산업 프로토콜 연결 - 일반적인 데이터 소스 연결 예시입니다.

TIER0 사용

Digital Twin 구축하기

실시간 UNS 데이터를 3D 모델에 바인딩하고 완전한 공장 장면으로 조합해 Digital Twin 시각화를 구성합니다.

Tier0에 연결된 공장 데이터는 실제와 유사한 동작과 환경을 갖춘 Digital Twin으로 시각화할 수 있습니다. 전체 과정은 데이터를 3D 모델에 매핑한 뒤 여러 모델을 조합해 완전한 공장 장면을 만드는 흐름입니다.

> ⓘ **2D 대시보드를 찾고 계신가요?**
> 이 페이지는 **3D Digital Twin 시각화** —— UNS 데이터를 3D 모델에 매핑하고 공장 장면을 구성하는 방법을 다룹니다. **2D 분석 대시보드** (차트, 테이블, KPI 패널)를 Notebook으로 구축하려면 Building Analytics Apps를 참고하세요.

데이터를 3D 모델에 매핑하기

1. **Model**에 모델을 업로드한 뒤, UNS 데이터로 실시간 동작을 제어할 노드를 선택합니다.
2. 같은 모델에 서로 다른 UNS 데이터를 가진 인스턴스를 추가해 여러 설비를 시뮬레이션합니다.
3. 데이터를 모델에 바인딩하고 **Preview**를 사용해 데이터 변화에 따라 모델이 이동하고 회전하는지 확인합니다.
   - **Position**: 모델이 선택한 좌표 방향으로 이동합니다.
   - **Rotation**: 모델이 좌표 원점을 기준으로 회전합니다.
4. **Data Tagging**을 사용해 정적 정보 표시용 태그를 추가합니다.

데이터 시각화하기

1. **Instance Library**에서 장면에 있어야 하는 인스턴스를 추가합니다.
2. 상단 도구를 사용해 모델을 이동, 확대/축소, 회전하여 장면에 맞추고, 시점과 좌표 위치를 조정합니다.
3. 오른쪽의 **Object**에서 모델 위치를 확인할 수 있으며, 여기에 바인딩된 UNS 데이터가 표시됩니다.

4 뷰포트와 조명은 환경 세부 설정에 사용합니다.

외부 공유

인스턴스만 모델 단위 또는 장면 단위로 공유할 수 있습니다.

- **Share Model**: 실시간 UNS 데이터가 표시되는 모델의 Tier0 플랫폼 링크를 생성합니다.
- **Share Scene**: 실시간 UNS 데이터와 환경 설정이 포함된 모델 및 장면의 Tier0 플랫폼 링크를 생성합니다.

TIER0 사용

데이터 분석

Tier0 Notebook에서 live namespace data를 쿼리하고 인터랙티브 분석 또는 머신러닝 앱을 구축합니다. Cloud와 Enterprise에서 사용할 수 있습니다.

Tier0에서는 UNS에 모델링되고 연결된 공장 데이터를 기반으로 Marimo Notebook을 사용해 **Python** 스크립트로 고급 데이터 분석을 수행합니다.

> 시작 전 확인
> **Enable History**가 켜진 UNS 데이터만 **Notebook**에서 사용 가능한 데이터 소스로 표시됩니다.

1. UNS 데이터에서 **VARIABLES** > **PostgreSQL** > **tenant_id** > **uns**를 선택합니다.
2. **GENERATE WITH AI**를 클릭한 뒤 **@**로 분석할 변수를 선택합니다. Notebook은 변수 검색을 돕기 위해 fuzzy match를 지원합니다.
3. 필요에 따라 분석을 계속하거나 생성 결과를 롤백합니다.

> **Notebook의 AI 지원**
> - Tier0 Cloud에서는 AI 지원을 바로 사용할 수 있습니다.
> - Tier0 Enterprise에서는 자체 AI key를 설정해야 합니다.

다음

- 공장 데이터로 앱 구축하기 - UNS 데이터로 산업 애플리케이션을 구축합니다.
- 공장 데이터 기반 - Unified Namespace의 데이터 모델링 기반을 이해합니다.

TIER0 사용

Agents로 Tier0 운영하기

AI agents가 Skills와 CLI 명령으로 Tier0를 운영하는 방법입니다.

Tier0는 AI agents가 플랫폼을 운영할 수 있도록 Skills와 CLI 명령을 제공합니다.

선호하는 **AI agent**로 **Tier0**를 운영하세요.

Claude Code Codex Gemini CLI Cursor GitHub Copilot

Skills와 CLI 설치

환경에 맞는 설치 방식을 선택하세요.

Node.js가 있는 경우

Node.js 16 이상을 사용할 수 있다면 `npx`로 CLI와 skills를 설치합니다.

```
npx -y @tier0/cli@latest install
```

이 명령은 전체 설치 프로그램을 실행하며 다음 두 가지를 수행합니다.

- `tier0` CLI binary를 `~/.tier0/bin/`에 설치합니다.
- 내장 Tier0 Skill baseline을 설치하고 Codex, Claude Code 등 감지된 agents에 동기화합니다.

Node.js가 없는 경우

1 `tier0` CLI binary를 설치합니다.

탭: CURL

```
curl -fsSL https://raw.githubusercontent.com/FREEZONEX/Tier0-cli/main/install.sh | bash
```

탭: IWR

```
iwr https://raw.githubusercontent.com/FREEZONEX/Tier0-cli/main/install.ps1 | iex
```

2 최신 Tier0 Skills를 다운로드하고 감지된 agents에 동기화합니다.

```
tier0 skills update
```

3 설치된 Skill 버전과 health를 확인합니다.

```
tier0 skills status
```

Skills 관리

- 설치된 Skill 버전과 health를 확인합니다.

```
tier0 skills status
```

- 누락되었거나 손상된 내장 Skill baseline을 복구합니다.

```
tier0 skills install
```

- 최신 Tier0 Skill로 업데이트합니다.

```
```

```
tier0 skills update
```

Skills와 CLI가 함께 작동하는 방식

Tier0 CLI는 실행 가능한 명령을 제공하고, Skills는 agents가 이 명령을 선택하고 안전하게 사용하도록 안내합니다.

> ⓘ 참고
> Tier0 Skills는 prompt에 의해 자동으로 트리거되어 agents에게 무엇을 해야 하는지, 어떤 명령을 사용해야 하는지 안내하도록 설계되었습니다.

예시

- Prompt: `Plant/Line1/Metric/Temperature`의 현재 온도를 확인합니다.
- Skill
  - `tier0-uns`
  - 현재 값은 전체 topic path에서 읽어야 한다고 agent에게 알려 주고 적절한 UNS read 작업을 선택합니다.
- CLI: `tier0 uns read Plant/Line1/Metric/Temperature --json`

Skills

Skill	목적
`tier0`	Tier0 작업을 적절한 skill로 라우팅하고 공통 설정, 인증, 안전 규칙을 제공합니다.
`tier0-uns`	Browse, read, write, history, search, create, update, delete, restore를 포함한 UNS 작업을 agents에게 안내합니다.
`tier0-flow`	SourceFlow와 EventFlow 관리를 agents에게 안내합니다. 여기에는 검사, 내보내기, 생성, 업데이트, 배포, 삭제가 포함됩니다.

Skill	목적
`tier0-mqtt`	MQTT 자격 증명 관리, publish, 범위가 제한된 실시간 subscribe를 agents에게 안내합니다.
`tier0-files`	Upload, download, 접근 URL 생성, delete를 포함한 Tier0 파일 작업을 agents에게 안내합니다.

> **agent가 skill repo를 읽게 하기**
> 자세한 내용은 agent가 **Tier0 Skill repo**를 읽게 하세요.

CLI 작업

운영 인스턴스 설정

> 참고
> Cloud는 기본적으로 `https://tier0.dev`를 사용하므로 base URL 설정이 필요하지 않습니다.

Private deployment는 로그인 전에 base URL을 설정해야 합니다. 설정 후 모든 명령은 해당 인스턴스에서 실행됩니다.

```
tier0 config --base-url https://your-tier0.example.com
tier0 login # interactive browser login
```

인증

탭: API KEY 있음

1. 운영 중인 Tier0 인스턴스에서 받은 API key를 설정합니다.

```
tier0 config --api-key <api-key>
```

2 API key의 identity와 permissions를 확인합니다.

```
tier0 auth whoami --json
```

3 `tier0` CLI로 다른 작업을 수행합니다.

탭: API KEY 없음

1 브라우저 인증 링크를 가져옵니다.

```
tier0 login --no-wait --json
```

2 반환된 `verification_url`에 접속해 브라우저에서 인증을 완료합니다.

3 반환된 `setup_code`로 로그인 결과를 확인합니다.

```
tier0 login --setup-code <setup_code>
```

> ⚠ 중요
> API key를 설정하거나 브라우저 로그인을 시작하기 전에 `config`를 실행해 운영 인스턴스를 설정하세요. 그렇지 않으면 CLI가 잘못된 인스턴스에서 동작할 수 있습니다.

작업 미리보기

`--dry-run`을 사용하면 지원되는 쓰기 작업을 Tier0에 요청하지 않고 검증하고 미리 볼 수 있습니다.

지원 작업

- UNS: `write/create/update/delete/restore`
- Flow: `create/update/delete/deploy`
- API: `tier0 api`

- MQTT: `auth create/auth delete/publish`
- Files: `delete`

예시

```
tier0 uns write --topic demo --value '{"value":1}' --dry-run
tier0 flow deploy --id 1 --flows-file flows.json --dry-run --json
tier0 api /openapi/v1/uns/write --body-file body.json --dry-run --json
tier0 mqtt auth create --name my-client --save my-client --dry-run --json
tier0 assets delete --file-path workspace/path/to/file.csv --dry-run --json
```

> ⓘ 참고
> `--json` 옵션은 요청을 JSON 형식으로 출력합니다.

> ⚠ **Request body는 유효한 JSON이어야 합니다**
> 유효한 JSON이 제공되지 않으면 CLI는 잘못된 JSON을 추측하거나 다시 작성하지 않습니다.

반환되는 미리보기

> ⓘ 참고
> dry-run 출력에는 headers와 API keys가 노출되지 않습니다.

```
{
  "ok": true,
  "dry_run": true,
  "data": {
    "api": [
      {"method": "POST", "url": "https://tier0.dev/openapi/v1/uns/write", "body":
{}}
    ]
  }
}
```

오류 처리

`--json`을 사용하면 명령 실패 시 오류 메시지와 코드를 포함한 JSON object가 반환됩니다.

```
{"ok":false,"error":
{"type":"validation","subtype":"invalid_argument","param":"--qos","message":"-
-qos must be 0, 1, or 2"}}
```

Skills와 CLI 제거

- 제거 명령

```
# Tier0 CLI를 제거하고 local configuration과 skills는 유지합니다.
npx @tier0/cli@latest uninstall

# Tier0 CLI를 제거하고 local configuration, credentials, cache를 삭제하지만 skills는
유지합니다.
npx @tier0/cli@latest uninstall --purge

# skills 제거
npx @tier0/cli@latest uninstall --remove-skills

# 위 항목을 모두 제거
npx @tier0/cli@latest uninstall --purge --remove-skills
```

- Global NPM 제거

```
npm uninstall -g @tier0/cli
```

- 수동 정리

```
rm -rf ~/.tier0/bin/tier0 ~/.tier0/skills
npx -y --package=skills -- skills remove tier0 -y -g
```

CLI Reference

Tier0 CLI Reference에는 모든 CLI 명령이 정리되어 있습니다.

다음

- 공장 데이터 작업 — MQTT와 API로 공장 데이터 모델을 운영하는 방법입니다.

베스트 프랙티스

공장 데이터 모델링

Demo UNS 설계: topic 폭증 방지와 payload 구조 베스트 프랙티스.

Demo UNS 구조

> 🚀 먼저 해야 할 일
> 데이터를 모델링하기 전에 다음 질문에 답하세요.
>
> - UNS에서 어떤 data를 consume할 것인가?
> - UNS로 어떤 data를 다시 publish할 것인가?

물리 데이터

물리적 위치가 있는 기계와 설비의 산업 데이터는 일반적으로 ISA-95 표준을 기반으로 모델링합니다.

Demo Model

탭: ISA-95 HIERARCHY

```
Enterprise (Level 4)
└── Site (Level 3/4)
    └── Area (Level 3)
        └── Work Center / Process Cell (Level 2/3)
            └── Equipment / Unit (Level 1/2)
                └── Control Module / Device (Level 0/1)
```

탭: TREE STRUCTURE

```
DemoFactory
└── Site_01
    └── Production
        └── Line_01
```

```
        └── WorkOrderExecution
            ├── Metric
            │   ├── Target_Qty
            │   ├── Produced_Qty
            │   ├── Defect_Qty
            │   └── Completion_Rate
            └── State
                ├── CurrentWorkOrder
                └── WorkOrderStatus
```

탭: JSON


```
{
  "namespace": [
    {
      "name": "DemoFactory",
      "alias": "DemoFactory_9e50bbcd833d",
      "type": "PATH",
      "children": [
        {
          "name": "Site_01",
          "alias": "DemoFactory__6d1fcad14e86",
          "type": "PATH",
          "children": [
            {
              "name": "Production",
              "alias": "__Site_01_Pro_91523d2ad304",
              "type": "PATH",
              "children": [
                {
                  "name": "Line_01",
                  "alias": "__Production__acd5841c1ada",
                  "type": "PATH",
                  "children": [
                    {
                      "name": "WorkOrderExecution",
                      "alias": "__Line_01_Wor_54904ff56e91",
                      "type": "PATH",
                      "children": [
                        {
                          "name": "Metric",
                          "alias": "metric_233fd84305c4",
                          "type": "PATH",
                          "children": [
                            {
                              "name": "Target_Qty",
                              "alias": "Target_Qty_8646b092e543",
```

```
                    "type": "TOPIC",
                    "topicType": "METRIC",
                    "description": "Target quantity for the current
Work Order.",
                    "fields": [
                      {
                        "name": "production_order_id",
                        "type": "STRING"
                      },
                      {
                        "name": "plan_id",
                        "type": "STRING"
                      },
                      {
                        "name": "plan_number",
                        "type": "STRING"
                      },
                      {
                        "name": "work_order_id",
                        "type": "STRING"
                      },
                      {
                        "name": "value",
                        "type": "DOUBLE"
                      },
                      {
                        "name": "unit",
                        "type": "STRING"
                      },
                      {
                        "name": "timestamp",
                        "type": "DATETIME"
                      }
                    ],
                    "enableHistory": "TRUE",
                    "mockData": "FALSE"
                  },
                  {
                    "name": "Produced_Qty",
                    "alias": "Produced_Qty_1f8cd3e11f77",
                    "type": "TOPIC",
                    "topicType": "METRIC",
                    "description": "Produced quantity for the
current Work Order.",
                    "fields": [
                      {
                        "name": "production_order_id",
                        "type": "STRING"
                      },
                      {
                        "name": "plan_id",
```

```
            "type": "STRING"
          },
          {
            "name": "plan_number",
            "type": "STRING"
          },
          {
            "name": "work_order_id",
            "type": "STRING"
          },
          {
            "name": "value",
            "type": "DOUBLE"
          },
          {
            "name": "unit",
            "type": "STRING"
          },
          {
            "name": "timestamp",
            "type": "DATETIME"
          }
        ],
        "enableHistory": "TRUE",
        "mockData": "FALSE"
      },
      {
        "name": "Defect_Qty",
        "alias": "Defect_Qty_b48fb3563ec1",
        "type": "TOPIC",
        "topicType": "METRIC",
        "description": "Defect quantity for the current
Work Order.",
        "fields": [
          {
            "name": "production_order_id",
            "type": "STRING"
          },
          {
            "name": "plan_id",
            "type": "STRING"
          },
          {
            "name": "plan_number",
            "type": "STRING"
          },
          {
            "name": "work_order_id",
            "type": "STRING"
          },
          {
```

```
            "name": "value",
            "type": "DOUBLE"
          },
          {
            "name": "unit",
            "type": "STRING"
          },
          {
            "name": "timestamp",
            "type": "DATETIME"
          }
        ],
        "enableHistory": "TRUE",
        "mockData": "FALSE"
      },
      {
        "name": "Completion_Rate",
        "alias": "Completion_R_7294e38e1530",
        "type": "TOPIC",
        "topicType": "METRIC",
        "description": "Completion percentage for the
current Work Order.",
        "fields": [
          {
            "name": "production_order_id",
            "type": "STRING"
          },
          {
            "name": "plan_id",
            "type": "STRING"
          },
          {
            "name": "plan_number",
            "type": "STRING"
          },
          {
            "name": "work_order_id",
            "type": "STRING"
          },
          {
            "name": "value",
            "type": "DOUBLE"
          },
          {
            "name": "unit",
            "type": "STRING"
          },
          {
            "name": "timestamp",
            "type": "DATETIME"
          }
```

```
                ],
                "enableHistory": "TRUE",
                "mockData": "FALSE"
              }
            ]
          },
          {
            "name": "State",
            "alias": "state_539a0f331fe0",
            "type": "PATH",
            "children": [
              {
                "name": "CurrentWorkOrder",
                "alias": "CurrentWorkO_a638320784dd",
                "type": "TOPIC",
                "topicType": "STATE",
                "description": "Current active Work Order on
Line_01.",
                "fields": [
                  {
                    "name": "production_order_id",
                    "type": "STRING"
                  },
                  {
                    "name": "plan_id",
                    "type": "STRING"
                  },
                  {
                    "name": "plan_number",
                    "type": "STRING"
                  },
                  {
                    "name": "work_order_id",
                    "type": "STRING"
                  },
                  {
                    "name": "target_line",
                    "type": "STRING"
                  },
                  {
                    "name": "workstation",
                    "type": "STRING"
                  },
                  {
                    "name": "product_code",
                    "type": "STRING"
                  },
                  {
                    "name": "target_qty",
                    "type": "DOUBLE"
                  },
```

```
                {
                  "name": "unit",
                  "type": "STRING"
                },
                {
                  "name": "started_at",
                  "type": "DATETIME"
                },
                {
                  "name": "updated_at",
                  "type": "DATETIME"
                }
              ],
              "enableHistory": "TRUE",
              "mockData": "FALSE"
            },
            {
              "name": "WorkOrderStatus",
              "alias": "WorkOrderSta_e24e3bf113d1",
              "type": "TOPIC",
              "topicType": "STATE",
              "description": "Execution status of the current
Work Order.",
              "fields": [
                {
                  "name": "production_order_id",
                  "type": "STRING"
                },
                {
                  "name": "plan_id",
                  "type": "STRING"
                },
                {
                  "name": "plan_number",
                  "type": "STRING"
                },
                {
                  "name": "work_order_id",
                  "type": "STRING"
                },
                {
                  "name": "status",
                  "type": "STRING"
                },
                {
                  "name": "reason",
                  "type": "STRING"
                },
                {
                  "name": "updated_at",
                  "type": "DATETIME"
```

```
                        }
                    ],
                    "enableHistory": "TRUE",
                    "mockData": "FALSE"
                }
            ]
        }
    ]
}
```

비즈니스 데이터

ERP의 생산 주문, 작업 지시 계획 같은 비즈니스 계획 및 프로세스 객체는 비즈니스 context에 맞게 모델링합니다.

> 🚀 비즈니스 데이터를 식별하는 방법
> 물리적 위치가 없는 데이터입니다.

비즈니스 데이터 모델링 **Guide**

```
Factory / Business Domain / Business Object / Topic Type / Topic
```

- **Business Domain**: data가 속한 business process 또는 system을 확인합니다.

```
ERP
MES
WMS
QMS
```

```
EAM
LIMS
Planning
Inventory
Maintenance
Quality
Logistics
...
```

- **Business Object**: business domain에서 관리되는 핵심 object입니다.

```
ProductionOrders
WorkOrderPlan
MaterialInventory
QualityInspection
MaintenanceOrders
PurchaseOrders
Shipments
...
```

- **Topic Type**: **Metric**, **Action**, **State** 중 data type을 선택합니다.

Topic Type	Used for	Examples
STATE	현재 상태 또는 현재 사실	work order status, order status, equipment status
METRIC	지속적으로 변하는 숫자 값	Temperature, speed, output, OEE, energy consumption
ACTION	외부 실행자를 위한 command	설비 시작, 설비 정지, alarm reset, work order dispatch 요청

Demo Model

탭: TREE STRUCTURE

```
DemoFactory
└── ERP
    ├── ProductionOrders
    │   └── State
    │       ├── UpsertProductionOrder
    │       └── OrderList
    └── WorkOrderPlan
        ├── Metric
        │   └── SplitCount
```

```
        └── State
            ├── PlanStatus
            └── WorkOrderList
```

탭: JSON


```
{
  "namespace": [
    {
      "name": "DemoFactory",
      "alias": "DemoFactory_f925c1ec5041",
      "type": "PATH",
      "children": [
        {
          "name": "ERP",
          "alias": "DemoFactory__e504805b1973",
          "type": "PATH",
          "children": [
            {
              "name": "ProductionOrders",
              "alias": "__ERP_Product_1e7984ac3a1d",
              "type": "PATH",
              "children": [
                {
                  "name": "State",
                  "alias": "state_c2616c8698fa",
                  "type": "PATH",
                  "children": [
                    {
                      "name": "UpsertProductionOrder",
                      "alias": "UpsertProduc_d5f166da35c6",
                      "type": "TOPIC",
                      "topicType": "STATE",
                      "description": "Event for creating or updating a
Production Order from WOM or ERP simulation.",
                      "fields": [
                        {
                          "name": "event_id",
                          "type": "STRING"
                        },
                        {
                          "name": "event_type",
                          "type": "STRING"
                        },
                        {
                          "name": "source",
                          "type": "STRING"
```

```
                    },
                    {
                      "name": "production_order_id",
                      "type": "STRING"
                    },
                    {
                      "name": "order_number",
                      "type": "STRING"
                    },
                    {
                      "name": "product_code",
                      "type": "STRING"
                    },
                    {
                      "name": "planned_qty",
                      "type": "DOUBLE"
                    },
                    {
                      "name": "unit",
                      "type": "STRING"
                    },
                    {
                      "name": "status",
                      "type": "STRING"
                    },
                    {
                      "name": "due_date",
                      "type": "DATETIME"
                    },
                    {
                      "name": "updated_at",
                      "type": "DATETIME"
                    }
                  ],
                  "enableHistory": "TRUE",
                  "mockData": "FALSE"
                },
                {
                  "name": "OrderList",
                  "alias": "OrderList_11fbca51a2bd",
                  "type": "TOPIC",
                  "topicType": "STATE",
                  "description": "Retained compatibility snapshot of
current Production Orders.",
                  "fields": [
                    {
                      "name": "orders",
                      "type": "STRING"
                    },
                    {
                      "name": "updated_at",
```

```
                        "type": "DATETIME"
                      }
                    ],
                    "enableHistory": "TRUE",
                    "mockData": "FALSE"
                  }
                ]
              }
            ]
          },
          {
            "name": "WorkOrderPlan",
            "alias": "__ERP_WorkOrd_19f04960daf5",
            "type": "PATH",
            "children": [
              {
                "name": "Metric",
                "alias": "metric_17258b809a4f",
                "type": "PATH",
                "children": [
                  {
                    "name": "SplitCount",
                    "alias": "SplitCount_032e86f5fbfc",
                    "type": "TOPIC",
                    "topicType": "METRIC",
                    "description": "Number of executable Work Orders
generated from a Work Order Plan.",
                    "fields": [
                      {
                        "name": "production_order_id",
                        "type": "STRING"
                      },
                      {
                        "name": "plan_id",
                        "type": "STRING"
                      },
                      {
                        "name": "plan_number",
                        "type": "STRING"
                      },
                      {
                        "name": "split_count",
                        "type": "INTEGER"
                      },
                      {
                        "name": "updated_at",
                        "type": "DATETIME"
                      }
                    ],
                    "enableHistory": "TRUE",
                    "mockData": "FALSE"
```

```
                    }
                  ]
                },
                {
                  "name": "State",
                  "alias": "state_884cee548bf7",
                  "type": "PATH",
                  "children": [
                    {
                      "name": "PlanStatus",
                      "alias": "PlanStatus_2e21c1327c49",
                      "type": "TOPIC",
                      "topicType": "STATE",
                      "description": "Current lifecycle status of a Work Order
Plan.",
                      "fields": [
                        {
                          "name": "production_order_id",
                          "type": "STRING"
                        },
                        {
                          "name": "plan_id",
                          "type": "STRING"
                        },
                        {
                          "name": "plan_number",
                          "type": "STRING"
                        },
                        {
                          "name": "status",
                          "type": "STRING"
                        },
                        {
                          "name": "updated_at",
                          "type": "DATETIME"
                        }
                      ],
                      "enableHistory": "TRUE",
                      "mockData": "FALSE"
                    },
                    {
                      "name": "WorkOrderList",
                      "alias": "WorkOrderLis_887b92f26fc2",
                      "type": "TOPIC",
                      "topicType": "STATE",
                      "description": "Split result list generated from a sent
Work Order Plan.",
                      "fields": [
                        {
                          "name": "production_order_id",
                          "type": "STRING"
```

```
                    },
                    {
                      "name": "plan_id",
                      "type": "STRING"
                    },
                    {
                      "name": "plan_number",
                      "type": "STRING"
                    },
                    {
                      "name": "split_count",
                      "type": "INTEGER"
                    },
                    {
                      "name": "lines",
                      "type": "STRING"
                    },
                    {
                      "name": "updated_at",
                      "type": "DATETIME"
                    }
                  ],
                  "enableHistory": "TRUE",
                  "mockData": "FALSE"
                }
              ]
            }
          ]
        }
      ]
    }
  ]
}
```

Topic 폭증을 피하는 방법

Tier0는 topic 폭증을 피하기 위해 `Metric`, `Action`, `State` 세 가지 고정 topic type을 사용합니다.

- **Use `Metric` for high-frequency real-time data**

  각 signal 세부 항목마다 namespace를 확장하지 말고, 고빈도 측정값과 telemetry는 `Metric` 아래에 둡니다.

탭: BAD

```
Factory_A
└── SMT_Line_1
    └── Machine_001
        ├── Temperature_Value
        ├── Temperature_Unit
        ├── Temperature_Status
        └── Speed_Value
```

탭: BETTER

```
Factory_A
└── SMT_Line_1
    └── Metric
        └── Machine_001
```

Model fields:

```
{
  "temperature": 80,
  "temperature_unit": "Celsius",
  "temperature_status": "Normal",
  "speed": 300
}
```

- **Use `State` for low-frequency status data**

  저빈도 상태, snapshot, 현재 비즈니스 상태는 `State` 아래에 둡니다. 작업, batch, 주문처럼 변하는 entity마다 새 topic을 만들지 않습니다.

탭: BAD

```
Factory_A
```

```
└── SMT_Line_1
    └── State
        ├── Job_001
        ├── Job_002
        └── Job_003
```

탭: BETTER

```
Factory_A
└── SMT_Line_1
    └── State
        └── Machine_001
```

Model fields:

```
{
  "current_job": "Job_003",
  "product": "Product_A"
}
```

- **Use `Action` for low-frequency action data**

  명령, 요청, operator action은 `Action` 아래에 둡니다. action 입력은 parameter별 topic이 아니라 topic의 field로 유지합니다.

탭: BAD

```
Factory_A
└── SMT_Line_1
    └── Machine_001
        ├── Start_Command
        ├── Target_Height
        └── Light_On
```

탭: BETTER

```
Factory_A
└── SMT_Line_1
    └── Action
        └── StartJob
```

Model fields:

```
{
  "target_height": 12.5,
  "light_on": true,
  "requested_by": "operator"
}
```

Payload 구조

topic payload를 설계할 때는 소비할 데이터와 다시 쓸 데이터를 모두 고려해야 합니다.

- **Action**

  Action payload에는 데이터가 Tier0 데이터베이스에 저장된 시간을 기록하는 `_timestamp`가 자동 포함됩니다.

  탭: START_JOB

  ```
  {
    "light_turn_on": true,
    "height_adjust": 12.5,
    "_timestamp": "2026-07-10T10:00:00Z"
  }
  ```

  탭: STOP_JOB

  ```
  {
  ```

```
  "water_pump_off": true,
  "window_close": true,
  "_timestamp": "2026-07-10T10:00:00Z"
}
```

- **Metric**

  Metric payload에는 데이터가 Tier0 데이터베이스에 저장된 시간을 기록하는 `_timestamp`와 데이터 품질을 나타내는 `_quality`가 자동 포함됩니다.

  탭: ERP / SPLITCOUNT

  ```
  {
    "production_order_id": "PO-20260710-001",
    "plan_id": "PLAN-001",
    "plan_number": "WOP-20260710-001",
    "split_count": 3,
    "updated_at": "2026-07-10T10:00:00Z",
    "_timestamp": "2026-07-10T10:00:00Z",
    "_quality": "Good"
  }
  ```

  탭: SITE_01 / TARGET_QTY

  ```
  {
    "production_order_id": "PO-20260710-001",
    "plan_id": "PLAN-001",
    "plan_number": "WOP-20260710-001",
    "work_order_id": "WO-001",
    "value": 1200,
    "unit": "pcs",
    "timestamp": "2026-07-10T10:00:00Z",
    "_timestamp": "2026-07-10T10:00:00Z",
    "_quality": "Good"
  }
  ```

- **State**

State payload에는 데이터가 Tier0 데이터베이스에 저장된 시간을 기록하는 `_timestamp` 가 자동 포함됩니다.

탭: ERP / UPSERTPRODUCTIONORDER

```
{
  "event_id": "EVT-001",
  "event_type": "UPSERT",
  "source": "ERP",
  "production_order_id": "PO-20260710-001",
  "order_number": "PO-001",
  "product_code": "SKU-1001",
  "planned_qty": 1200,
  "unit": "pcs",
  "status": "Released",
  "due_date": "2026-07-12T00:00:00Z",
  "updated_at": "2026-07-10T10:00:00Z",
  "_timestamp": "2026-07-10T10:00:00Z"
}
```

탭: SITE_01 / WORKORDERSTATUS

```
{
  "production_order_id": "PO-20260710-001",
  "plan_id": "PLAN-001",
  "plan_number": "WOP-20260710-001",
  "work_order_id": "WO-001",
  "status": "Running",
  "reason": "In production",
  "updated_at": "2026-07-10T10:00:00Z",
  "_timestamp": "2026-07-10T10:00:00Z"
}
```

베스트 프랙티스

산업 프로토콜 연결

Node-RED로 OPC UA, Modbus 및 일반 프로토콜 장치를 UNS에 연결합니다. 실제 flow JSON reference를 포함합니다.

이 섹션은 OPC UA, Modbus, API 같은 일반 데이터 유형을 예로 들어 **Source Flow**를 통해 데이터 소스를 **UNS**에 연결하는 과정을 설명합니다.

OPC UA

데이터 흐름 구성

1. Tier0에 로그인하고 **Flow**로 이동해 새 **Source Flow**를 만듭니다.
2. flow에 **inject**, **OPC UA Client**, **mqtt out** node를 추가합니다.
3. 순서대로 연결한 뒤 각 node를 double-click해 detail을 설정합니다.

- **inject**
  1. 왼쪽 아래의 **add**를 클릭해 property entry를 추가합니다.
  2. properties를 아래 항목으로 설정합니다.

```
{
  "msg.payload": "[multiple data tags]",
  "msg.topic": "multiple"
}
```

- **OPC UA Client**

```
"Endpoint": "<OPC UA server address and port>", //e.g.
opc.tcp://127.0.0.1:4850
"Action": "SUBSCRIBE",
"Interval": "<data collecting interval>" //e.g. 10 seconds
```

- **mqtt out**

```
"Server": "<embedded UNS broker>", //same name as the flow
"Topic": "<UNS model>",
```

4 flow를 deploy하고 trigger합니다.

Flow 참조

OPC UA flow JSON 보기


```
[
    {
        "id": "fn_mapper_01",
        "type": "function",
        "z": "b002a07d1163c3de",
        "name": "ISA95 Mapping (Air Compressor)",
        "func": "// OPC UA NodeId → ISA-95 MQTT Topic Mapping\n\nconst mapping
= {\n    \"ns=2;i=2\": {\n        topic:
\"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_
Air_System/Air_Compressor_Station/Air_Compressor_01/Metric/outlet_temperature\
",\n        field: \"temperature\"\n    },\n    \"ns=2;i=3\": {\n
topic:
\"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_
Air_System/Air_Compressor_Station/Air_Compressor_01/Metric/discharge_pressure\
",\n        field: \"pressure\"\n    },\n    \"ns=2;i=4\": {\n        topic:
\"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_
Air_System/Air_Compressor_Station/Air_Compressor_01/Metric/vibration_rms\",\n
field: \"vibration\"\n    }\n};\n\nlet value = msg.payload;\n\n// OPC UA
structure normalize\nif (value && typeof value === \"object\") {\n    if
(value.value?.value !== undefined) {\n        value = value.value.value;\n
} else if (value.value !== undefined) {\n        value = value.value;\n
}\n}\n\nconst m = mapping[msg.topic];\nif (!m) return null;\n\nmsg.topic =
m.topic;\n\nmsg.payload = {\n    [m.field]: Number(value)\n};\n\nreturn msg;",
        "outputs": 1,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 340,
        "y": 320,
        "wires": [
            [
                "mqtt_out_01"
```

```
            ]
        ]
    },
    {
        "id": "mqtt_out_01",
        "type": "mqtt out",
        "z": "b002a07d1163c3de",
        "name": "MQTT Air Compressor Metrics",
        "topic": "",
        "qos": "0",
        "retain": "false",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "broker-djnwpg00y370",
        "x": 570,
        "y": 200,
        "wires": []
    },
    {
        "id": "1f242deefa9fdba9",
        "type": "OpcUa-Client",
        "z": "b002a07d1163c3de",
        "endpoint": "298ef5d50e9dc411",
        "action": "subscribe",
        "deadbandtype": "a",
        "deadbandvalue": 1,
        "time": 10,
        "timeUnit": "s",
        "certificate": "n",
        "localfile": "",
        "localkeyfile": "",
        "securitymode": "None",
        "securitypolicy": "None",
        "useTransport": false,
        "maxChunkCount": 1,
        "maxMessageSize": 8192,
        "receiveBufferSize": 8192,
        "sendBufferSize": 8192,
        "setstatusandtime": false,
        "keepsessionalive": false,
        "name": "",
        "x": 220,
        "y": 220,
        "wires": [
            [
                "fn_mapper_01"
            ],
            [],
```

```
            []
        ]
    },
    {
        "id": "7be5f021ed1f10ef",
        "type": "inject",
        "z": "b002a07d1163c3de",
        "name": "",
        "props": [
            {
                "p": "payload"
            },
            {
                "p": "topic",
                "vt": "str"
            }
        ],
        "repeat": "",
        "crontab": "",
        "once": false,
        "onceDelay": 0.1,
        "topic": "multiple",
        "payload": "[{\"nodeId\":\"ns=2;i=2\",\"datatype\":\"Float\"},
{\"nodeId\":\"ns=2;i=3\",\"datatype\":\"Float\"},
{\"nodeId\":\"ns=2;i=4\",\"datatype\":\"Float\"}]",
        "payloadType": "json",
        "x": 110,
        "y": 160,
        "wires": [
            [
                "1f242deefa9fdba9"
            ]
        ]
    },
    {
        "id": "broker-djnwpg00y370",
        "type": "mqtt-broker",
        "z": "b002a07d1163c3de",
        "name": "emqx:1883",
        "broker": "emqx",
        "port": "1883",
        "clientid": "355424069147520",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
        "closeQos": "0",
```

```
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    },
    {
        "id": "298ef5d50e9dc411",
        "type": "OpcUa-Endpoint",
        "endpoint": "opc.tcp://172.31.151.237:4841",
        "secpol": "None",
        "secmode": "None",
        "none": true,
        "login": false,
        "usercert": false,
        "usercertificate": "",
        "userprivatekey": ""
    },
    {
        "id": "a003de8af8e62c75",
        "type": "global-config",
        "env": [],
        "modules": {
            "node-red-contrib-opcua": "0.2.339"
        }
    }
]
```


Modbus

데이터 흐름 구성

1. flow에 **modbus-read**와 **mqtt out** node를 추가합니다.
2. node를 double-click하고 필요한 정보를 설정합니다.
    - **modbus-read**
        - **Settings**
            - **FC**: Modbus action을 지정하는 function code입니다. 예: FC 3 = holding register 읽기.
            - **Address**: 읽기를 시작할 register address입니다(보통 0부터 시작).
            - **Quantity**: The number of consecutive registers to read.
            - **Poll Rate**: node가 Modbus device를 polling하는 주기입니다(예: 10초마다).

        - **Server**: 설정된 Modbus server(IP, port, protocol 등)에 대한 reference입니다.
    - **Server**
        - **Host**: Modbus server IP입니다.
        - **Port**: Modbus server port입니다.
- **mqtt out**

```
"Server": "<embedded UNS broker>", //same name as the flow
"Topic": "<UNS model>",
```

3 flow를 deploy하고 trigger합니다.

Flow 참조

<details>
<summary>Modbus flow JSON 보기</summary>

```
[
    {
        "id": "d3becb19e0d762f4",
        "type": "modbus-read",
        "z": "b002a07d1163c3de",
        "name": "status",
        "topic": "",
        "showStatusActivities": false,
        "logIOActivities": false,
        "showErrors": false,
        "showWarnings": true,
        "unitid": "1",
        "dataType": "HoldingRegister",
        "adr": "0",
        "quantity": "1",
        "rate": "10",
        "rateUnit": "s",
        "delayOnStart": false,
        "startDelayTime": "",
        "server": "c665b74952f6cf5a",
        "useIOFile": false,
        "ioFile": "",
        "useIOForPayload": false,
        "emptyMsgOnFail": false,
```

</details>

```
        "x": 130,
        "y": 500,
        "wires": [
            [
                "2987638967f632b4",
                "09b8538e09d0766a"
            ],
            []
        ]
    },
    {
        "id": "b068c119928c029a",
        "type": "mqtt out",
        "z": "b002a07d1163c3de",
        "name": "status",
        "topic":
"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_A
ir_System/Air_Compressor_Station/Air_Compressor_01/State/compressor_status",
        "qos": "",
        "retain": "",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "broker-djnwpg00y370",
        "x": 510,
        "y": 540,
        "wires": []
    },
    {
        "id": "2987638967f632b4",
        "type": "debug",
        "z": "b002a07d1163c3de",
        "name": "debug 3",
        "active": true,
        "tosidebar": true,
        "console": false,
        "tostatus": false,
        "complete": "false",
        "statusVal": "",
        "statusType": "auto",
        "x": 260,
        "y": 600,
        "wires": []
    },
    {
        "id": "09b8538e09d0766a",
        "type": "function",
        "z": "b002a07d1163c3de",
        "name": "function 1",
```

```
        "func": "const value = msg.payload;\n\n// 1️⃣ verify\nif (value === undefined || value === null) {\n    node.warn(\"Empty Modbus payload\");\n return null;\n}\n\n// 2️⃣ convert to number\nconst status_code = Number(value);\n\n// 3️⃣ mapping to status\nconst statusMap = {\n    0: \"maintenance\",\n    1: \"running\",\n    2: \"standby\",\n    3: \"debug\",\n    4: \"fault\"\n};\n\n// 4️⃣ filtering\nif (!(status_code in statusMap)) {\n    node.warn(\"Invalid status_code: \" + status_code);\n return null;\n}\n\n// 5️⃣ contextualized output\nmsg.payload = {\n status_code: status_code,\n    status_label: statusMap[status_code],\n _valid: true,\n    _ts: new Date().toISOString()\n};\n\n// 6️⃣ optional  set topic\nmsg.topic = \"v1/aircompressor01/state\";\n\nreturn msg;",
        "outputs": 1,
        "timeout": 0,
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 320,
        "y": 500,
        "wires": [
            [
                "b068c119928c029a"
            ]
        ]
    },
    {
        "id": "c665b74952f6cf5a",
        "type": "modbus-client",
        "name": "modbus",
        "clienttype": "tcp",
        "bufferCommands": true,
        "stateLogEnabled": false,
        "queueLogEnabled": false,
        "failureLogEnabled": true,
        "tcpHost": "172.31.151.237",
        "tcpPort": "5020",
        "tcpType": "DEFAULT",
        "serialPort": "/dev/ttyUSB",
        "serialType": "RTU-BUFFERD",
        "serialBaudrate": 9600,
        "serialDatabits": 8,
        "serialStopbits": 1,
        "serialParity": "none",
        "serialConnectionDelay": 100,
        "serialAsciiResponseStartDelimiter": "0x3A",
        "unit_id": 1,
        "commandDelay": 1,
        "clientTimeout": 1000,
        "reconnectOnTimeout": true,
        "reconnectTimeout": 2000,
        "parallelUnitIdsAllowed": true,
```

```
        "showErrors": false,
        "showWarnings": true,
        "showLogs": true
    },
    {
        "id": "broker-djnwpg00y370",
        "type": "mqtt-broker",
        "z": "b002a07d1163c3de",
        "name": "emqx:1883",
        "broker": "emqx",
        "port": "1883",
        "clientid": "355424069147520",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
        "closeQos": "0",
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    },
    {
        "id": "5cc203d0788c324d",
        "type": "global-config",
        "env": [],
        "modules": {
            "node-red-contrib-modbus": "5.43.0"
        }
    }
]
```

API

데이터 흐름 구성

1. flow에 **inject**, **http request**, **mqtt out** node를 추가합니다.
2. node를 double-click하고 필요한 정보를 설정합니다.
   - **http request**

```
"Method": "<API method>",
```

```
"URL": "<API URL>",
"Return": "a parsed JSON object",
"Headers": "<API headers>"
```

- **mqtt out**

```
"Server": "<embedded UNS broker>", //same name as the flow
"Topic": "<UNS model>",
```

3 flow를 deploy하고 trigger합니다.

Flow 참조

API flow JSON 보기

```
[
    {
        "id": "46273fcbb98afa11",
        "type": "inject",
        "z": "932355c6886aaf27",
        "name": "",
        "props": [
            {
                "p": "payload"
            },
            {
                "p": "topic",
                "vt": "str"
            }
        ],
        "repeat": "",
        "crontab": "",
        "once": false,
        "onceDelay": 0.1,
        "topic": "",
        "payload": "",
        "payloadType": "date",
        "x": 170,
        "y": 380,
        "wires": [
            [
                "3aca156076ad6dd1"
            ]
```

```
        ]
    },
    {
        "id": "3aca156076ad6dd1",
        "type": "http request",
        "z": "932355c6886aaf27",
        "name": "",
        "method": "GET",
        "ret": "obj",
        "paytoqs": "ignore",
        "url": "http://127.0.0.1:1880/api/wms/picking-zone-a/status",
        "tls": "",
        "persist": false,
        "proxy": "",
        "insecureHTTPParser": false,
        "authType": "",
        "senderr": false,
        "headers": [],
        "x": 380,
        "y": 380,
        "wires": [
            [
                "83fd65cf177c2ddf"
            ]
        ]
    },
    {
        "id": "83fd65cf177c2ddf",
        "type": "mqtt out",
        "z": "932355c6886aaf27",
        "name": "",
        "topic": "v1/Suzhou_Plant/Assembly-Area-1/Line-01/Station-
Screwing/Metric/torque",
        "qos": "",
        "retain": "",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "broker-djm245hxt9d9",
        "x": 780,
        "y": 440,
        "wires": []
    },
    {
        "id": "broker-djm245hxt9d9",
        "type": "mqtt-broker",
        "name": "emqx:1883",
        "broker": "emqx",
        "port": "1883",
```

```
        "clientid": "355039328257392",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
        "closeQos": "0",
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    }
]
```


🚀 함께 알아두면 좋은 내용

- **UNS**에 publish하기 전에 필요에 따라 **function** node로 data를 정리할 수 있습니다.
- **debug** node를 사용해 현장에서 data를 확인할 수 있습니다.
- node detail은 **Node-RED flows**를 확인하세요.

베스트 프랙티스

보안 및 복원력

Tier0의 ID, 데이터 접근, 배포 가용성 및 복구 제어에 대한 개요입니다.

Tier0는 다음 두 가지 측면에서 데이터를 보호합니다.

- 보안: 플랫폼, 애플리케이션 및 데이터에 접근할 수 있는 사용자를 제어합니다.
- 복원력: 장애 발생 후 서비스 연속성과 데이터 복구를 다룹니다.

에디션별 보안 범위

에디션	보안 지원 범위
Cloud	플랫폼 접근 제어와 인증이 필요한 MQTT 및 API 접근을 제공합니다.
Enterprise	플랫폼 및 데이터 접근 제어 외에도 프라이빗 배포를 위한 배포 가용성, 백업 및 복구를 제공합니다.
Edge(오픈 소스)	로컬 데이터 연결 및 모델링에 사용합니다. 내장 MQTT 브로커는 인증 없는 구독을 허용합니다.

각 에디션의 용도는 가장 적합한 제품 선택을 참조하세요.

> ⓘ **Tier0 Cloud** 보안 안내
> Cloud 프로덕션 서비스의 네트워크 경계, 테넌트 격리, 데이터 보호 및 보안 운영 조치는 **Tier0 Cloud** 네트워크 및 데이터 보안 안내를 참조하세요. 이 안내는 Tier0 Cloud에만 적용됩니다.

보안

ID 및 접근 제어

범위	Tier0 방식	상세 정보
플랫폼 접근	• Cloud는 Workspace 멤버 자격으로 접근을 제어합니다. • Enterprise는 사용자 계정으로 접근을 제어합니다.	사용자 관리
Agent 접근	Agent는 API 키로 인증하고 Tier0 CLI를 사용해 플랫폼에 접근하고 작업을 수행합니다.	CLI 인증
프로젝트 및 애플리케이션 접근	Tier0는 RBAC를 사용해 애플리케이션 접근을 제어합니다.	접근 제어

ⓘ 참고
Workspace 멤버 자격이 계정을 자동으로 생성하지는 않습니다. 사용자가 직접 계정을 등록하고 로그인해야 합니다. 계정 등록에는 2단계 확인이 필요합니다.

데이터 접근 제어

접근 방식	Tier0 방식	상세 정보
MQTT	• 클라이언트는 Tier0가 생성한 자격 증명으로 인증합니다. • TLS 및 보안 WebSocket이 전송 중인 데이터를 암호화합니다.	MQTT 자격 증명 및 연결
REST API	데이터 접근에는 Tier0가 생성한 키가 필요합니다.	API 레퍼런스

복원력

배포 가용성

Enterprise는 현재 중앙 노드에 대해 두 가지 배포 계획을 제공합니다.

계획	가용성 경계	상세 정보
표준 배포	• 데이터베이스 기본/대기 장애 조치를 지원합니다. • Enterprise 애플리케이션과 EMQX는 여전히 단일 장애 지점입니다.	표준 배포 계획
고가용성 배포	• Enterprise 활성/대기 노드를 추가하고 데이터베이스, 캐시, 객체 스토리지 및 관리 대상 런타임 파일의 장애 조치 동작을 정의합니다. • 무중단이나 데이터 손실 없음을 보장하지 않습니다.	고가용성 배포 계획

> ⓘ 참고
> 자세한 가용성 제한은 **SLA 및 고가용성 경계**를 참조하세요.

백업 및 복구

Enterprise는 데이터베이스 백업 및 복구를 제공합니다. 복구 후에는 배포 상태와 기본 기능을 검증하고 플랫폼에 로그인할 수 있는지 확인하세요.

- 백업은 **운영 Runbook**을 참조하세요.
- 복구 검증 요건은 **SLA 및 고가용성 경계**의 훈련 요건을 참조하세요.

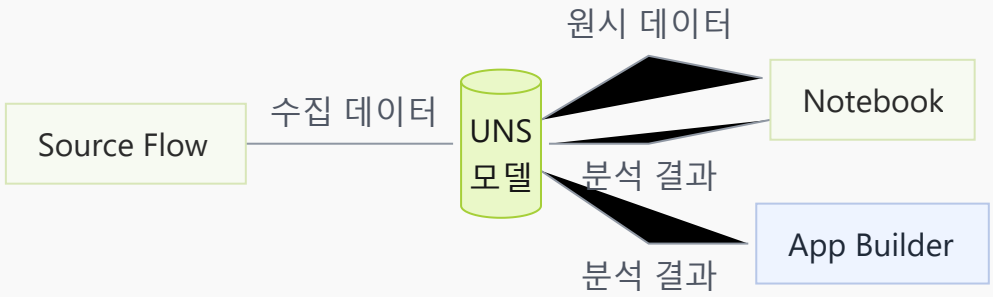
ⓘ 응답 시간
일반적인 응답 시간은 영업일 기준 3일입니다.

베스트 프랙티스

분석 앱 구축

Aramco Bowtie 사례(비식별 데이터)를 사용해 Notebook 기반 분석 앱을 구축합니다.

Tier0는 Python 기반 고급 데이터 분석에 Marimo Notebook을 사용합니다. 이 가이드는 Bowtie 애플리케이션을 예제로 사용합니다.



예제 배경

정유 공정의 부식은 여러 운전 요인의 영향을 받는 복잡한 현상입니다. 이 예제는 공정 데이터를 결합해 부식 발생 가능성을 추정하고 선제 정비를 지원하는 실시간 부식 위험 평가를 보여줍니다.

원천 데이터 가져오기

1 Tier0에서 **UNS**로 이동해 데이터 모델을 가져옵니다.

UNS 모델 JSON 보기

```
{
  "notes":
"type:PATH|TOPIC,topicType:STATE|ACTION|METRIC,schema.type:INTEGER|STRING
|FLOAT|DOUBLE|BOOLEAN|LONG|DATETIME",
  "namespace": [
    {
      "name": "Refinery",
```

```
      "type": "PATH",
      "children": [
        {
          "name": "CDU_Plant",
          "type": "PATH",
          "children": [
            {
              "name": "State",
              "type": "PATH",
              "topicType": "STATE",
              "children": [
                {
                  "name": "Corrosion_Risk",
                  "type": "TOPIC",
                  "topicType": "STATE",
                  "description": "Current corrosion risk state inferred
by Bayesian Network",
                  "enableHistory": "TRUE",
                  "schema": [
                    {
                      "name": "risk_state",
                      "type": "STRING"
                    },
                    {
                      "name": "previous_state",
                      "type": "STRING"
                    },
                    {
                      "name": "model_confidence",
                      "type": "FLOAT"
                    },
                    {
                      "name": "timestamp",
                      "type": "DATETIME"
                    }
                  ]
                }
              ]
            },
            {
              "name": "Metric",
              "type": "PATH",
              "topicType": "METRIC",
              "children": [
                {
                  "name": "Corrosion_Monitoring",
                  "type": "TOPIC",
                  "topicType": "METRIC",
                  "description": "Real-time process measurements for CDU
atmospheric overhead corrosion monitoring",
                  "enableHistory": "TRUE",
```

```
  "mockData": "FALSE",
  "schema": [
    {
      "name": "d103_chloride",
      "type": "FLOAT"
    },
    {
      "name": "d103_ph",
      "type": "FLOAT"
    },
    {
      "name": "wash_water_flow",
      "type": "FLOAT"
    },
    {
      "name": "desalter_salt_ptb",
      "type": "FLOAT"
    },
    {
      "name": "desalter_bsw",
      "type": "FLOAT"
    },
    {
      "name": "wash_water_rate",
      "type": "FLOAT"
    },
    {
      "name": "rrd_ph",
      "type": "FLOAT"
    },
    {
      "name": "rrd_chloride",
      "type": "FLOAT"
    },
    {
      "name": "rrd_total_iron",
      "type": "FLOAT"
    },
    {
      "name": "sap_evidence",
      "type": "STRING"
    },
    {
      "name": "injection_quill_fail",
      "type": "BOOLEAN"
    },
    {
      "name": "timestamp",
      "type": "DATETIME"
    }
  ]
```

```
                },
                {
                  "name": "Corrosion_Analysis",
                  "type": "TOPIC",
                  "topicType": "METRIC",
                  "description": "Posterior probabilities generated by
the Bayesian Network",
                  "enableHistory": "TRUE",
                  "mockData": "FALSE",
                  "schema": [
                    {
                      "name": "p_normal",
                      "type": "DOUBLE"
                    },
                    {
                      "name": "p_developing",
                      "type": "DOUBLE"
                    },
                    {
                      "name": "p_confirmed",
                      "type": "DOUBLE"
                    },
                    {
                      "name": "total_corrosion_risk",
                      "type": "DOUBLE"
                    },
                    {
                      "name": "lopc_probability",
                      "type": "DOUBLE"
                    },
                    {
                      "name": "shutdown_probability",
                      "type": "DOUBLE"
                    },
                    {
                      "name": "escalation_probability",
                      "type": "DOUBLE"
                    },
                    {
                      "name": "timestamp",
                      "type": "DATETIME"
                    }
                  ]
                }
              ]
            }
          ]
        }
      ]
    }
  ]
```

```
}
```

2 **Flows**로 이동해 **Source Flow**를 만들고 원시 데이터를 연결한 뒤 **UNS**에 게시합니다.

Source Flow JSON 보기

```
[
    {
        "id": "6173469ed5064ccf",
        "type": "tab",
        "label": "corrosion",
        "disabled": false,
        "info": ""
    },
    {
        "id": "ad20b016baf7247f",
        "type": "OpcUa-Client",
        "z": "6173469ed5064ccf",
        "endpoint": "66bd118c33eb12f8",
        "action": "subscribe",
        "deadbandtype": "a",
        "deadbandvalue": 1,
        "time": 10,
        "timeUnit": "s",
        "certificate": "n",
        "localfile": "",
        "localkeyfile": "",
        "securitymode": "None",
        "securitypolicy": "None",
        "useTransport": false,
        "maxChunkCount": 1,
        "maxMessageSize": 8192,
        "receiveBufferSize": 8192,
        "sendBufferSize": 8192,
        "setstatusandtime": false,
        "keepsessionalive": false,
        "name": "",
        "x": 280,
        "y": 440,
        "wires": [
            [
                "ac9195bb4863dbdd",
                "5a3d61b2ae40dcb6"
            ],
            [],
            []
        ]
```

```
    },
    {
        "id": "b6746ed066ba0e1a",
        "type": "inject",
        "z": "6173469ed5064ccf",
        "name": "",
        "props": [
            {
                "p": "payload"
            },
            {
                "p": "topic",
                "vt": "str"
            }
        ],
        "repeat": "",
        "crontab": "",
        "once": false,
        "onceDelay": 0.1,
        "topic": "multiple",
        "payload": "[   {     \"name\": \"D103_CHLORIDE\",
\"nodeId\": \"ns=2;i=3\",     \"datatype\": \"Float\"   },   {
\"name\": \"D103_PH\",     \"nodeId\": \"ns=2;i=4\",     \"datatype\":
\"Float\"   },   {     \"name\": \"WASH_WATER_FLOW\",     \"nodeId\":
\"ns=2;i=5\",     \"datatype\": \"Float\"   },   {     \"name\":
\"DESALTER_SALT_PTB\",     \"nodeId\": \"ns=2;i=6\",     \"datatype\":
\"Float\"   },   {     \"name\": \"DESALTER_BSW\",     \"nodeId\":
\"ns=2;i=7\",     \"datatype\": \"Float\"   },   {     \"name\":
\"WASH_WATER_RATE\",     \"nodeId\": \"ns=2;i=8\",     \"datatype\":
\"Float\"   },   {     \"name\": \"RRD_PH\",     \"nodeId\":
\"ns=2;i=9\",     \"datatype\": \"Float\"   },   {     \"name\":
\"RRD_CHLORIDE\",     \"nodeId\": \"ns=2;i=10\",     \"datatype\":
\"Float\"   },   {     \"name\": \"RRD_TOTAL_IRON\",     \"nodeId\":
\"ns=2;i=11\",     \"datatype\": \"Float\"   },   {     \"name\":
\"SAP_EVIDENCE\",     \"nodeId\": \"ns=2;i=12\",     \"datatype\":
\"UInt16\"   },   {     \"name\": \"INJECTION_QUILL_FAIL\",
\"nodeId\": \"ns=2;i=13\",     \"datatype\": \"Boolean\"   } ]",
        "payloadType": "json",
        "x": 90,
        "y": 440,
        "wires": [
            [
                "ad20b016baf7247f"
            ]
        ]
    },
    {
        "id": "ac9195bb4863dbdd",
        "type": "debug",
        "z": "6173469ed5064ccf",
        "name": "debug 2",
```

```
            "active": true,
            "tosidebar": true,
            "console": false,
            "tostatus": false,
            "complete": "false",
            "statusVal": "",
            "statusType": "auto",
            "x": 490,
            "y": 480,
            "wires": []
        },
        {
            "id": "5a3d61b2ae40dcb6",
            "type": "function",
            "z": "6173469ed5064ccf",
            "name": "function 1",
            "func": "const TOPIC =
\"Refinery/CDU_Plant/Metric/Corrosion_Monitoring\";\n\nconst nodeMap =
{\n    \"ns=2;i=3\": { field: \"d103_chloride\", type: \"float\" },\n
\"ns=2;i=4\": { field: \"d103_ph\", type: \"float\" },\n    \"ns=2;i=5\":
{ field: \"wash_water_flow\", type: \"float\" },\n    \"ns=2;i=6\": {
field: \"desalter_salt_ptb\", type: \"float\" },\n    \"ns=2;i=7\": {
field: \"desalter_bsw\", type: \"float\" },\n    \"ns=2;i=8\": { field:
\"wash_water_rate\", type: \"float\" },\n    \"ns=2;i=9\": { field:
\"rrd_ph\", type: \"float\" },\n    \"ns=2;i=10\": { field:
\"rrd_chloride\", type: \"float\" },\n    \"ns=2;i=11\": { field:
\"rrd_total_iron\", type: \"float\" },\n    \"ns=2;i=12\": { field:
\"sap_evidence\", type: \"sap\" },\n    \"ns=2;i=13\": { field:
\"injection_quill_fail\", type: \"boolean\" }\n};\n\nconst sapMap = {\n
0: \"none\",\n    1: \"work_order\",\n    2:
\"inspection_finding\"\n};\n\nfunction normalizeNodeId(v) {\n    if (!v)
return \"\";\n    if (typeof v === \"string\") return v;\n    if
(v.toString) return v.toString();\n    return \"\";\n}\n\nfunction
extractValue(payload) {\n    // OPC UA DataValue: payload.value.value\n
if (payload && payload.value && payload.value.value !== undefined) {\n
return payload.value.value;\n    }\n\n    // Some OPC UA nodes output
Variant directly.\n    if (payload && payload.value !== undefined) {\n
return payload.value;\n    }\n\n    return payload;\n}\n\nfunction
extractNodeId(msg) {\n    return normalizeNodeId(\n        msg.nodeId
||\n        msg.topic ||\n        msg.payload?.nodeId ||\n
msg.payload?.nodeid\n    );\n}\n\nconst nodeId =
extractNodeId(msg);\nconst spec = nodeMap[nodeId];\n\nif (!spec) {\n
node.warn(`Unknown OPC UA nodeId: ${nodeId}`);\n    return
null;\n}\n\nlet value = extractValue(msg.payload);\n\nif (spec.type ===
\"float\") {\n    value = Number(value);\n} else if (spec.type ===
\"boolean\") {\n    value = Boolean(value);\n} else if (spec.type ===
\"sap\") {\n    value = sapMap[Number(value)] || \"unknown\";\n}\n\nlet
latest = context.get(\"latest\") || {};\nlatest[spec.field] =
value;\nlatest.timestamp = new
Date().toISOString();\ncontext.set(\"latest\", latest);\n\n// Publish
only after all 11 fields have been received at least once.\nconst
```

```
requiredFields = Object.values(nodeMap).map(x => x.field);\nconst ready =
requiredFields.every(field => latest[field] !== undefined);\n\nif
(!ready) {\n    node.status({\n        fill: \"yellow\",\n        shape:
\"ring\",\n        text: `waiting ${Object.keys(latest).length - 1}/11`\n
});\n    return null;\n}\n\nmsg.topic = TOPIC;\nmsg.payload = {\n
d103_chloride: latest.d103_chloride,\n    d103_ph: latest.d103_ph,\n
wash_water_flow: latest.wash_water_flow,\n    desalter_salt_ptb:
latest.desalter_salt_ptb,\n    desalter_bsw: latest.desalter_bsw,\n
wash_water_rate: latest.wash_water_rate,\n    rrd_ph: latest.rrd_ph,\n
rrd_chloride: latest.rrd_chloride,\n    rrd_total_iron:
latest.rrd_total_iron,\n    sap_evidence: latest.sap_evidence,\n
injection_quill_fail: latest.injection_quill_fail,\n    timestamp:
latest.timestamp\n};\n\nnode.status({\n    fill: \"green\",\n    shape:
\"dot\",\n    text: \"published corrosion monitoring\"\n});\n\nreturn
msg;",
          "outputs": 1,
          "timeout": 0,
          "noerr": 0,
          "initialize": "",
          "finalize": "",
          "libs": [],
          "x": 370,
          "y": 580,
          "wires": [
              [
                  "aa15be33a84fb812",
                  "4fdd68dda3ee5c9e"
              ]
          ]
      },
      {
          "id": "aa15be33a84fb812",
          "type": "mqtt out",
          "z": "6173469ed5064ccf",
          "name": "",
          "topic": "",
          "qos": "",
          "retain": "",
          "respTopic": "",
          "contentType": "",
          "userProps": "",
          "correl": "",
          "expiry": "",
          "broker": "broker-djx75cwgiywq",
          "x": 550,
          "y": 580,
          "wires": []
      },
      {
          "id": "4fdd68dda3ee5c9e",
          "type": "debug",
```

```
        "z": "6173469ed5064ccf",
        "name": "debug 3",
        "active": true,
        "tosidebar": true,
        "console": false,
        "tostatus": false,
        "complete": "false",
        "statusVal": "",
        "statusType": "auto",
        "x": 450,
        "y": 680,
        "wires": []
    },
    {
        "id": "66bd118c33eb12f8",
        "type": "OpcUa-Endpoint",
        "endpoint": "opc.tcp://172.31.151.237:4850",
        "secpol": "None",
        "secmode": "None",
        "none": true,
        "login": false,
        "usercert": false,
        "usercertificate": "",
        "userprivatekey": ""
    },
    {
        "id": "broker-djx75cwgiywq",
        "type": "mqtt-broker",
        "z": "6173469ed5064ccf",
        "name": "corrosion",
        "broker": "emqx",
        "port": "1883",
        "clientid": "357356352615696",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
        "closeQos": "0",
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    },
    {
        "id": "c5201f1ad482b182",
        "type": "global-config",
        "env": [],
```

```
            "modules": {
                "node-red-contrib-opcua": "0.2.339"
            }
        }
]
```

Notebook에서 분석 앱 구축

1. Tier0에서 **Notebook**으로 이동해 새 notebook을 만듭니다.
2. notebook을 열고 **UNS** 데이터를 분석하기 위해 다음 cell을 추가합니다.
    - 셀 1 - 필요한 의존성을 가져오고 MQTT broker, topics, runtime 설정을 구성합니다.

코드 보기

```
@app.cell
def _():
    """Imports and runtime configuration."""
    import json
    import math
    import os
    import threading
    import time
    from datetime import datetime, timezone

    import marimo as mo
    import pandas as pd
    import paho.mqtt.client as mqtt

    MQTT_BROKER_HOST = os.getenv("TIER0_MQTT_HOST", "emqx")
    MQTT_BROKER_PORT = int(os.getenv("TIER0_MQTT_PORT", "1883"))
    MQTT_USERNAME = os.getenv("TIER0_MQTT_USERNAME",
"357890055321856")
    MQTT_PASSWORD = os.getenv("TIER0_MQTT_PASSWORD",
"caa6b34242002a3dff4948dd73ce8a")
    MQTT_CLIENT_ID = os.getenv(
        "TIER0_MQTT_CLIENT_ID",
        "357890055321856_bn",
    )

    SOURCE_TOPIC = "Refinery/CDU_Plant/Metric/Corrosion_Monitoring"
    ANALYSIS_TOPIC = "Refinery/CDU_Plant/Metric/Corrosion_Analysis"
    RISK_TOPIC = "Refinery/CDU_Plant/State/Corrosion_Risk"
```

```
    SOURCE_QOS = int(os.getenv("TIER0_SOURCE_QOS", "1"))
    OUTPUT_QOS = int(os.getenv("TIER0_OUTPUT_QOS", "1"))
    AUTOSTART = os.getenv("TIER0_BN_AUTOSTART", "true").lower() in {
        "1", "true", "yes", "on"
    }
    ROOT_CONFIDENCE = float(os.getenv("TIER0_BN_ROOT_CONFIDENCE",
"0.85"))

    def utc_now_iso() -> str:
        return (
            datetime.now(timezone.utc)
            .isoformat(timespec="milliseconds")
            .replace("+00:00", "Z")
        )

    mo.md(
        """
        # CDU Corrosion Bayesian Analysis

        This notebook subscribes to a complete corrosion-monitoring
snapshot,
        performs one Bayesian inference for each new source message,
and publishes
        the analysis results back to the UNS.
        """
    )
    return (
        ANALYSIS_TOPIC,
        AUTOSTART,
        MQTT_BROKER_HOST,
        MQTT_BROKER_PORT,
        MQTT_CLIENT_ID,
        MQTT_PASSWORD,
        MQTT_USERNAME,
        OUTPUT_QOS,
        RISK_TOPIC,
        ROOT_CONFIDENCE,
        SOURCE_QOS,
        SOURCE_TOPIC,
        datetime,
        json,
        math,
        mo,
        mqtt,
        os,
        pd,
        threading,
        time,
        timezone,
        utc_now_iso,
```

```
    )
```

> 🚀 **UNS broker 외부 연결**
> MQTT Clients 연결을 참고하세요.

- 셀 2 - Bayesian Network topology, 상태 공간, CPT generator, 확률 규칙을 정의합니다.

코드 보기

```
@app.cell
def _(math, pd):
    """Static Bayesian-network definition and CPT generators."""
    import itertools

    from pgmpy.factors.discrete import TabularCPD
    from pgmpy.inference import VariableElimination
    from pgmpy.models import DiscreteBayesianNetwork

    STATE_SPACES = {
        "STATE_3": ["LOW", "NORMAL", "HIGH"],
        "STATE_YN": ["NO", "YES"],
        "PB2_3STATE": ["FAIL", "IL_OK", "SL_OK"],
        "TOP_EVENT_STATE": ["NORMAL", "DEVELOPING", "CONFIRMED"],
    }

    NODE_STATE_SPACE = {
        "T1_HighChlorideLoad": "STATE_3",
        "T2_ChemProtectionFail": "STATE_3",
        "T3_WashWaterLow": "STATE_3",
        "PB1_Desalter_OK": "STATE_YN",
        "PB2_WashQty_State": "PB2_3STATE",
        "PB3_Neutralization_OK": "STATE_YN",
        "MB1_ESD_OK": "STATE_YN",
        "MB2_TSV_OK": "STATE_YN",
        "TE_CorrosionState": "TOP_EVENT_STATE",
        "EV_TOTIRON_High": "STATE_YN",
        "EV_SAP_Positive": "STATE_YN",
        "EV_QuillFailure": "STATE_YN",
        "C1_LOPC": "STATE_YN",
        "C2_UnplannedShutdown": "STATE_YN",
        "C3_Escalation": "STATE_YN",
    }

    EDGES = [
```

```
        ("T1_HighChlorideLoad", "TE_CorrosionState"),
        ("T2_ChemProtectionFail", "TE_CorrosionState"),
        ("T3_WashWaterLow", "TE_CorrosionState"),
        ("PB1_Desalter_OK", "TE_CorrosionState"),
        ("PB2_WashQty_State", "TE_CorrosionState"),
        ("PB3_Neutralization_OK", "TE_CorrosionState"),
        ("TE_CorrosionState", "EV_TOTIRON_High"),
        ("TE_CorrosionState", "EV_SAP_Positive"),
        ("TE_CorrosionState", "EV_QuillFailure"),
        ("TE_CorrosionState", "C1_LOPC"),
        ("MB1_ESD_OK", "C1_LOPC"),
        ("MB2_TSV_OK", "C1_LOPC"),
        ("TE_CorrosionState", "C2_UnplannedShutdown"),
        ("MB1_ESD_OK", "C2_UnplannedShutdown"),
        ("C1_LOPC", "C3_Escalation"),
        ("MB1_ESD_OK", "C3_Escalation"),
        ("MB2_TSV_OK", "C3_Escalation"),
    ]

    def states_for(node_id: str) -> list[str]:
        return STATE_SPACES[NODE_STATE_SPACE[node_id]]

    def softmax(values: list[float]) -> list[float]:
        maximum = max(values)
        exps = [math.exp(value - maximum) for value in values]
        total = sum(exps)
        return [value / total for value in exps]

    THREAT_SCORE = {"LOW": 0.0, "NORMAL": 1.0, "HIGH": 2.0}
    PB1_OK_BONUS = {"NO": 0.8, "YES": -0.6}
    PB3_OK_BONUS = {"NO": 1.0, "YES": -0.7}
    PB2_STATE_BONUS = {"FAIL": 1.2, "IL_OK": 0.3, "SL_OK": -0.6}

    def top_event_distribution(t1, t2, t3, pb1, pb2, pb3) ->
dict[str, float]:
        score = (
            THREAT_SCORE[t1]
            + THREAT_SCORE[t2]
            + THREAT_SCORE[t3]
            + PB1_OK_BONUS[pb1]
            + PB2_STATE_BONUS[pb2]
            + PB3_OK_BONUS[pb3]
        )
        probabilities = softmax(
            [
                2.2 - score,
                -0.2 + 0.6 * score,
                -1.2 + 0.8 * score,
            ]
        )
```

```
        return dict(zip(STATE_SPACES["TOP_EVENT_STATE"],
probabilities))

    TE_PARENTS = [
        "T1_HighChlorideLoad",
        "T2_ChemProtectionFail",
        "T3_WashWaterLow",
        "PB1_Desalter_OK",
        "PB2_WashQty_State",
        "PB3_Neutralization_OK",
    ]

    TE_PARENT_SPACES = [states_for(parent) for parent in TE_PARENTS]
    te_rows = []
    for combination in itertools.product(*TE_PARENT_SPACES):
        distribution = top_event_distribution(*combination)
        te_rows.append(
            [
                *combination,
                distribution["NORMAL"],
                distribution["DEVELOPING"],
                distribution["CONFIRMED"],
            ]
        )
    TE_CPT_DF = pd.DataFrame(
        te_rows,
        columns=TE_PARENTS
        + ["P_NORMAL", "P_DEVELOPING", "P_CONFIRMED"],
    )

    def evidence_yes_probability(te_state: str, kind: str) -> float:
        mappings = {
            "TOTIRON": {"NORMAL": 0.15, "DEVELOPING": 0.65,
"CONFIRMED": 0.85},
            "SAP": {"NORMAL": 0.08, "DEVELOPING": 0.35, "CONFIRMED":
0.75},
            "QUILL": {"NORMAL": 0.03, "DEVELOPING": 0.15,
"CONFIRMED": 0.55},
        }
        return mappings[kind][te_state]

    def lopc_yes_probability(te: str, mb1: str, mb2: str) -> float:
        base = {"NORMAL": 0.02, "DEVELOPING": 0.08, "CONFIRMED":
0.25}[te]
        return min(0.98, base + (0.10 if mb1 == "NO" else 0) + (0.18
if mb2 == "NO" else 0))

    def shutdown_yes_probability(te: str, mb1: str) -> float:
        base = {"NORMAL": 0.05, "DEVELOPING": 0.35, "CONFIRMED":
0.60}[te]
        return min(0.98, base + (0.10 if mb1 == "NO" else 0))
```

```
    def escalation_yes_probability(lopc: str, mb1: str, mb2: str) ->
float:
        if lopc == "NO":
            return 0.01
        return min(0.98, 0.10 + (0.18 if mb1 == "NO" else 0) + (0.35
if mb2 == "NO" else 0))

    def peaked_prior(node_id: str, center_state: str, confidence:
float) -> list[float]:
        states = states_for(node_id)
        confidence = min(max(float(confidence), 0.0), 1.0)
        if center_state not in states:
            return [1.0 / len(states)] * len(states)
        if len(states) == 1:
            return [1.0]
        remainder = (1.0 - confidence) / (len(states) - 1)
        probabilities = [remainder] * len(states)
        probabilities[states.index(center_state)] = confidence
        return probabilities

    def binary_cpd(child: str, parent: str, yes_probability_fn):
        parent_states = states_for(parent)
        yes_values = [float(yes_probability_fn(state)) for state in
parent_states]
        return TabularCPD(
            variable=child,
            variable_card=2,
            values=[[1.0 - value for value in yes_values],
yes_values],
            evidence=[parent],
            evidence_card=[len(parent_states)],
            state_names={child: states_for(child), parent:
parent_states},
        )

    def binary_multi_cpd(child: str, parents: list[str],
yes_probability_fn):
        parent_spaces = [states_for(parent) for parent in parents]
        combinations = list(itertools.product(*parent_spaces))
        yes_values = [float(yes_probability_fn(*combination)) for
combination in combinations]
        return TabularCPD(
            variable=child,
            variable_card=2,
            values=[[1.0 - value for value in yes_values],
yes_values],
            evidence=parents,
            evidence_card=[len(space) for space in parent_spaces],
            state_names={
                child: states_for(child),
```

```
                **{parent: states_for(parent) for parent in
parents},
            },
        )

    return (
        DiscreteBayesianNetwork,
        EDGES,
        NODE_STATE_SPACE,
        STATE_SPACES,
        TE_CPT_DF,
        TE_PARENTS,
        TabularCPD,
        VariableElimination,
        binary_cpd,
        binary_multi_cpd,
        escalation_yes_probability,
        evidence_yes_probability,
        itertools,
        lopc_yes_probability,
        peaked_prior,
        shutdown_yes_probability,
        states_for,
    )
```

- 셀 3 - 수신 MQTT payload를 검증하고 연속 process 값을 Bayesian Network 상태로 변환합니다.

코드 보기

```
@app.cell
def _(ROOT_CONFIDENCE, utc_now_iso):
    """Source validation, discretization, and one-snapshot
analysis."""
    import hashlib

    REQUIRED_SOURCE_FIELDS = [
        "d103_chloride",
        "d103_ph",
        "wash_water_flow",
        "desalter_salt_ptb",
        "desalter_bsw",
        "wash_water_rate",
        "rrd_ph",
        "rrd_chloride",
        "rrd_total_iron",
        "sap_evidence",
```

```
        "injection_quill_fail",
        "timestamp",
    ]

    def parse_boolean(value) -> bool:
        if isinstance(value, bool):
            return value
        if isinstance(value, (int, float)):
            return bool(value)
        if isinstance(value, str):
            normalized = value.strip().lower()
            if normalized in {"true", "1", "yes", "y", "on"}:
                return True
            if normalized in {"false", "0", "no", "n", "off"}:
                return False
        raise ValueError(f"invalid boolean value: {value!r}")

    def normalize_sap(value) -> str:
        normalized = str(value).strip().lower()
        if normalized in {"0", "none", "normal", "", "nan"}:
            return "None"
        if normalized in {"1", "work_order", "work order", "minor",
"warning"}:
            return "Minor corrosion observed"
        if normalized in {
            "2",
            "inspection_finding",
            "inspection finding",
            "confirmed",
            "abnormal",
        }:
            return "Confirmed corrosion / thickness loss"
        raise ValueError(f"unsupported sap_evidence value:
{value!r}")

    def normalize_source_payload(payload: dict) -> dict:
        if not isinstance(payload, dict):
            raise TypeError("source MQTT payload must be a JSON
object")

        lower = {str(key).lower(): value for key, value in
payload.items()}
        missing = [field for field in REQUIRED_SOURCE_FIELDS if
field not in lower]
        if missing:
            raise ValueError(f"source payload is missing fields:
{missing}")

        normalized = {
            "d103_chloride": float(lower["d103_chloride"]),
            "d103_ph": float(lower["d103_ph"]),
```

```
            "wash_water_flow": float(lower["wash_water_flow"]),
            "desalter_salt_ptb": float(lower["desalter_salt_ptb"]),
            "desalter_bsw": float(lower["desalter_bsw"]),
            "wash_water_rate": float(lower["wash_water_rate"]),
            "rrd_ph": float(lower["rrd_ph"]),
            "rrd_chloride": float(lower["rrd_chloride"]),
            "rrd_total_iron": float(lower["rrd_total_iron"]),
            "sap_evidence": normalize_sap(lower["sap_evidence"]),
            "injection_quill_fail":
parse_boolean(lower["injection_quill_fail"]),
            "timestamp": str(lower["timestamp"]).strip(),
        }

        numeric_ranges = {
            "d103_chloride": (0.0, 1000.0),
            "d103_ph": (0.0, 14.0),
            "wash_water_flow": (0.0, 10000.0),
            "desalter_salt_ptb": (0.0, 1000.0),
            "desalter_bsw": (0.0, 100.0),
            "wash_water_rate": (0.0, 100.0),
            "rrd_ph": (0.0, 14.0),
            "rrd_chloride": (0.0, 10000.0),
            "rrd_total_iron": (0.0, 10000.0),
        }
        for field, (minimum, maximum) in numeric_ranges.items():
            value = normalized[field]
            if not minimum <= value <= maximum:
                raise ValueError(f"{field} out of accepted range:
{value}")

        if not normalized["timestamp"]:
            raise ValueError("timestamp cannot be empty")
        return normalized

    def source_message_id(payload: dict) -> str:
        # Timestamp is the primary event key. Hash protects against
two different
        # snapshots accidentally sharing the same timestamp.
        ordered = "|".join(str(payload[field]) for field in
REQUIRED_SOURCE_FIELDS)
        return hashlib.sha256(ordered.encode("utf-8")).hexdigest()

    def discretize_source(payload: dict) -> tuple[dict, dict]:
        t1 = (
            "LOW"
            if payload["d103_chloride"] < 5.0
            else "NORMAL"
            if payload["d103_chloride"] <= 20.0
            else "HIGH"
        )
```

```
        chemical_failure_score = int(payload["d103_ph"] < 5.5) +
int(
            payload["d103_chloride"] > 10.0
        )
        t2 = (
            "HIGH"
            if chemical_failure_score == 2
            else "NORMAL"
            if chemical_failure_score == 1
            else "LOW"
        )

        t3 = (
            "HIGH"
            if payload["wash_water_flow"] < 6.0
            else "NORMAL"
            if payload["wash_water_flow"] <= 10.0
            else "LOW"
        )

        pb1 = (
            "YES"
            if payload["desalter_salt_ptb"] <= 1.0
            and payload["desalter_bsw"] <= 0.2
            else "NO"
        )

        pb2 = (
            "FAIL"
            if payload["wash_water_rate"] < 4.0
            else "IL_OK"
            if payload["wash_water_rate"] < 6.0
            else "SL_OK"
        )

        pb3 = (
            "YES"
            if payload["rrd_ph"] >= 5.5 and payload["rrd_chloride"]
<= 10.0
            else "NO"
        )

        root_centers = {
            "T1_HighChlorideLoad": t1,
            "T2_ChemProtectionFail": t2,
            "T3_WashWaterLow": t3,
            "PB1_Desalter_OK": pb1,
            "PB2_WashQty_State": pb2,
            "PB3_Neutralization_OK": pb3,
            # The source model currently has no MB fields. Keep both
mitigations
```

```
            # available by default, matching the previous notebook
UI defaults.
            "MB1_ESD_OK": "YES",
            "MB2_TSV_OK": "YES",
        }

        evidence = {
            "EV_TOTIRON_High": "YES" if payload["rrd_total_iron"] >=
10.0 else "NO",
            "EV_SAP_Positive": "YES" if payload["sap_evidence"] !=
"None" else "NO",
            "EV_QuillFailure": "YES" if
payload["injection_quill_fail"] else "NO",
        }
        return root_centers, evidence

    return (
        REQUIRED_SOURCE_FIELDS,
        discretize_source,
        normalize_source_payload,
        parse_boolean,
        source_message_id,
    )
```

- 셀 4 - 현재 snapshot에 대한 Bayesian Network를 구축하고 추론을 실행한 뒤 output payload를 생성합니다.

코드 보기

```
@app.cell
def _(
    DiscreteBayesianNetwork,
    EDGES,
    ROOT_CONFIDENCE,
    STATE_SPACES,
    TE_CPT_DF,
    TE_PARENTS,
    TabularCPD,
    VariableElimination,
    binary_cpd,
    binary_multi_cpd,
    discretize_source,
    escalation_yes_probability,
    evidence_yes_probability,
    itertools,
    lopc_yes_probability,
    peaked_prior,
```

```
    shutdown_yes_probability,
    states_for,
    utc_now_iso,
):
    """Build a fresh BN for the current snapshot and execute
inference."""

    def factor_probability(factor, variable: str, state: str) ->
float:
        states = factor.state_names[variable]
        values = list(map(float, factor.values))
        return float(values[states.index(state)])

    def build_model(root_centers: dict):
        model = DiscreteBayesianNetwork(EDGES)

        root_cpds = []
        for node_id, center_state in root_centers.items():
            states = states_for(node_id)
            probabilities = peaked_prior(node_id, center_state,
ROOT_CONFIDENCE)
            root_cpds.append(
                TabularCPD(
                    variable=node_id,
                    variable_card=len(states),
                    values=[[probability] for probability in
probabilities],
                    state_names={node_id: states},
                )
            )

        te_values = [
            TE_CPT_DF["P_NORMAL"].astype(float).tolist(),
            TE_CPT_DF["P_DEVELOPING"].astype(float).tolist(),
            TE_CPT_DF["P_CONFIRMED"].astype(float).tolist(),
        ]
        te_cpd = TabularCPD(
            variable="TE_CorrosionState",
            variable_card=3,
            values=te_values,
            evidence=TE_PARENTS,
            evidence_card=[len(states_for(parent)) for parent in
TE_PARENTS],
            state_names={
                "TE_CorrosionState":
STATE_SPACES["TOP_EVENT_STATE"],
                **{parent: states_for(parent) for parent in
TE_PARENTS},
            },
        )
```

```
        evidence_cpds = [
            binary_cpd(
                "EV_TOTIRON_High",
                "TE_CorrosionState",
                lambda state: evidence_yes_probability(state,
"TOTIRON"),
            ),
            binary_cpd(
                "EV_SAP_Positive",
                "TE_CorrosionState",
                lambda state: evidence_yes_probability(state,
"SAP"),
            ),
            binary_cpd(
                "EV_QuillFailure",
                "TE_CorrosionState",
                lambda state: evidence_yes_probability(state,
"QUILL"),
            ),
        ]

        consequence_cpds = [
            binary_multi_cpd(
                "C1_LOPC",
                ["TE_CorrosionState", "MB1_ESD_OK", "MB2_TSV_OK"],
                lopc_yes_probability,
            ),
            binary_multi_cpd(
                "C2_UnplannedShutdown",
                ["TE_CorrosionState", "MB1_ESD_OK"],
                shutdown_yes_probability,
            ),
            binary_multi_cpd(
                "C3_Escalation",
                ["C1_LOPC", "MB1_ESD_OK", "MB2_TSV_OK"],
                escalation_yes_probability,
            ),
        ]

        model.add_cpds(*root_cpds, te_cpd, *evidence_cpds,
*consequence_cpds)
        if not model.check_model():
            raise RuntimeError("Bayesian network model validation
failed")
        return model

    def analyze_snapshot(payload: dict, previous_risk_state: str |
None = None) -> dict:
        root_centers, evidence = discretize_source(payload)
        model = build_model(root_centers)
        inference = VariableElimination(model)
```

```
        q_te = inference.query(["TE_CorrosionState"],
evidence=evidence, show_progress=False)
        q_c1 = inference.query(["C1_LOPC"], evidence=evidence,
show_progress=False)
        q_c2 = inference.query(["C2_UnplannedShutdown"],
evidence=evidence, show_progress=False)
        q_c3 = inference.query(["C3_Escalation"], evidence=evidence,
show_progress=False)

        p_normal = factor_probability(q_te, "TE_CorrosionState",
"NORMAL")
        p_developing = factor_probability(q_te, "TE_CorrosionState",
"DEVELOPING")
        p_confirmed = factor_probability(q_te, "TE_CorrosionState",
"CONFIRMED")
        probabilities = {
            "NORMAL": p_normal,
            "DEVELOPING": p_developing,
            "CONFIRMED": p_confirmed,
        }
        risk_state = max(probabilities, key=probabilities.get)
        analysis_timestamp = utc_now_iso()

        analysis_payload = {
            "p_normal": p_normal,
            "p_developing": p_developing,
            "p_confirmed": p_confirmed,
            "total_corrosion_risk": p_developing + p_confirmed,
            "lopc_probability": factor_probability(q_c1, "C1_LOPC",
"YES"),
            "shutdown_probability": factor_probability(
                q_c2, "C2_UnplannedShutdown", "YES"
            ),
            "escalation_probability": factor_probability(q_c3,
"C3_Escalation", "YES"),
            "timestamp": analysis_timestamp,
        }

        risk_payload = {
            "risk_state": risk_state,
            "previous_state": previous_risk_state or risk_state,
            "model_confidence": max(probabilities.values()),
            "timestamp": analysis_timestamp,
        }

        return {
            "source_timestamp": payload["timestamp"],
            "root_states": root_centers,
            "evidence": evidence,
            "analysis_payload": analysis_payload,
```

```
            "risk_payload": risk_payload,
            "risk_state": risk_state,
        }

    return analyze_snapshot, build_model, factor_probability
```

- 셀 5 - source topic을 구독하고 각 incoming snapshot을 처리한 뒤 Bayesian analysis를 실행하고 result를 게시합니다.

코드 보기

```
@app.cell
def _(
    ANALYSIS_TOPIC,
    AUTOSTART,
    MQTT_BROKER_HOST,
    MQTT_BROKER_PORT,
    MQTT_CLIENT_ID,
    MQTT_PASSWORD,
    MQTT_USERNAME,
    OUTPUT_QOS,
    RISK_TOPIC,
    SOURCE_QOS,
    SOURCE_TOPIC,
    analyze_snapshot,
    json,
    mqtt,
    normalize_source_payload,
    source_message_id,
    threading,
    utc_now_iso,
):
    """Long-running MQTT subscriber and publisher service."""

    class CorrosionBNService:
        def __init__(self):
            self.lock = threading.RLock()
            self.client = None
            self.running = False
            self.connected = False
            self.last_message_id = None
            self.last_source_timestamp = None
            self.last_analysis_timestamp = None
            self.previous_risk_state = None
            self.processed_count = 0
            self.duplicate_count = 0
            self.error_count = 0
```

```
            self.last_error = None
            self.last_result = None
            self.connection_rc = None

        def _create_client(self):
            # Use MQTT 3.1.1 and callback API v1 for broad
compatibility with
            # EMQX and different paho-mqtt releases bundled in
Notebook images.
            kwargs = {
                "client_id": MQTT_CLIENT_ID,
                "clean_session": True,
                "protocol": mqtt.MQTTv311,
                "transport": "tcp",
            }
            try:
                client = mqtt.Client(

callback_api_version=mqtt.CallbackAPIVersion.VERSION1,
                    **kwargs,
                )
            except (AttributeError, TypeError):
                client = mqtt.Client(**kwargs)

            client.username_pw_set(MQTT_USERNAME, MQTT_PASSWORD)
            client.on_connect = self._on_connect
            client.on_disconnect = self._on_disconnect
            client.on_message = self._on_message
            client.reconnect_delay_set(min_delay=1, max_delay=30)
            client.enable_logger()
            return client

        def _on_connect(self, client, userdata, flags, rc,
properties=None):
            code = int(rc)
            with self.lock:
                self.connected = code == 0
                self.connection_rc = code
                self.last_error = None if code == 0 else f"MQTT
connection rejected, rc={code}"
            if code == 0:
                sub_rc, mid = client.subscribe(SOURCE_TOPIC,
qos=SOURCE_QOS)
                if sub_rc != mqtt.MQTT_ERR_SUCCESS:
                    with self.lock:
                        self.last_error = f"MQTT subscribe failed,
rc={sub_rc}"
                    return
                print(f"[BN SERVICE] connected and subscribed to
{SOURCE_TOPIC}")
            else:
```

```
                print(f"[BN SERVICE ERROR] connection rejected rc=
{code}")

        def _on_disconnect(self, client, userdata, rc,
properties=None):
            with self.lock:
                self.connected = False
                self.connection_rc = int(rc) if rc is not None else
None
                if self.running and rc:
                    self.last_error = f"MQTT disconnected
unexpectedly, rc={rc}"
            print(f"[BN SERVICE] disconnected: {rc}")

        def _publish_json(self, topic: str, payload: dict):
            info = self.client.publish(
                topic,
                payload=json.dumps(payload, separators=(",", ":"),
ensure_ascii=False),
                qos=OUTPUT_QOS,
                retain=False,
            )
            info.wait_for_publish(timeout=10)
            if info.rc != mqtt.MQTT_ERR_SUCCESS:
                raise RuntimeError(f"MQTT publish failed for
{topic}: rc={info.rc}")

        def _on_message(self, client, userdata, msg):
            try:
                raw_payload = json.loads(msg.payload.decode("utf-
8"))
                payload = normalize_source_payload(raw_payload)
                message_id = source_message_id(payload)

                with self.lock:
                    if message_id == self.last_message_id:
                        self.duplicate_count += 1
                        return
                    previous_risk_state = self.previous_risk_state

                result = analyze_snapshot(payload,
previous_risk_state)

                # Publish only after both payloads have been
generated successfully.
                self._publish_json(ANALYSIS_TOPIC,
result["analysis_payload"])
                self._publish_json(RISK_TOPIC,
result["risk_payload"])

                with self.lock:
```

```
                    self.last_message_id = message_id
                    self.last_source_timestamp =
payload["timestamp"]
                    self.last_analysis_timestamp =
result["analysis_payload"]["timestamp"]
                    self.previous_risk_state = result["risk_state"]
                    self.processed_count += 1
                    self.last_result = result
                    self.last_error = None

                print(
                    "[BN SERVICE] processed",
                    payload["timestamp"],
                    "->",
                    result["risk_state"],
                )
            except Exception as exc:
                with self.lock:
                    self.error_count += 1
                    self.last_error = f"{type(exc).__name__}: {exc}"
                print("[BN SERVICE ERROR]", self.last_error)

        def start(self):
            with self.lock:
                if self.running:
                    return False
                if not MQTT_PASSWORD:
                    raise RuntimeError(
                        "TIER0_MQTT_PASSWORD is empty. Set it before
starting the service."
                    )
                self.client = self._create_client()
                self.running = True
            try:
                # Synchronous connect surfaces DNS, socket, and
authentication
                # failures immediately instead of leaving
running=True forever.
                rc = self.client.connect(
                    MQTT_BROKER_HOST,
                    MQTT_BROKER_PORT,
                    keepalive=30,
                )
                if rc != mqtt.MQTT_ERR_SUCCESS:
                    raise RuntimeError(f"MQTT connect() failed, rc=
{rc}")
                self.client.loop_start()
                return True
            except Exception as exc:
                with self.lock:
                    self.running = False
```

```
                    self.connected = False
                    self.last_error = f"{type(exc).__name__}: {exc}"
                    self.client = None
                raise

        def stop(self):
            with self.lock:
                if not self.running:
                    return False
                client = self.client
                self.running = False
                self.connected = False
                self.client = None
            try:
                client.disconnect()
            finally:
                client.loop_stop()
            return True

        def restart(self):
            self.stop()
            return self.start()

        def status(self) -> dict:
            with self.lock:
                return {
                    "running": self.running,
                    "connected": self.connected,
                    "connection_rc": self.connection_rc,
                    "client_id": MQTT_CLIENT_ID,
                    "broker": f"{MQTT_BROKER_HOST}:
{MQTT_BROKER_PORT}",
                    "source_topic": SOURCE_TOPIC,
                    "analysis_topic": ANALYSIS_TOPIC,
                    "risk_topic": RISK_TOPIC,
                    "processed_count": self.processed_count,
                    "duplicate_count": self.duplicate_count,
                    "error_count": self.error_count,
                    "last_source_timestamp":
self.last_source_timestamp,
                    "last_analysis_timestamp":
self.last_analysis_timestamp,
                    "previous_risk_state": self.previous_risk_state,
                    "last_error": self.last_error,
                }

    # Reuse a previous service instance when the cell is re-
executed, preventing
    # duplicate subscribers inside the same Python process.
    existing = globals().get("_CORROSION_BN_SERVICE")
    if existing is not None:
```

```
        try:
            existing.stop()
        except Exception:
            pass

    CORROSION_BN_SERVICE = CorrosionBNService()
    globals()["_CORROSION_BN_SERVICE"] = CORROSION_BN_SERVICE

    AUTOSTART_ERROR = None
    if AUTOSTART:
        try:
            CORROSION_BN_SERVICE.start()
        except Exception as exc:
            AUTOSTART_ERROR = f"{type(exc).__name__}: {exc}"

    return AUTOSTART_ERROR, CORROSION_BN_SERVICE, CorrosionBNService
```

- 셀 6 - MQTT connection status와 service statistics를 표시합니다.

코드 보기

```
@app.cell
def _(AUTOSTART_ERROR, CORROSION_BN_SERVICE, mo, pd):
    """Service status and usage instructions."""
    status = CORROSION_BN_SERVICE.status()

    blocks = [
        mo.md("## MQTT analysis service"),
        pd.DataFrame([status]),
    ]
    if status.get("last_error"):
        blocks.append(mo.md(f"### Connection / processing
error\n`{status['last_error']}`"))
    elif status.get("running") and not status.get("connected"):
        blocks.append(mo.md("### Connecting\nThe MQTT client has
started but has not completed the broker connection yet."))

    if AUTOSTART_ERROR:
        blocks.append(
            mo.md(
                f"""
                ### Service not started

                `{AUTOSTART_ERROR}`

                Set the MQTT password in the Notebook environment
and run this cell again:
```

```
                ~~~python
                import os
                # MQTT credentials are already configured in this
notebook.
                ~~~
                """
            )
        )
    else:
        blocks.append(
            mo.md(
                """
                The service starts automatically and processes one
complete source
                snapshot for every new MQTT message. No manual
analysis trigger or SQL
                polling is required.

                Available controls in Python:

                ~~~python
                CORROSION_BN_SERVICE.status()
                CORROSION_BN_SERVICE.stop()
                CORROSION_BN_SERVICE.start()
                CORROSION_BN_SERVICE.restart()
                ~~~
                """
            )
        )

    mo.vstack(blocks)
    return status,
```

- 셀 7 - 가장 최근의 Bayesian analysis result를 표시합니다.

코드 보기

```
@app.cell
def _(CORROSION_BN_SERVICE, mo):
    """Display the most recent analysis result when one is
available."""
    latest = CORROSION_BN_SERVICE.last_result
    if latest is None:
        mo.md("### Latest result\nNo source message has been
processed yet.")
    else:
```

```
        mo.vstack(
            [
                mo.md("### Latest source snapshot result"),
                mo.md(f"- Source timestamp:
`{latest['source_timestamp']}`"),
                mo.md(f"- Risk state: `{latest['risk_state']}`"),
                mo.md("#### Root states"),
                latest["root_states"],
                mo.md("#### Evidence"),
                latest["evidence"],
                mo.md("#### Corrosion analysis payload"),
                latest["analysis_payload"],
                mo.md("#### Corrosion risk payload"),
                latest["risk_payload"],
            ]
        )
    return


if __name__ == "__main__":
    app.run()
```

3. 모든 cell을 실행한 뒤 **UNS**로 이동해 결과를 확인합니다.

Bow-tie 앱 구축

1. Tier0에서 **Builder**로 이동합니다.
2. 대화 상자에 애플리케이션 요구 사항을 입력하고 구축을 시작합니다.

> ⚠ 반드시 명시할 내용
> agent에게 UNS 데이터를 사용하라고 알려주고, 구체적인 topic path를 제공해야 합니다. 그렇지 않으면 agent가 올바른 데이터를 표시하는 데 어려움을 겪을 수 있습니다.

> 🚀 다른 Tier0에서 앱을 구축하는 경우
> 연결 충돌을 피하려면 서로 다른 **clientId**(다른 suffix 추가)를 사용해 UNS broker에 연결합니다.

Builder prompt 보기



CDU Atmospheric Overhead Corrosion을 위한 현대적인 산업용 Bow-Tie Analysis 애플리케이션을 구축하세요.

다음 UNS topic만 데이터 소스로 사용하세요.

- Refinery/CDU_Plant/Metric/Corrosion_Monitoring
- Refinery/CDU_Plant/Metric/Corrosion_Analysis
- Refinery/CDU_Plant/State/Corrosion_Risk

새 UNS 메시지가 도착하면 애플리케이션이 자동으로 업데이트되어야 합니다.

이 애플리케이션은 운영자가 현재 부식 위험, 관련 위협, barrier 상태, 가능한 결과를 이해하는 데 도움이 되어야 합니다.

다음을 포함하는 single-page dashboard를 만드세요.

1. 메인 섹션으로 Bow-Tie diagram:
- 왼쪽에 threats 표시
- threats와 top event 사이에 preventative barriers 표시
- 중앙 top event로 corrosion 표시
- top event 이후 mitigative barriers 표시
- 오른쪽에 consequences 표시

2. 눈에 잘 띄는 risk summary:
- Current risk state
- Model confidence
- Normal, developing, confirmed probabilities
- Total corrosion risk

3. Corrosion_Monitoring의 주요 값을 보여주는 compact live process data panel을 만드세요. chloride, pH, wash water flow, desalter performance, wash water rate, total iron, SAP evidence, injection quill status를 포함하세요.

4. Consequence indicators:
- Loss of primary containment
- Unplanned shutdown
- Escalation risk

5. Visual status rules:
- Green은 normal 또는 healthy condition
- Amber/yellow는 developing risk 또는 warning condition
- Red는 confirmed risk, failed barrier 또는 severe consequence
- Gray는 data unavailable
- Blue는 interaction 및 informational highlight에만 사용

6. Interaction:
- threat, barrier 또는 consequence를 선택하면 현재 상태, 관련 process value, 짧은 설명을 표시해야 합니다.
- 새 UNS 데이터가 도착하면 interface를 자동으로 업데이트하세요.

```
    - 최신 데이터 timestamp와 built-in SDK의 connection/subscription status
를 표시하세요.

    Design requirements:

    애플리케이션 전체에서 기존 visual design language와 styling을 유지하세요.

    현대적인 industrial operations dashboard 스타일을 사용하세요.
    - pure black 대신 dark blue-gray color palette를 사용합니다.
    - subtle gradients가 있는 layered panels를 사용합니다.
    - 약 16-20px의 큰 rounded corners를 가진 card-based layout을 사용합니다.
    - heavy shadows 대신 thin borders로 구분합니다.
    - soft depth와 restrained glow effects를 사용합니다.
    - 충분한 padding, 일관된 alignment, 깨끗한 spacing을 유지합니다.
    - body text에는 IBM Plex Sans를 사용합니다.
    - headings에는 Space Grotesk를 사용합니다.
    - clear visual hierarchy를 가진 compact, information-dense cards를 사용
합니다.
    - letter spacing을 늘린 uppercase section labels를 사용합니다.
    - business analytics가 아니라 professional refinery / SCADA interface
처럼 보이게 합니다.
    - skeuomorphic elements 대신 flat illustrations와 SVG graphics를 사용합
니다.
    - smooth하고 subtle한 transitions 및 hover effects를 사용합니다.

    전체 외관은 clean, modern, technical, operational하게 느껴져야 합니다.

    피해야 할 것:
    - Pure black backgrounds
    - Glassmorphism
    - Cyberpunk styling
    - Neon lighting
    - Overly decorative effects
    - Business analytics dashboard styling

    clear hierarchy, compact cards, readable labels, responsive layout을
갖춘 clean, professional refinery operations style을 사용하세요.

    해당 UNS model에 정의된 payload fields를 정확히 사용하세요. 추가 fields,
calculated values 또는 mock data를 만들지 마세요.
```

3. Once the application is complete after certain rounds of refining, deploy it.
4. Go to **Launchpad**, open the application and check.

베스트 프랙티스

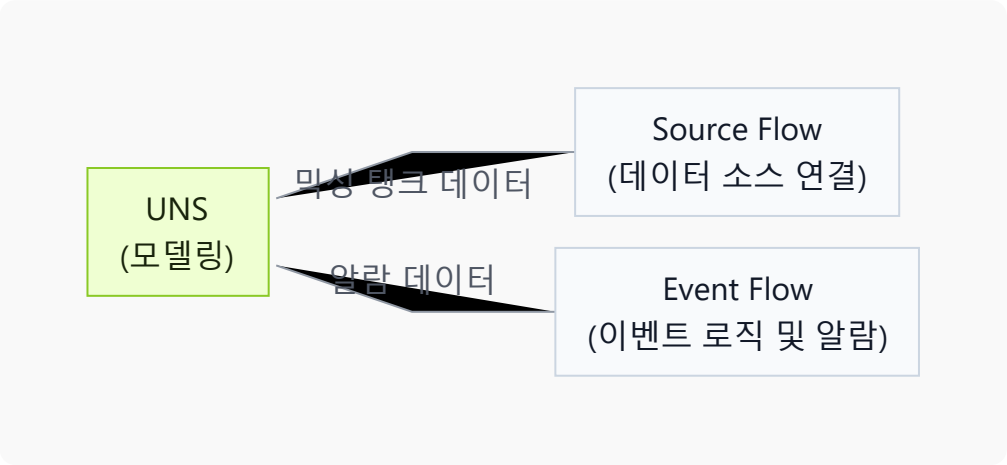
Agent로 데이터 워크플로 구축하기

UNS Agent를 사용해 데이터 모델링, 데이터 연결, 데이터 이벤트를 포함하는 워크플로를 구축합니다.

여러 모듈에서 데이터 모델, 연결, 이벤트를 단계별로 만드는 대신, UNS Agent가 대화를 통해 워크플로를 처리합니다.

워크플로 배경

믹싱 탱크는 재료를 섞으면서 가열합니다. 데이터 모델을 만들고 온도, 수위, 히터 상태를 수집해 탱크 과열 여부를 판단하고 알람을 트리거합니다.



워크플로 구축 방법

1. **UNS**에서 `full_access`로 UNS Agent와 대화를 시작합니다.
2. UNS 모델을 만들기 위한 프롬프트를 입력합니다.

```
믹싱 탱크의 온도, 수위, 히터 상태를 나타내는 데이터 모델을 만듭니다. 온도와 수위는
하나의 metric topic에, 히터 상태는 하나의 state topic에 넣어 주세요.
```

3. 모델이 완성되었는지 확인한 뒤, 해당 데이터를 모델에 연결하도록 프롬프트를 입력합니다.

```
이 2개 topic으로 데이터를 보내는 Source Flow를 만들고 합리적인 데이터를 시뮬레이션해 주세요.
```

> 🚀 실제 데이터 소스가 있는 경우
> 데이터 소스 정보를 Agent에게 알려 연결하게 합니다.

4 다음 로직으로 이벤트를 만들기 위한 프롬프트를 입력합니다.

- 낮은 수위 경고: 수위가 20% 미만이고 히터가 켜져 있을 때 트리거됩니다.
- 건식 가열 알람: 수위가 15% 미만이고 온도가 90°C를 초과하며 히터가 켜져 있을 때 트리거됩니다.

```
다음 알람 로직을 위한 Event Flow를 만들어 주세요:
  - Warning: level < 20 이고 heater_status = true 일 때 트리거.
  - Critical: level < 15, temperature > 90, heater_status = true 일 때 트리거.
  - level > 25 또는 heater_status = false 이면 활성 알람을 해제.
기존 UNS 모델 아래에 알람 결과를 받을 새 topic을 만듭니다. 출력에는 알람 레벨, 메시지, 활성 상태, 타임스탬프를 포함합니다.
```

5 **UNS**에서 결과를 확인합니다.

> ⓘ 참고
> 각 단계 이후에도 Agent와 계속 대화해 결과를 조정할 수 있습니다.

베스트 프랙티스

Digital Twin으로 데이터 표시하기

3D 모델과 장면에서 UNS 데이터를 실시간으로 확인합니다.

Tier0의 digital twin 기능을 사용하면 실제와 같은 3D 장면과 모델에서 UNS 데이터를 실시간으로 볼 수 있습니다.

공장 씬 시각화

1. **Model**에 모델을 업로드한 뒤, 실시간 동작에 사용할 UNS 데이터 매핑 노드를 선택합니다.
2. 서로 다른 UNS 데이터를 가진 인스턴스를 모델에 추가해 여러 기계를 시뮬레이션합니다.
3. **Scene**에서 **Instance Library**의 인스턴스를 씬에 추가합니다.
4. 상단 도구로 모델을 이동, 확대/축소, 회전해 씬에 맞추고 시점과 좌표 위치를 조정합니다.
5. **Object**에서 모델 위치와 바인딩된 UNS 데이터를 확인합니다.

참조

API 레퍼런스

Tier0 API 통합 핵심 사항과 기계가 읽을 수 있는 OpenAPI 레퍼런스입니다.

시작하기

- 기본 URL로 `https://<your-tier0-host>/openapi/v1`을 사용합니다.
- `/assets/files/public`을 제외한 OpenAPI 엔드포인트에 `x-api-key`를 보냅니다. `public`으로 표시된 파일은 API 키 없이 다운로드할 수 있습니다.
- 먼저 `POST /auth/whoami`를 호출하여 API 키의 ID와 유효 권한을 확인합니다.

> ⓘ **OpenAPI 참조**
> 요청 스키마, 응답 스키마 및 실행 가능한 요청은 대화형 **API 레퍼런스**에서 확인할 수 있습니다.

결과 처리

HTTP `200`을 반환하더라도 UNS 일괄 작업은 일부 실패할 수 있습니다. 성공으로 간주하기 전에 각 결과를 확인하세요.

Flow의 사용자 지정 HTTP In 경로

Source Flow 및 **Event Flow** 안에서 개발하는 API의 입력과 응답은 배포된 Flow가 정의합니다.

- **Source Flow**는 `/flow/source/`를 사용하며 기본적으로 공개됩니다. 민감한 동작을 노출하기 전에 인그레스 또는 Flow 수준 접근 제어를 추가하세요.
- **Event Flow**는 `/flow/event/`를 사용하며 `flow.read` 권한이 있는 `x-api-key`가 필요합니다.

> ⓘ **Source Flow 또는 Event Flow의 사용자 지정 API 예시**
> 예를 들어 API URL이 `/integrations/echo`이고 API 개발에 `HTTP In` 노드가 필요한 경우:
>
> - `HTTP In`의 경로는 `/integrations/echo`이며, 나머지는 Tier0가 완성합니다.

- 외부 호출 URL은 `https://tier0.example.com/flow/source/integrations/echo` 또는 `https://tier0.example.com/flow/event/integrations/echo` 입니다.

문제 해결

- `401` / `403` : API 키, 범위 및 소유자 권한을 확인하세요.
- `404` : HTTP 메서드, 경로 접두사, 노드 URL 및 배포 상태를 확인하세요.
- `No response` : 모든 Flow 경로가 원래 요청 및 응답 속성을 버리지 않고 `HTTP Response` 노드에 도달하는지 확인하세요.

참조

CLI 명령 참조

CLI 및 Agent Skill 명령 참조입니다.

명령, 플래그 또는 예시 검색...

전역 플래그 이 플래그는 모든 `tier0` 명령에서 사용할 수 있습니다.

`--json` 원본 JSON 출력 `--debug` HTTP 요청/응답 상세 출력

기본 작업

`tier0 doctor`

로컬 Tier0 CLI 연결 및 인증 진단

사용법

```
tier0 doctor
```

참고

- 로컬 설정, OpenAPI 연결, API key 존재 여부와 API key 신원을 확인합니다.

예시

```
tier0 doctor
```

`tier0 config`

설정 보기 또는 업데이트

사용법

```
tier0 config
```

플래그

`--base-url`
플랫폼 base URL 설정

`--api-key`
API key 설정(tier0 login 대체 가능)

`--lang`
지원 중단된 호환성 플래그입니다. CLI 출력은 영어만 지원합니다
지원 중단

예시

```
tier0 config
```

```
tier0 config --base-url https://tier0.dev
```

```
tier0 config --api-key sk-per-xxxxxx
```

`tier0 login`

Device Flow로 인증

사용법

```
tier0 login
```

플래그

`--no-wait`

인증 URL을 출력하고 폴링 없이 종료

`--setup-code`

새로 만들지 않고 기존 setup code로 폴링

`--base-url`

플랫폼 base URL 재정의

참고

- `--no-wait` 를 사용하지 않으면 CLI가 최대 10분 동안 5초마다 폴링합니다.
- `--no-wait` `--json` 을 사용하면 CLI가 `verification_url` , `setup_code` , `expires_in` 을 반환합니다.

예시

```
tier0 login
```

```
tier0 login --no-wait --json
```

```
tier0 login --setup-code <setup_code>
```

`tier0 api`

API endpoint 직접 호출

사용법

```
tier0 api <endpoint>
```

플래그

`--body`
요청 본문 JSON 문자열

`--body-file`
파일에서 요청 본문 읽기

`--method`
HTTP 메서드(GET|POST|PUT|DELETE)
기본값: `POST`

`--dry-run`
실행하지 않고 요청 미리보기

참고

- `--body` 와 `--body-file` 은 함께 사용할 수 없습니다.

예시

```
tier0 api /openapi/v1/uns/browse --body '{"path":"/"}'
```

```
tier0 api /openapi/v1/uns/read --body '{"topics":["demo"]}'
```

```
tier0 api /openapi/v1/uns/write --body-file body.json
```

`tier0 upgrade`

CLI를 최신 버전으로 업그레이드

사용법

```
tier0 upgrade
```

플래그

`--dry-run`
설치하지 않고 업데이트 확인

`-v, --version`
특정 버전으로 업그레이드(기본값: latest)

예시

```
tier0 upgrade
```

```
tier0 upgrade --dry-run
```

`tier0 uninstall`

tier0 CLI 제거

사용법

```
tier0 uninstall
```

플래그

`--purge`
설정 파일(자격 증명)도 삭제

`--remove-skills`
감지된 AI agents에서도 Tier0 Skill 제거

`--keep-skills`
agent skills 유지(지원 중단됨. 현재는 기본 동작)
지원 중단

참고

- Agent Skills와 설정은 기본적으로 유지됩니다.
- CLI 제거 중에는 로컬 내장 skills 디렉터리가 삭제됩니다.

예시

```
tier0 uninstall
```

```
tier0 uninstall --purge
```

```
tier0 uninstall --remove-skills
```

```
tier0 uninstall --purge --remove-skills
```

`tier0 version`

버전 표시

사용법

```
tier0 version
```

예시

```
tier0 version
```

스킬

`tier0 skills install`

이 CLI에 내장된 Skill 설치 또는 복구

사용법

```
tier0 skills install
```

플래그

`--force`
활성 Skill을 내장 기준 버전으로 교체

`--no-sync`
감지된 AI agents와 동기화하지 않고 로컬에만 설치

예시

```
tier0 skills install
```

```
tier0 skills install --force
```

`tier0 skills list`

설치된 skills 목록 표시

별칭: ls

사용법

```
tier0 skills list
```

예시

```
tier0 skills list
```

`tier0 skills update`

skills를 최신 버전으로 업데이트

별칭: upgrade

사용법

```
tier0 skills update
```

플래그

`--dry-run`
설치하지 않고 업데이트 확인

예시

```
tier0 skills update
```

```
tier0 skills update --dry-run
```

`tier0 skills version`

skills 버전 정보 표시

별칭: ver

사용법

```
tier0 skills version
```

예시

```
tier0 skills version
```

`tier0 skills status`

활성 Skill과 내장 Skill 상태 표시

사용법

```
tier0 skills status
```

예시

```
tier0 skills status
```

`tier0 skills sync`

활성 Skill을 감지된 AI agents에 동기화

사용법

```
tier0 skills sync
```

예시

```
```

```
tier0 skills sync
```

인증

`tier0 auth whoami`

현재 API key 신원 표시

사용법

```
tier0 auth whoami
```

참고

- 사용자, workspace, key 이름, key 유형, 역할 및 권한을 표시합니다.

예시

```
tier0 auth whoami
```

```
tier0 auth whoami --json
```

UNS

`tier0 uns browse`

UNS namespace 트리 탐색

사용법

```
tier0 uns browse
```

플래그

`-p, --path`
UNS 트리에서 탐색할 경로
기본값: `/`

`-d, --max-depth`
최대 재귀 깊이(0 = 무제한)
기본값: `1`

`--include-metadata`
노드 메타데이터 포함

`--include-leaf-value`
리프 노드 값 포함

예시

```
tier0 uns browse
```

```
tier0 uns browse --path /devices
```

```
tier0 uns browse --path / --max-depth 2
```

`tier0 uns search`

UNS topics 검색

사용법

```
tier0 uns search
```

플래그

`-k, --keyword`
이름 키워드로 검색

`--path-prefix`
경로 접두사로 필터링
기본값: `/`

`--topic-type`
topic type으로 필터링

`--page`
페이지 번호
기본값: `1`

`-l, --size`
페이지 크기(최대 결과 수)
기본값: `20`

`--include-metadata`
노드 메타데이터 포함

`--include-leaf-value`
리프 노드 값 포함

예시

```
tier0 uns search --keyword temp
```

```
tier0 uns search --path-prefix /devices --size 50
```

```
tier0 uns search --keyword temp --include-metadata
```

`tier0 uns read`

UNS topics의 현재 값 읽기

사용법

```
tier0 uns read [topic...]
```

```
tier0 uns read --topic <topic>
```

플래그

`-t, --topic`
읽을 topic 이름(반복 가능, 위치 인수도 허용)
반복 가능

`--include-metadata`
topic 메타데이터(topicType, fields, description) 포함

`--include-leaf-value`
리프 노드 값 포함

참고

- 위치 인수 또는 `--topic`으로 최소 하나의 topic이 필요합니다.

예시

```
tier0 uns read demo
```

```
tier0 uns read --topic demo
```

```

```

```
tier0 uns read temp humidity
```

```
tier0 uns read --topic sensor1 --include-metadata
```

`tier0 uns write`

UNS topic에 값 쓰기

사용법

```
tier0 uns write
```

플래그

`-t, --topic`
쓸 대상 topic 이름(필수)
필수

`-v, --value`
JSON 값 문자열(--file과 함께 사용할 수 없음)

`-f, --file`
파일에서 값 읽기(--value와 함께 사용할 수 없음)

`--qos`
MQTT QoS 레벨(0/1/2)
기본값: `0`

`--retain`
MQTT retain 플래그 설정

`--dry-run`
실행하지 않고 요청 미리보기

참고

- `--value`와 `--file`은 함께 사용할 수 없습니다.
- 값은 topic fields와 일치하는 유효한 JSON object여야 합니다.

예시

```
tier0 uns write --topic demo --value '{"temp":25}'
```

```
tier0 uns write --topic demo --file payload.json
```

```
tier0 uns write --topic sensor1 --value '{"on":true}' --qos 1 --retain
```

`tier0 uns history`

topics의 이력 데이터 조회

사용법

```
tier0 uns history
```

플래그

`-t, --topic`
topic 이름(반복 가능, 필수)
필수 반복 가능

`--start`
시작 시간: 상대값(-1h/-30m/-7d), ISO 8601 또는 'now'(필수)
필수

`--end`
종료 시간: 상대값, ISO 8601 또는 'now'(기본값: now)
기본값: `now`

`--page`
페이지 번호
기본값: `1`

`-l, --size`

페이지 크기(최대 데이터 포인트 수)

기본값: `100`

`--interval`

집계 간격(예: 1m, 1h, 1d)

`--fn`

집계 함수(avg/max/min/sum/count)

`--field`

집계 필드 이름

예시

```
tier0 uns history -t demo --start -1h
```

```
tier0 uns history -t demo --start -24h --end now --fn avg --interval 1h
```

```
tier0 uns history -t demo --start 2026-01-01T00:00:00Z --end 2026-01-02T00:00:00Z
```

`tier0 uns create`

UNS namespace 노드 생성

사용법

```
tier0 uns create
```

플래그

`-t, --topic`

topic 경로 또는 리프 이름(--file을 사용하지 않을 때 필수)

`--parent`
상위 경로 접두사(선택 사항, --topic과 결합)

`--type`
노드 유형: 'path'(폴더) 또는 'topic'(데이터 포인트)

`-d, --display-name`
표시 이름

`--description`
설명

`--alias`
별칭

`-f, --file`
JSON 파일에서 namespace 정의 읽기({"namespace":[]} 또는 배열만)

`--topic-type`
지원 중단: topic type은 이제 경로(리프 앞의 Metric/Action/State 폴더)에서 파생됩니다.
지원 중단

`--fields`
schema fields JSON 배열

`--fields-file`
파일에서 schema fields JSON 배열 읽기

`--dry-run`
실행하지 않고 요청 미리보기

참고

- `--file`을 사용하지 않으면 `--topic`과 `--type`이 필요합니다.
- `--fields`와 `--fields-file`은 함께 사용할 수 없습니다.
- topic 노드에서는 리프 바로 앞 경로 세그먼트가 Metric, Action 또는 State여야 합니다.

예시

```
tier0 uns create --topic Plant/Line1/Metric/Temperature --type topic
```

```
tier0 uns create --parent Factory1/Line1/Station1 --topic
Metric/ProductionCount --type topic
```

```
tier0 uns create --topic Plant/Line1 --type path --display-name 'Line 1'
```

```
tier0 uns create --file namespace.json
```

`tier0 uns update`

UNS topic 메타데이터 업데이트

사용법

```
tier0 uns update
```

플래그

`-p, --path`
업데이트할 topic 경로(필수)
필수

`-n, --name`
새 이름

`--alias`
새 별칭

`--description`
새 설명

`--clear-description`
빈 shell 인수에 의존하지 않고 설명 지우기

`-d, --display-name`
새 표시 이름

`--extend-properties`
확장 속성 JSON object

`--fields`
schema fields JSON 배열

`--fields-file`
파일에서 schema fields JSON 배열 읽기

`--update-mask`
업데이트할 필드(반복 가능, 예: name,description,fields)
반복 가능

`--dry-run`
실행하지 않고 요청 미리보기

참고

- `--description` 과 `--clear-description` 은 함께 사용할 수 없습니다.
- `--fields` 와 `--fields-file` 은 함께 사용할 수 없습니다.
- 업데이트할 필드를 하나 이상 지정해야 합니다.

예시

```
tier0 uns update --path Plant/Line1/Metric/Temperature --display-name
'Line 1 Temp'
```

```
tier0 uns update --path Plant/Line1 --description 'Production line 1' --
update-mask description
```

```
tier0 uns update --path Plant/Line1/Metric/Temperature --fields
'[{"name":"temp","type":"float","unit":"C"}]' --update-mask fields
```

`tier0 uns delete`

UNS 노드 삭제

사용법

```
tier0 uns delete
```

플래그

`-p, --path`
삭제할 노드 경로(반복 가능, 필수)
필수 반복 가능

`--hard`
하드 삭제(되돌릴 수 없음)

`-y, --yes`
고위험 작업 확인(필수)
필수

`--dry-run`
실행하지 않고 요청 미리보기

참고

- 기본값은 소프트 삭제이며 복원할 수 있습니다.
- `--hard` 는 되돌릴 수 없습니다.
- `--dry-run` 을 사용하지 않는 한 고위험 작업에는 `--yes` 가 필요합니다.

예시

```
tier0 uns delete --path factory/line1/sensor/temp --yes
```

```
tier0 uns delete --path factory/line1/sensor/temp --path
factory/line1/sensor/humi --yes
```

```
tier0 uns delete --path factory/line1/sensor/temp --hard --yes
```

`tier0 uns restore`

이력에서 topic 복원

사용법

```
tier0 uns restore
```

플래그

`-p, --path`
복원할 topic 경로(필수)
필수

`-y, --yes`
고위험 작업 확인(필수)
필수

`--dry-run`
실행하지 않고 요청 미리보기

참고

- `--dry-run`을 사용하지 않는 한 고위험 작업에는 `--yes`가 필요합니다.

예시

```
tier0 uns restore --path Plant/Line1/Metric/Temperature --yes
```

Node-RED 플로우

`tier0 flow list`

flows 목록 표시

별칭: ls

사용법

```
tier0 flow list
```

플래그

`-k, --keyword`
이름 키워드로 필터링

`-t, --type`
유형별 필터링(SourceFlow/EventFlow)

`--source`
SourceFlow만 표시

`--event`
EventFlow만 표시

참고

- `--source` 와 `--event` 는 함께 사용할 수 없습니다.
- `--type` 은 `--source` 또는 `--event` 와 함께 사용할 수 없습니다.

예시

```
tier0 flow list
```

```
tier0 flow list --source --json
```

`tier0 flow get`

flow 상세 정보 가져오기

사용법

```
tier0 flow get --id <id>
```

```
tier0 flow get <id>
```

플래그

`--id`
Flow ID
기본값: `0`

참고

- `--id` 또는 위치 인수로 양수 Flow ID를 지정해야 합니다.

예시

```
tier0 flow get --id 1 --json
```

```
tier0 flow get 1 --json
```

`tier0 flow create`

새 flow 생성

사용법

```
tier0 flow create
```

플래그

`-n, --name`
Flow 이름(필수)
필수

`-t, --type`
Flow 유형: SourceFlow | EventFlow(필수)
필수

`--source`
유형을 SourceFlow로 설정

`--event`
유형을 EventFlow로 설정

`--desc`
설명

`--template`
초기 템플릿 JSON 문자열

`--template-file`
파일에서 초기 템플릿 읽기

`--dry-run`
실행하지 않고 요청 미리보기

참고

- Flow 유형을 선택하려면 `--type`, `--source`, `--event` 중 정확히 하나만 사용해야 합니다.
- `--template`과 `--template-file`은 함께 사용할 수 없습니다.

예시

```
tier0 flow create --name "modbus-collector" --source --desc "Modbus TCP
collector"
```

```
tier0 flow create --name "alert-handler" --event --desc "Temperature
alarm processor"
```

`tier0 flow update`

flow 메타데이터 업데이트

사용법

```
tier0 flow update --id <id>
```

```
tier0 flow update <id>
```

플래그

`--id`
Flow ID(필수)
기본값: `0`

`-n, --name`
새 이름

`--desc`
새 설명

`--template`
새 템플릿 JSON 문자열

`--template-file`
파일에서 새 템플릿 읽기

`--favorite`
즐겨찾기로 표시

`--unfavorite`
즐겨찾기에서 제거

`--dry-run`
실행하지 않고 요청 미리보기

참고

- `--id` 또는 위치 인수로 양수 Flow ID를 지정해야 합니다.
- `--template` 과 `--template-file` 은 함께 사용할 수 없습니다.

- `--favorite`과 `--unfavorite`은 함께 사용할 수 없습니다.
- 업데이트할 필드를 하나 이상 지정해야 합니다.

예시

```
tier0 flow update --id 1 --name "line1-collector"
```

```
tier0 flow update --id 1 --favorite
```

`tier0 flow data`

Node-RED canvas JSON 가져오기

사용법

```
tier0 flow data --id <id>
```

```
tier0 flow data <id>
```

플래그

`--id`
Flow ID
기본값: `0`

`-o, --out`
출력을 파일에 저장

참고

- `--id` 또는 위치 인수로 양수 Flow ID를 지정해야 합니다.

예시

```
tier0 flow data --id 1 --out flows.json
```

`tier0 flow deploy`

Node-RED canvas JSON 배포

사용법

```
tier0 flow deploy --id <id>
```

```
tier0 flow deploy <id>
```

플래그

`--id`
Flow ID(필수)
기본값: `0`

`--flows-json`
Node-RED canvas JSON 문자열

`-f, --flows-file`
파일에서 Node-RED canvas JSON 읽기(권장)

`-y, --yes`
고위험 작업 확인(필수)
필수

`--dry-run`
실행하지 않고 요청 미리보기

참고

- `--id` 또는 위치 인수로 양수 Flow ID를 지정해야 합니다.
- `--flows-json`과 `--flows-file`은 함께 사용할 수 없습니다.
- 배포하면 해당 Flow의 기존 Node-RED 노드가 모두 교체됩니다.

- `--dry-run`을 사용하지 않는 한 고위험 작업에는 `--yes`가 필요합니다.

예시

```
tier0 flow deploy --id 1 -f flows.json --dry-run --json
```

```
tier0 flow deploy --id 1 -f flows.json --yes
```

`tier0 flow delete`

flow 삭제

별칭: del, rm

사용법

```
tier0 flow delete --id <id>
```

```
tier0 flow delete <id>[,<id>...]
```

플래그

`--id`
삭제할 Flow ID(반복 가능)
반복 가능

`-y, --yes`
고위험 작업 확인(필수)
필수

`--dry-run`
실행하지 않고 요청 미리보기

참고

- 위치 인수에는 쉼표로 구분된 Flow ID가 포함될 수 있습니다.
- 삭제하면 Node-RED 컨테이너가 중지되며 되돌릴 수 없습니다.
- `--dry-run`을 사용하지 않는 한 고위험 작업에는 `--yes`가 필요합니다.

예시

```
tier0 flow delete --id 1 --yes
```

`tier0 flow nodes`

사용 가능한 Node-RED 노드 유형 목록 표시

사용법

```
tier0 flow nodes [source|event]
```

```
tier0 flow nodes --type <SourceFlow|EventFlow>
```

```
tier0 flow nodes --source
```

```
tier0 flow nodes --event
```

플래그

`-t, --type`
Flow 유형(SourceFlow/EventFlow)

`--source`
SourceFlow 노드 표시

`--event`
EventFlow 노드 표시

참고

- 위치 인수 Flow 유형은 최대 하나만 허용됩니다.
- `--source`와 `--event`는 함께 사용할 수 없습니다.
- 위치/유형 입력을 `--source` 또는 `--event`와 함께 사용하지 마세요.

예시

```
tier0 flow nodes source --json
```

```
tier0 flow nodes --source --json
```

```
tier0 flow nodes --event --json
```

파일

`tier0 assets upload`

Tier0 object storage에 파일 업로드

사용법

```
tier0 assets upload <local-file>
```

플래그

`--business`
비즈니스 장면

기본값: `attachment`

`--use-by`

사용 범위: user|workspace|platform
기본값: `workspace`

`--visibility`

가시성: public|private
기본값: `private`

`--app-instance-id`

AI 앱 인스턴스 ID

`--session-id`

AI 세션 ID

`--multipart-size`

멀티파트 파트 크기(예: 10MB, 최소 5MB)
기본값: `CLI-defined default`

`--concurrency`

멀티파트 업로드 동시성(병렬 파트 수)
기본값: `4`

`--resume`

중단된 멀티파트 업로드 재개

`--abort`

중단된 멀티파트 업로드를 중단하고 상태 정리

참고

- 로컬 파일 인수는 정확히 하나만 필요합니다.
- 100MB보다 큰 파일은 자동으로 멀티파트 업로드를 사용합니다.
- CLI는 엄격한 최대 파일 크기를 강제하지 않으며, 서버가 플랜/할당량 제한을 적용합니다.

예시

```
tier0 assets upload ./report.csv
```

```
tier0 assets upload ./report.csv --visibility public --business
attachment
```

```
tier0 assets upload ./report.csv --use-by workspace --visibility private
```

```
tier0 assets upload ./large-backup.tar.gz
```

```
tier0 assets upload ./large-backup.tar.gz --resume
```

```
tier0 assets upload ./large-backup.tar.gz --abort
```

```
tier0 assets upload ./large-backup.tar.gz --multipart-size 20MB --
concurrency 8
```

`tier0 assets download`

Tier0 object storage에서 파일 다운로드

사용법

```
tier0 assets download
```

플래그

`--file-path`
업로드에서 반환된 파일 경로
필수

`-o, --output`

출력 파일 경로(기본값: stdout)

`--content-disposition`
사용자 지정 Content-Disposition

예시

```
tier0 assets download --file-path workspace/.../report.csv -o
./report.csv
```

```
tier0 assets download --file-path workspace/.../report.csv
```

`tier0 assets url`

Tier0 object storage 파일 접근 URL 가져오기

사용법

```
tier0 assets url
```

플래그

`--file-path`
업로드에서 반환된 파일 경로
필수

`--expired-sec`
사전 서명 URL 만료 시간(초)
기본값: `3600`

`--content-disposition`
사용자 지정 Content-Disposition

예시

```
tier0 assets url --file-path workspace/.../report.csv
```

```
tier0 assets url --file-path workspace/.../report.csv --expired-sec 300
```

`tier0 assets delete`

Tier0 object storage 파일 삭제

사용법

```
tier0 assets delete
```

플래그

`--file-path`
업로드에서 반환된 파일 경로
필수

`-y, --yes`
고위험 작업 확인(필수)
필수

`--dry-run`
실행하지 않고 요청 미리보기

참고

- 삭제는 되돌릴 수 없습니다.
- `--dry-run`을 사용하지 않는 한 고위험 작업에는 `--yes`가 필요합니다.

예시

```
tier0 assets delete --file-path workspace/.../report.csv --dry-run --
json
```

```
tier0 assets delete --file-path workspace/.../report.csv --yes
```

참조

내장 Builder Skills

Tier0 Builder에 내장된 AI Skills 참조입니다.

Tier0 App Builder에는 현재 7개의 active 내장 AI Skills가 포함되어 있습니다. 이 Skills는 agent가 요구사항을 산업용 애플리케이션으로 전환하고, 애플리케이션을 Tier0 platform scaffold에 맞추며, UI 문제를 복구하고, preview 환경을 안정적으로 유지하도록 돕습니다.

`app-srs`

의미: 새로운 산업용 enterprise app의 requirements analysis entry point입니다. WMS, MES, ERP, CRM, QMS, EAM, 승인, 재고, 공급업체, 고객, 스케줄링 및 유사한 enterprise workflow에 사용합니다.

예시:

```
$app-srs를 사용해 자동차 부품 공장을 위한 품질 예외 관리 시스템을 설계해 주세요.
```

자연어로도 트리거할 수 있습니다:

```
입고, 적치, 피킹, 출고, 재고 실사를 포함한 창고 관리 앱을 생성해 주세요.
```

수행 작업:

1. 비즈니스 역할, workflow, 상태, 예외를 분석합니다.
2. 요구사항이 충분히 명확한지 확인합니다.
3. 완전한 SRS를 생성합니다.
4. 기본적으로 `specs/spec.md` 에 저장합니다.
5. 사용자가 요구사항만 요청하지 않는 한 기본적으로 결과를 `app-builder` 에 전달합니다.

`app-builder`

의미: 실제로 앱을 구축하는 핵심 Skill입니다.

예시:

```
$app-builder를 사용해 specs/spec.md에서 첫 번째 버전을 구축해 주세요.
```

내부 작업 유형: agent는 prompt와 현재 project state를 기준으로 사용할 work type을 결정합니다.

- **First Release**: SRS에서 첫 번째 버전을 구축합니다.
- **Delta Feature**: 기존 앱에 큰 module 또는 workflow를 추가합니다.
- **Light Edit**: 전체 build flow를 실행하지 않고 작은 field, copy, style, validation 변경을 처리합니다.

확인 항목:

- 보이는 진입점.
- 실행 가능한 사용자 경로.
- 데이터 또는 상태 변화.
- 역할 권한.
- build 및 preview 검증.

> 팁
> 첫 build에 `specs/spec.md`가 없으면 agent가 먼저 `app-srs`로 생성합니다.

`align-platform-app`

의미: 기존 앱을 Tier0 platform scaffold에 맞추는 migration skill입니다.

```
$align-platform-app를 사용해 이 오래된 Vite React project를 Tier0 platform
scaffold로 마이그레이션해 주세요.
```

적합한 **source**:

- Vite, CRA, Next.js, React Router, static HTML, full-stack Node/React apps.
- 이전 Tier0 apps.
- LocalStorage, IndexedDB, Zustand persist, in-memory mocks, simulated data에 의존하는 apps.

수행 작업:

- 앱을 Tier0 TanStack Start scaffold에 맞춥니다.
- 영속 비즈니스 데이터를 Drizzle/PostgreSQL로 마이그레이션합니다.
- routes, APIs, services, database setup을 정렬합니다.
- 앱 자체 login, logout, role-switching flows를 제거합니다.
- Gateway가 관리하는 identity와 roles에 연결합니다.
- `/api/health`, `/api/manifest`, `artifact.toml` 같은 platform contracts를 유지합니다.
- preview 및 production startup을 검증합니다.

`migrate-node-red-app`

의미: Node-RED application을 Tier0 platform scaffold에 맞추는 Node-RED migration skill입니다. 보통 `align-platform-app`와 함께 사용합니다.

예시:

```
$align-platform-app와 $migrate-node-red-app를 사용해 이 Node-RED backend를 Tier0 TanStack Start app으로 마이그레이션해 주세요.
```

수행 작업:

- flows, subflows, function nodes, change nodes, switch nodes를 인벤토리화합니다.
- HTTP, MQTT, database, scheduled jobs, external API calls를 분석합니다.
- migration mapping table을 만듭니다.
- CRUD, workflow, scheduling, integration, temporary simulation behavior를 분리합니다.
- business logic을 services로 옮깁니다.

- interfaces를 API routes로 옮깁니다.
- state를 PostgreSQL로 옮깁니다.
- long-running work와 integrations를 workers, queues 또는 external services로 옮깁니다.
- browser, CRUD, workflow, integration cutover를 별도로 검증합니다.

> ⚠ 주의
> 일반적인 preview 또는 packaging 문제는 단독으로 처리하지 않습니다.

`ui-repair`

의미: 기존 앱을 위한 repair skill입니다. 최초 app generation이 아니라 UI quality issues에 사용합니다.

예시:

```
$ui-repair를 사용해 작은 화면에서 주문 편집 dialog의 overflow와 form misalignment를 수정해 주세요.
```

적합한 문제:

- form 또는 required marker 정렬 불량.
- dialog 또는 drawer overflow.
- 의도하지 않은 horizontal scrolling.
- select menu 또는 dropdown clipping.
- table 또는 chart layout 깨짐.
- visual hierarchy가 약한 flat page.
- UI language 혼재.
- PDA, kiosk, dashboard, industrial interface layout 문제.

> ⚠ 주의

이 Skill은 가장 작은 owning component를 찾고 영향을 받은 surface를 수정해야 하며, 전체 앱을 다시 구축하지 않습니다. 복잡한 responsive work에는 `responsive-craft`를 사용하세요.

`responsive-craft`

의미: responsive layout audit, 신규 responsive build, multi-breakpoint preview를 위한 skill입니다.

호출 모드:

- **audit**: 기존 responsive layout을 audit하고 복구합니다.
- **build**: responsive page를 처음부터 구축합니다.
- **preview**: multi-breakpoint live preview를 시작합니다.

예시:

```
$responsive-craft audit를 사용해 이 dashboard의 mobile layout을 수정해 주세요.
```

```
$responsive-craft build를 사용해 phone, tablet, desktop용 inventory workspace를 만들어 주세요.
```

```
$responsive-craft preview를 사용해 375, 768, 1024, 1440 px breakpoints를 표시해 주세요.
```

실행 스타일:

- **Adaptive**: 빠르게 진행하고 design fork가 나타나면 확인합니다.
- **Guided**: 구현 전에 각 component의 responsive behavior spec을 작성합니다.

`preview-runtime-stability`

의미: app development 중 preview failure를 해결하기 위한 runtime troubleshooting skill입니다. business feature를 구축하지 않습니다.

예시:

```
$preview-runtime-stability를 사용해 "Preview loading timed out"을 진단해 주세요.
```

적합한 문제:

- 누락되었거나 부분적인 `node_modules`.
- npm `ERESOLVE`.
- Vite 또는 TanStack Start startup failures.
- port `5173` 충돌.
- `/api/health` 실패.
- SDK SSR module errors.
- `artifact.toml` preview port mismatch.
- HMR 중 frequent full-page reloads.
- 종료되지 않은 stale Node 또는 Vite processes.

표준 흐름:

1. dependency installation을 확인합니다.
2. port `5173`을 확인합니다.
3. health endpoint를 probe합니다.
4. healthy preview는 재사용합니다.
5. unhealthy preview는 정리합니다.
6. restart 후 port, health endpoint, HMR behavior를 검증합니다.

Skill 선택 가이드

- 요구사항이 불명확하거나 durable specification이 필요하면 먼저 `app-srs`를 사용합니다.
- implementation scope가 이미 명확하면 `app-builder`를 사용합니다.

- migration work에는 `align-platform-app`와 `migrate-node-red-app`를 사용합니다.
- 앱이 이미 있고 문제가 visual, responsive, runtime 관련이면 `ui-repair`, `responsive-craft`, `preview-runtime-stability`를 사용합니다.

참조

사용자 관리

Tier0는 멤버 자격, 라이선스 및 사용자 지정 계정을 통해 고객이 플랫폼 접근과 사용자 권한을 관리할 수 있도록 합니다.

Tier0는 에디션에 따라 서로 다른 방식으로 사용자 권한과 접근을 관리합니다.

- Cloud는 Workspace 멤버 초대를 통해 Workspace에 접근할 수 있는 사용자를 관리합니다.
- Enterprise에서는 관리자가 사용자 계정과 역할을 할당하여 접근 가능한 기능을 결정합니다.

접근 제어

탭: CLOUD

- Workspace 소유자 또는 기존 멤버는 전화번호나 이메일 주소를 통해 새 멤버를 초대할 수 있습니다.
- 새 멤버는 초대 코드 또는 링크를 받은 후 계정을 등록하고 Workspace에 참여합니다.
- 초대할 때 새 멤버의 시트 유형(Operator 또는 Builder)을 설정하여 접근할 수 있는 기능 범위를 결정합니다.
    - **Builder**: 모든 기능 모듈에 접근할 수 있습니다.
    - **Operator**: 애플리케이션 센터에만 접근할 수 있습니다.
- 계정 등록에는 2단계 확인이 필요합니다.

> ⚠ 주의
> Workspace 멤버 자격만으로 플랫폼 접근 권한이 자동으로 부여되지는 않습니다. 플랫폼을 사용하려면 초대를 받은 전화번호 또는 이메일 주소로 계정을 등록해야 합니다.

> ⓘ 참고
> 계정을 등록한 후 새 Workspace를 신청할 수도 있습니다.

탭: ENTERPRISE

- 관리자는 기본 계정으로 처음 로그인한 후 비밀번호를 변경하고, **설정**에서 사용자 계정을 추가하고 역할을 할당합니다.
- 관리자는 계정을 할당하여 플랫폼을 사용할 수 있는 사용자를 결정하고, 역할을 할당하여 각 계정이 접근할 수 있는 기능을 결정합니다.
    - **Admin**: 모든 기능 모듈에 접근할 수 있습니다.
    - **Operator**: 접근 가능한 모든 애플리케이션이 나열된 **Launchpad**에만 접근할 수 있습니다.

> ⓘ 참고
> 관리자가 생성한 계정으로 로그인하고, 할당된 역할의 권한 범위 내에서 기능을 사용합니다.

계정 관리

탭: CLOUD

작업	수행자	설명
시트 유형 변경	생성자 / Builder	• 생성자는 모든 멤버의 시트 유형을 변경할 수 있습니다. • Builder는 Operator의 시트 유형만 변경할 수 있습니다.
Workspace 삭제	생성자	Workspace를 삭제하면 모든 데이터가 즉시 삭제되며 복구할 수 없습니다.
Workspace 이전	생성자	—
멤버 제거	생성자 / Builder	• 생성자는 모든 멤버를 제거할 수 있습니다. • Builder는 Operator만 제거할 수 있습니

작업	수행자	설명
		다.
계정 삭제	모든 멤버	• 생성자는 먼저 Workspace를 삭제하거나 다른 멤버에게 이전해야 합니다. • 멤버 계정은 즉시 모든 Workspace에서 탈퇴합니다. • 계정 데이터는 즉시 삭제되며 복구할 수 없습니다.

탭: ENTERPRISE

- Admin은 비밀번호를 재설정하고, 사용자 역할을 수정하고, 계정을 비활성화하거나 삭제할 수 있습니다.
- 최초 기본 계정의 역할은 수정할 수 없으며, 해당 계정을 비활성화하거나 삭제할 수도 없습니다.
- 계정을 삭제하면 모든 데이터가 즉시 삭제되며 복구할 수 없습니다.

참조

Tier0 Cloud 네트워크 및 데이터 보안 안내

Tier0 Cloud 프로덕션 SaaS 서비스의 네트워크 경계, 테넌트 격리, 데이터 보호 및 보안 운영 조치를 설명합니다.

적용 범위: AWS 싱가포르 리전에 배포된 Tier0 Cloud 프로덕션 SaaS 서비스.
문서 날짜: 2026년 10월 22일

Tier0 Cloud는 네트워크 접근 제어, 테넌트 격리, ID 및 접근 관리, 데이터 암호화, 백업 및 복구를 통해 네트워크 공격과 무단 접근의 위험을 줄입니다. 이 문서는 플랫폼의 보안 조치와 보안 사고가 다른 테넌트, 플랫폼 내부 서비스 및 고객 네트워크로 확산되는 것을 제한하는 통제 경계를 설명합니다.

> ⓘ **Edge** 에디션
> 이 안내는 Tier0 Cloud에만 적용됩니다. Edge 설정은 설치를 참조하세요.

1 네트워크 경계 및 접근 제어

Tier0 Cloud의 공개 Web 및 API 접근에는 TLS 암호화를 사용하며, 통합된 엣지 진입점과 애플리케이션 게이트웨이를 거칩니다. 핵심 서비스 진입점은 원본 서버 접근을 검증하고, 검증 요건을 충족하지 않는 직접 접근 요청을 거부하여 엣지 진입점을 우회한 원본 서버 접근 위험을 줄입니다.

프로덕션 환경은 클라우드 계정, 네트워크, 클러스터 및 데이터베이스 계층에서 개발·스테이징 환경과 격리되며, 접근 자격 증명도 별도로 관리합니다. 프로덕션 관리 접근에는 인증된 전용 네트워크 경로를 사용합니다.

2 테넌트 격리 및 사고 확산 방지

2.1 워크로드 격리

테넌트 워크로드는 별도의 Kubernetes 네임스페이스에서 실행되며, 전용 노드 풀과 스케줄링 제약을 통해 플랫폼 서비스 및 관리 구성 요소와 분리됩니다. 이 방식은 테넌트 애플리케이션과 플랫폼 구성 요소 사이의 직접적인 상호 작용 범위를 제한하여, 특정 애플리케이션에서 보안 사고가 발생했을 때 다른 서비스에 미치는 영향을 줄입니다.

2.2 네트워크 격리

테넌트 네임스페이스의 인바운드 및 아웃바운드 통신에는 기본적으로 거부하고 명시적인 규칙으로만 허용하는 네트워크 정책을 적용합니다. 플랫폼은 정책을 자동으로 구성하고 동기화하며, 클러스터 네트워크 구성 요소가 이를 적용합니다.

인바운드 통신은 동일 테넌트의 워크로드와 필수 업무 기능을 수행하는 플랫폼 서비스에서 오는 통신만 허용합니다. 아웃바운드 통신은 규칙에 따라 클러스터 DNS 이름 확인, 동일 테넌트의 서비스, 지정된 플랫폼 게이트웨이, 메시징 서비스 및 할당된 데이터베이스 엔드포인트에 대해 허용합니다. 공용 인터넷 접근도 네트워크 정책의 제약을 받습니다. 명시적으로 허용된 업무 의존 대상을 제외하고 사설 네트워크 주소 접근을 제한하며, 클라우드 인스턴스 메타데이터 주소에 대한 직접 접근을 차단합니다.

이러한 조치는 승인되지 않은 테넌트 간 연결과 테넌트 워크로드에서 플랫폼 내부 네트워크로의 측면 이동을 제한합니다. 특정 테넌트 애플리케이션에서 보안 사고가 발생하더라도 그 사실만으로 다른 테넌트나 플랫폼 관리 구성 요소에 접근할 수 있는 권한이 부여되지는 않습니다. 이미 승인된 업무 연결에도 해당 네트워크 규칙과 ID 및 접근 제어가 계속 적용됩니다.

2.3 데이터 및 ID 격리

테넌트 애플리케이션 데이터는 데이터베이스 계층에서 논리적으로 격리됩니다. 별도의 데이터베이스 단위가 할당되는 경우에는 네트워크 접근 규칙을 통해 연결 출발지도 제한합니다.

플랫폼은 서비스 역할에 따라 접근 권한을 설정하고 각 워크로드의 리소스 접근 범위를 필요한 기능으로 제한합니다. 네트워크 격리와 ID 기반 인가를 함께 적용하여 영향을 받은 워크로드가 접근할 수 있는 리소스 범위를 제한합니다.

2.4 고객 네트워크 접근 경계

이 절의 테넌트 격리 조치는 Tier0 Cloud 플랫폼 내부에 적용됩니다. 고객이 커넥터, 게이트웨이 또는 기타 통합 방식으로 내부 시스템을 연결하는 경우, 고객 네트워크의 접근 경계는 실제 연결 방식, 라우팅, 방화벽 규칙 및 사용된 자격 증명에 부여된 권한에도 영향을 받습니다.

고객은 업무에 필요한 대상 주소, 포트 및 계정 권한만 허용하고, 연결 구성 요소를 다른 내부 시스템과 네트워크상에서 분리해야 합니다. 플랫폼 내부 격리는 고객 측 네트워크 접근 제어를 대체할 수 없습니다. 양측은 구체적인 연결 구성에 따라 접근 범위를 정하고 검증하여, 승인된 연결을 통해 고객의 다른 시스템으로 사고가 확산될 위험을 제한해야 합니다.

3 데이터 보호

플랫폼은 데이터 전송, 저장 및 접근에 다음 보호 조치를 적용합니다.

- 공개 Web 및 API 통신과 관리형 데이터베이스 및 캐시 접근에 TLS 암호화를 사용합니다.
- 관리형 데이터베이스, 캐시 및 객체 스토리지에는 저장 데이터 암호화를 활성화합니다.
- 객체 스토리지는 공개 접근을 차단하며, 암호화 요건, HTTPS 접근 요건 및 리소스 권한을 통해 데이터 접근을 제한합니다.
- 테넌트 데이터 접근에는 데이터베이스 격리와 ID 및 접근 제어가 모두 적용됩니다.

4 백업 및 복구

Tier0 Cloud는 프로덕션 데이터베이스와 백업 범위에 포함된 서비스별 데이터베이스를 백업합니다. 백업 데이터는 접근이 제한된 전용 객체 스토리지에 저장하며, 해당 보존 정책에 따라 관리합니다.

백업 작업은 별도의 ID를 사용하며, 지정된 저장 위치에서 필요한 읽기 및 쓰기 작업만 허용합니다. 백업 객체를 삭제할 권한은 부여하지 않습니다. 보존 기간이 지난 백업은 스토리지 수명 주기 규칙으로 삭제하여, 백업 자격 증명이 유출되었을 때 복구 데이터 삭제에 악용될 위험을 줄입니다.

백업이 완료되면 무결성을 확인하고, 격리된 환경에서 복원 검증을 수행하여 백업의 사용 가능 여부를 확인합니다. 애플리케이션 구성은 버전 관리하며, 빌드 산출물은 비공개 컨테이너 이미지 레지스트리에 저장하여 서비스 복구와 버전 롤백을 지원합니다.

5 관리 접근 및 프로덕션 변경

프로덕션 관리 접근에는 개인별로 할당된 ID를 사용하고 최소 권한 원칙에 따라 권한을 부여합니다. 민감한 작업에 사용하는 콘솔 ID에는 다중 요소 인증을 의무화합니다.

일상 점검에는 워크로드 상태, 로그 및 이벤트를 확인할 수 있는 제한된 읽기 전용 ID를 사용합니다. 이 ID로는 민감한 자격 증명 읽기, 리소스 수정 또는 원격 명령 실행을 할 수 없습니다. 프로덕션 변경에는 별도로 승인된 ID와 절차를 사용합니다.

인프라 구성, 배포 매니페스트 및 운영 스크립트는 버전 관리합니다. 프로덕션 변경 시에는 목표, 영향 범위, 검증 단계 및 롤백 계획을 명확히 하고, 실행 ID, 구성 차이 및 검증 결과를 기록하여 추적과 복구를 지원합니다.

6 모니터링 및 보안 사고 대응

플랫폼은 프로덕션 환경의 운영 지표, 로그 및 상태 정보를 중앙에서 수집하고, 이상 알림을 운영 대응 채널로 전송합니다. 모니터링 범위에는 서비스 가용성, 오류율, 데이터베이스 상태, 인증서

유효성, 백업 작업 상태 및 클러스터 운영 상태가 포함됩니다.

보안 사고가 발생하면 영향 범위의 확산을 막는 데 우선순위를 두고 관련 로그, 구성 및 사고 타임라인을 보존합니다. 필요한 격리와 조사를 완료한 후 서비스를 복구하고, 이후 사고 검토와 시정 조치 추적을 수행합니다. 사고 대응 중의 프로덕션 변경도 정해진 절차에 따라 승인하고 기록합니다.

7 보안 문의

보안 평가, 고객 실사 또는 의심되는 취약점에 관해서는 Tier0 고객 담당자나 보안팀에 문의하세요. 비공개 아키텍처 및 통제 세부 정보는 적절한 비밀 유지 약정에 따라 제공될 수 있습니다. 최초 문의에는 접근 자격 증명, 개인 키 또는 고객의 민감한 데이터를 포함하지 마세요.

참조

ISA-95 설비 계층

ISA-95로 물리 자산과 제어 시스템을 모델링하기 위한 참조 계층입니다.

ISA-95 설비 계층은 조직 내 물리 자산과 제어 시스템이 어떻게 구조화되는지 설명합니다.

다음 계층으로 나뉩니다.

- **Enterprise**: 조직 전체를 나타내는 최상위 계층입니다. 전체 비즈니스 프로세스와 의사결정 주체를 포함하며, 일반적으로 ERP 시스템의 지원을 받습니다.
- **Site**: 특정 지리적 위치나 시설을 나타냅니다. 각 site는 자체 관리 시스템과 로컬 생산 제어를 가질 수 있습니다.
- **Area**: site 내의 운영 구역으로, 포장, 조립, 창고와 같은 기능 영역인 경우가 많습니다.
- **Process Cell**: area 내 제조의 기본 단위입니다. 특정 제품을 생산하거나 특정 기능을 수행하는 데 필요한 모든 설비를 포함합니다.
- **Unit**: process cell의 독립적인 부분으로, 혼합, 가열, 조립 같은 특정 기능을 담당합니다.
- **Equipment Module**: unit 내에서 특정 작업을 수행하는 모듈형 설비입니다. 예를 들어 혼합 unit에서는 가열 요소가 equipment module이 될 수 있습니다.
- **Control Module**: 최하위 계층으로, 센서, 펌프, 액추에이터, PID 제어 루프처럼 물리 프로세스를 직접 제어하는 개별 장치나 컨트롤러를 나타냅니다.

참조

표준 배포 계획

Tier0 Enterprise의 가상 머신 4대 기반 표준 배포 참조 구성입니다.

목표

간소화 구성은 Docker Swarm을 사용하지 않고 단일 `enterprise` 애플리케이션 노드를 배포합니다. 가상 머신은 총 `4` 대입니다.

- `1` 대의 enterprise 노드에서 `tier0-enterprise + EMQX` 를 실행합니다.
- `2` 대의 데이터베이스 노드로 PostgreSQL + TimescaleDB 기본/대기 클러스터를 구성합니다.
- `1` 대의 Witness 노드가 데이터베이스 감시, 쿼럼 및 장애 조치에 참여합니다.

간소화 구성은 배포 복잡성을 줄입니다. 대신 애플리케이션 계층과 EMQX는 단일 장애 지점이 됩니다. 데이터베이스 계층만 `DB VIP + Witness` 를 통한 고가용성을 유지합니다.

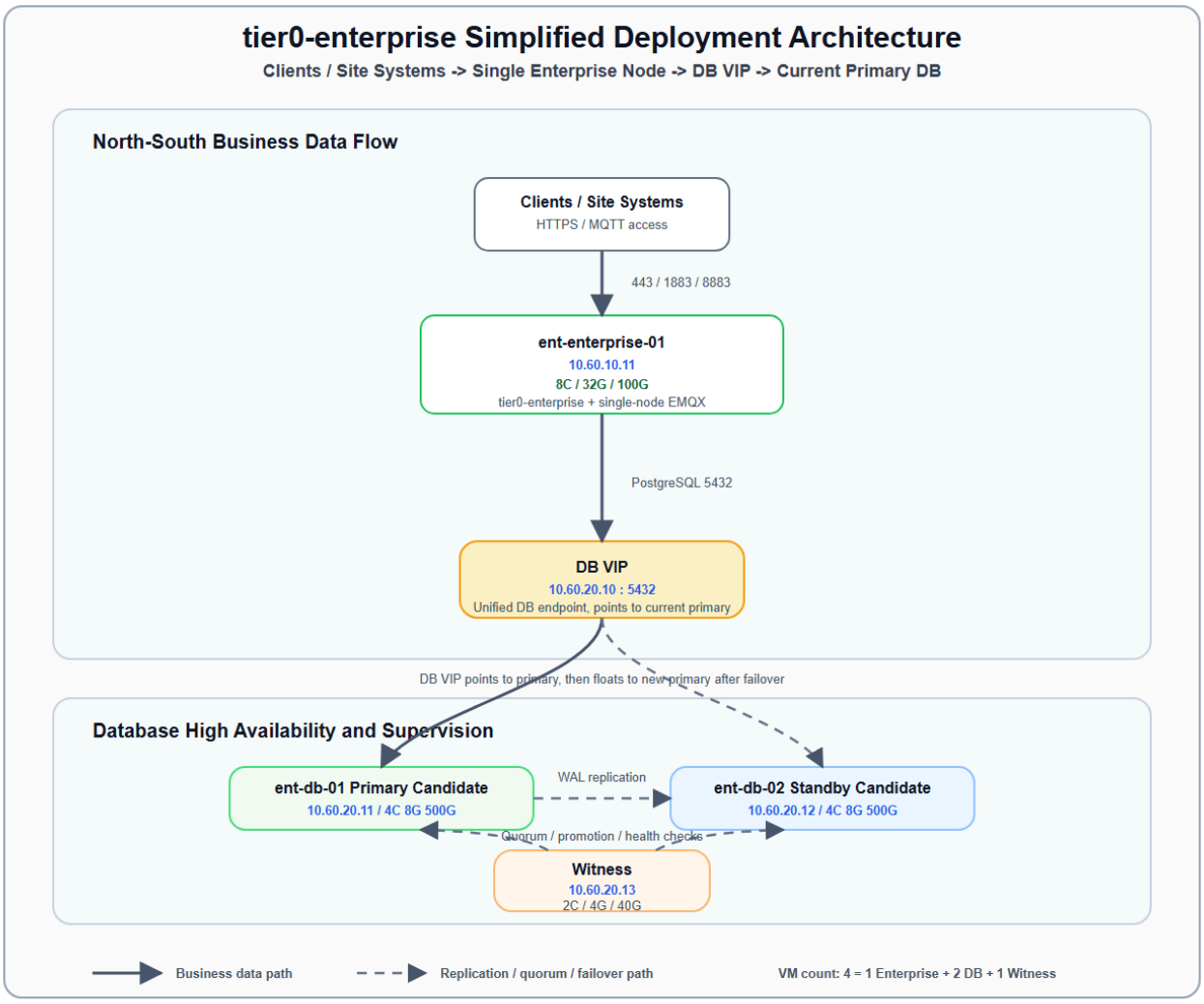
IP 주소 계획

다음 IP 주소는 계획용 예시입니다. 구현 전에 실제 현장 네트워크에 맞게 바꾸세요.

유형	이름	계획 IP	설명
애플리케이션 노드	`ent-enterprise-01`	`10.60.10.11`	`tier0-enterprise + EMQX` 를 실행하는 단일 enterprise 노드
논리 진입점	`DB VIP`	`10.60.20.10`	현재 기본 노드만 가리키는 통합 데이터베이스 엔드포인트
데이터베이스 노드	`ent-db-01`	`10.60.20.11`	PostgreSQL + TimescaleDB 기본 노드 후보

유형	이름	계획 IP	설명
데이터베이스 노드	`ent-db-02`	`10.60.20.12`	PostgreSQL + TimescaleDB 대기 노드 후보
쿼럼 노드	`ent-db-witness-01`	`10.60.20.13`	`repmgr witness` / 데이터베이스 쿼럼 감시

전체 토폴로지



주요 경로:

- 업무 경로: 클라이언트 / 현장 시스템 → `ent-enterprise-01` → `DB VIP` → 현재 기본 데이터베이스.
- `tier0-enterprise`와 `EMQX`는 모두 `ent-enterprise-01`에서 실행합니다.

- 데이터베이스 접근에는 계속 `DB VIP`를 사용하며, 애플리케이션에서 `ent-db-01/02`로 직접 접근하는 것은 금지합니다.
- `Witness`는 데이터베이스 쿼럼, 승격 및 상태 확인에만 참여하며 업무 경로에는 포함되지 않습니다.

데이터베이스 감시 및 장애 조치

단계	경로	설명
정상 접근	`ent-enterprise-01 -> DB VIP -> current primary`	애플리케이션은 항상 `10.60.20.10:5432`에 연결합니다
정상 복제	`current primary -> current standby`	PostgreSQL WAL 스트리밍 복제
정상 감시	`DB-01/DB-02 -> Witness`	`repmgrd`가 상태를 보고하고 Witness가 쿼럼에 참여합니다
기본 노드 장애	`standby -> Witness`	대기 노드가 쿼럼 확인을 요청하여 스플릿 브레인을 방지합니다
대기 노드 승격	`standby promote`	Witness 확인 후 대기 노드를 승격합니다
VIP 이동	`DB VIP -> new primary`	`10.60.20.10`을 새 기본 노드에 할당합니다
애플리케이션 복구	`ent-enterprise-01 -> DB VIP -> new primary`	연결 풀이 다시 연결되면 애플리케이션이 복구됩니다

노드 구성

노드	역할	vCPU	메모리	스토리지	권장 시스템 디스크	권장 데이터 디스크	주요 구성 요소
`ent-enterprise-01`	단일 enterprise 노드	`8`	`32G`	`100G`	단일 `/` 시스템 디스크, 100G	별도 데이터 디스크 없음	tier0-enterprise, EMQX, 선택 사항으로 Nginx/HAProxy
`ent-db-01`	PostgreSQL/TSDB 기본 노드 후보	`4`	`8G`	`500G`	`/` 80G	`/pgdata` 360G, `/pgwal` 40G, `/backup-staging` 20G	PostgreSQL, TimescaleDB, repmgr, keepalived
`ent-db-02`	PostgreSQL/TSDB 대기 노드 후보	`4`	`8G`	`500G`	`/` 80G	`/pgdata` 360G, `/pgwal` 40G, `/backup-staging` 20G	PostgreSQL, TimescaleDB, repmgr, keepalived

노드	역할	vCPU	메모리	스토리지	권장 시스템 디스크	권장 데이터 디스크	주요 구성 요소
`ent-db-witness-01`	Witness / 쿼럼 감시	`2`	`4G`	`40G`	`/` 40G	별도 계획 없음	repmgr witness, 모니터링 에이전트, 감사 로그

구성 요약

분류	수량	노드별 사양	vCPU 소계	메모리 소계	스토리지 소계
enterprise 애플리케이션 노드	`1`	`8C / 32G / 100G`	`8C`	`32G`	`100G`
PostgreSQL + TimescaleDB 노드	`2`	`4C / 8G / 500G`	`8C`	`16G`	`1000G`
Witness 노드	`1`	`2C / 4G / 40G`	`2C`	`4G`	`40G`
합계	`4`	-	`18C`	`52G`	`1140G`

총합:

- 가상 머신: `4` 대
- vCPU: `18`
- 메모리: `52G`
- 스토리지: `1.14T`

연결 매트릭스

운영 접근

대상	방법	포트	설명
모든 가상 머신	`ssh -i ~/.ssh/chen-rsa ops@<IP>`	`22/TCP`	통합 키 기반 SSH 를 구현한 후 사용합니다. 그전에는 현장 계정을 사용합니다
`ent-enterprise-01`	`https://10.60.10.11`	`443/TCP`	업무 진입점
`DB VIP`	`psql -h 10.60.20.10 -p 5432`	`5432/TCP`	enterprise 노드와 운영 허용 목록만 허용합니다

애플리케이션 및 **EMQX** 포트

출발지	대상	포트	용도
사용자 / 현장 시스템	`ent-enterprise-01`	`80/TCP`	HTTP 진입점 또는 HTTPS 리디렉션, 선택 사항
사용자 / 현장 시스템	`ent-enterprise-01`	`443/TCP`	HTTPS 업무 진입점
장치 / 현장 시스템	`ent-enterprise-01`	`1883/TCP`	EMQX MQTT 평문 통신, 선택 사항
장치 / 현장 시스템	`ent-enterprise-01`	`8883/TCP`	EMQX MQTT TLS, 권장
장치 / 현장 시스템	`ent-enterprise-01`	`8083/TCP`	MQTT WebSocket, 선택 사항

출발지	대상	포트	용도
장치 / 현장 시스템	`ent-enterprise-01`	`8084/TCP`	MQTT WebSocket TLS, 선택 사항
운영 네트워크	`ent-enterprise-01`	`18083/TCP`	EMQX Dashboard, 운영 허용 목록만 접근
`ent-enterprise-01`	`DB VIP`	`5432/TCP`	PostgreSQL 업무 접근

데이터베이스 및 감시 통신

출발지	대상	포트	용도
`ent-db-01`	`ent-db-02`	`5432/TCP`	PostgreSQL WAL 스트리밍 복제
`ent-db-02`	`ent-db-01`	`5432/TCP`	복구 후 역방향 복제
`ent-db-01/02`	`ent-db-witness-01`	`5432/TCP`	repmgr 메타데이터 / Witness 통신
`ent-db-01`	`ent-db-02`	`VRRP`	DB VIP 이동. 프로토콜 `112`를 허용합니다

배포 권장 사항

enterprise 애플리케이션 노드

- Docker Swarm을 배포하지 않습니다.
- `tier0-enterprise`와 `EMQX`를 동일한 `ent-enterprise-01` 노드에 배포합니다.
- `ent-enterprise-01`은 애플리케이션 계층의 단일 장애 지점입니다. 장애 시 수동으로 복구하거나 백업에서 재구축합니다.
- 이 노드에는 `100G` 시스템 디스크 하나만 있으므로 업무 데이터를 장기간 저장하기에 적합하지 않습니다.

- `tier0-enterprise` 업무 데이터는 PostgreSQL/TimescaleDB에 저장해야 합니다.
- 업로드, 가져오기/내보내기 및 보고서 파일은 로컬에 저장하고 백업할 수 있습니다. 나중에 표준 구성으로 전환하면 해당 파일을 NFS/NAS 또는 객체 스토리지로 이전하세요.

단일 노드 EMQX

- 간소화 구성에서 EMQX는 단일 노드의 상태 저장 서비스입니다.
- 데이터 디렉터리를 `ent-enterprise-01`에 고정합니다. 예: `/data/emqx`.
- 구성, 인증서, 플러그인 및 규칙 엔진 구성을 백업합니다.
- 메시징 계층의 고가용성은 제공하지 않습니다. 이 조건이 요구 사항을 충족하지 못하면 3 노드 EMQX 클러스터를 갖춘 표준 구성을 사용하세요.

PostgreSQL + TimescaleDB

- PostgreSQL: `16` 또는 현장에서 검증된 버전.
- TimescaleDB: 선택한 PostgreSQL 주 버전과 호환되는 안정 버전.
- 복제: 물리 스트리밍 복제.
- 장애 조치: `repmgrd + witness`.
- 데이터베이스 진입점: `DB VIP`.
- 연결 풀: `ent-enterprise-01`에서 PgBouncer를 실행하여 데이터베이스 연결 부하를 줄일 수 있습니다.

Witness

- Witness는 `repmgr witness` 메타데이터와 감시 로직만 저장합니다.
- Witness는 업무 SQL을 처리하지 않습니다.
- Witness 장애가 현재 기본 노드를 중지시키지는 않지만 자동 장애 조치의 신뢰성은 낮아집니다.

포트 및 방화벽 기준

포트 / 프로토콜	허용 범위	용도
`22/TCP`	운영 네트워크에서 가상 머신 4대로 접근	SSH
`80/TCP`	사용자 네트워크에서 `ent-enterprise-01`로 접근	HTTP 리디렉션 또는 내부 평문 진입점
`443/TCP`	사용자 네트워크에서 `ent-enterprise-01`로 접근	HTTPS 업무 진입점
`1883/TCP`	장치 / 현장 시스템에서 `ent-enterprise-01`로 접근	EMQX MQTT 평문 통신, 선택 사항
`8883/TCP`	장치 / 현장 시스템에서 `ent-enterprise-01`로 접근	EMQX MQTT TLS, 권장
`8083/TCP`	장치 / 현장 시스템에서 `ent-enterprise-01`로 접근	EMQX WebSocket, 선택 사항
`8084/TCP`	장치 / 현장 시스템에서 `ent-enterprise-01`로 접근	EMQX WebSocket TLS, 선택 사항
`18083/TCP`	운영 네트워크에서 `ent-enterprise-01`로 접근	EMQX Dashboard, 허용 목록만 접근
`5432/TCP`	enterprise에서 `DB VIP`로 접근; DB/Witness 내부 통신	PostgreSQL / repmgr
`112/VRRP`	데이터베이스 노드 간	DB VIP 이동
`9100/TCP`	모니터링 시스템에서 가상 머신 4대로 접근	node exporter, 선택 사항

고가용성 기능 및 장애 경계

장애 시나리오	예상 동작	업무 영향
`ent-enterprise-01` 장애	애플리케이션과 EMQX가 모두 중지됩니다. 수동 복구 또는 백업을 통한 재구축이 필요합니다	애플리케이션 계층 전체 중단
현재 DB 기본 노드 장애	Witness 쿼럼 확인 후 대기 노드가 승격되고 DB VIP가 이동합니다	수초에서 수분 동안 쓰기가 중단되며 애플리케이션이 다시 연결해야 합니다
DB 대기 노드 장애	기본 노드가 서비스를 계속 제공하고 복제 알림이 발생합니다	직접적인 업무 중단은 없지만 DB 핫 스탠바이를 잃습니다
Witness 장애	현재 기본 노드가 서비스를 계속 제공하지만 자동 장애 조치의 신뢰성이 낮아집니다	직접적인 업무 중단은 없습니다. Witness를 먼저 복구합니다
DB VIP 이동 실패	새 기본 노드가 있더라도 통합 진입점에 접근하지 못할 수 있습니다	업무가 중단되며 VIP를 수동으로 할당해야 합니다

운영 전환 전 점검 목록

점검 항목	통과 기준
가상 머신 4대의 호스트 이름 고정	`hostnamectl` 이 계획과 일치합니다
통합 SSH 정상 작동	운영 단말에서 키 기반 SSH로 가상 머신 4대 모두에 접근할 수 있습니다
시간 동기화	가상 머신 4대 간 시간 차이가 `1s` 미만입니다
방화벽 허용	9절의 포트가 정해진 범위에 따라 허용됩니다

점검 항목	통과 기준
enterprise 서비스 정상	`tier0-enterprise` 상태 확인을 통과합니다
EMQX 상태 검증	EMQX 노드, 수신 대기 포트 및 영구 데이터 디렉터리가 정상입니다
enterprise 디스크 사용량 안전	`/` 사용률이 `70%` 미만이며 로그 및 캐시 정리가 구성되어 있습니다
DB 기본/대기 상태 정상	`repmgr cluster show`에서 기본 노드, 대기 노드 및 Witness가 정상으로 표시됩니다
DB VIP 정상 작동	enterprise 노드가 `10.60.20.10:5432`에 연결할 수 있습니다
DB 기본 노드 장애 조치 훈련	대기 노드 승격과 DB VIP 이동이 가능합니다
백업 복원 훈련	테스트 데이터베이스에 백업을 복원할 수 있습니다

구현 순서

1. 가상 머신 4대를 생성하고 호스트 이름, IP, 시간 동기화 및 SSH를 설정합니다.
2. OS 기본 설정, 방화벽, 마운트 및 커널 매개변수를 초기화합니다.
3. `ent-enterprise-01`에 `tier0-enterprise`를 배포합니다.
4. `ent-enterprise-01`에 단일 노드 EMQX를 배포합니다.
5. PostgreSQL + TimescaleDB 기본/대기 노드를 배포합니다.
6. repmgr witness, repmgrd 및 DB VIP 이동 스크립트를 배포합니다.
7. 연결, EMQX 복구, 데이터베이스 장애 조치, 백업 복원 및 일상 점검을 검증합니다.
8. 운영 Runbook, 자격 증명 보관 및 일상 점검 작업을 정리합니다.

위험 및 미확정 사항

- 간소화 구성에는 애플리케이션 계층의 단일 장애 지점이 있습니다. `ent-enterprise-01`에 장애가 발생하면 전체 업무 서비스를 사용할 수 없습니다.
- EMQX는 단일 노드이며 메시징 계층의 고가용성을 제공하지 않습니다.
- `ent-enterprise-01`에는 `100G` 시스템 디스크 하나만 있습니다. 로그, 캐시, 첨부 파일 및 임시 파일의 사용량을 관리해야 합니다.
- 나중에 애플리케이션 계층이나 EMQX의 고가용성이 필요하면 표준 구성으로 전환합니다.
- 데이터베이스 RPO/RTO는 확정되지 않았습니다. 이 기본/대기 설계는 데이터 손실이 없음을 보장하지 않습니다.
- DB VIP 이동 스크립트는 현장에서 검증해야 합니다. 특히 가상화/네트워크 플랫폼이 VRRP를 허용하는지 확인하세요.

참조

고가용성 배포 계획

Tier0 Enterprise 고가용성 환경을 위한 참조 배포 계획입니다.

계획 목표

이 V2 계획은 오프라인 또는 프라이빗 환경에 논리적인 Tier0 Enterprise Fleet Center를 배포하기 위한 것입니다. 6대의 머신이 비즈니스, 미들웨어, 데이터베이스, 스토리지, Agent 역할을 나누어 담당합니다. 사용자와 Enterprise 내부 서비스는 항상 고정 VIP 주소로 접속합니다. 하나의 머신 또는 하나의 프록시에 장애가 발생하면 Keepalived, HAProxy, repmgr, RustFS, Fleet Agent가 각자의 계층에서 인계를 수행합니다.

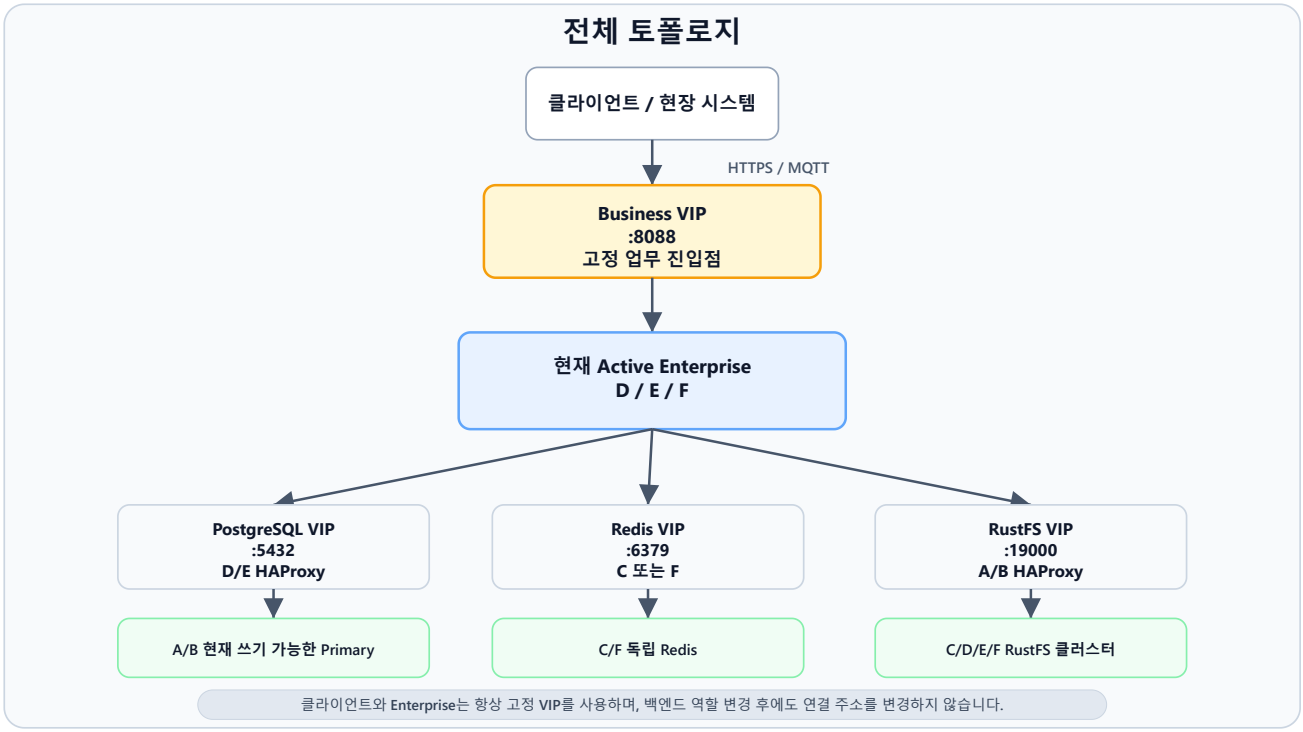
이 계획은 동일 네트워크 세그먼트 안에서 단일 머신 장애를 복구하는 것을 목표로 합니다. 이는 Active/Standby 재해 복구 설계이며, 다중 데이터센터, 합의 기반 선출, 또는 데이터 손실 0을 보장하는 엄격한 고가용성 클러스터가 아닙니다.

설계 목표는 다음과 같습니다.

- 현재 비즈니스 노드가 실패하면 Fleet Center 진입점을 정상 Standby 노드로 이동할 수 있습니다.
- PostgreSQL, Redis, RustFS는 고정 VIP 주소를 사용하므로 backend 역할이 바뀌어도 Enterprise의 `.env`를 변경하지 않습니다.
- App, Flow, Marimo, License와 같은 로컬 비즈니스 파일을 Active에서 Standby로 동기화할 수 있습니다.
- 하나의 Cluster License는 하나의 논리 설치에 매핑되며, Enterprise가 다른 노드로 전환되어도 다시 활성화하지 않습니다.
- 별도의 deployer가 6대 머신 초기화, 역할 할당, 서비스 배포, 상태 모니터링, failover 검증을 수행합니다.
- VIP failover, 파일 동기화, 독립 Redis 인스턴스, 데이터베이스 복제, 객체 스토리지 중복성을 명확히 분리합니다.

이 계획은 Fleet Center에만 적용됩니다. Branch Enterprise 노드는 일반 단일 머신 배포로 설치되며, 이 6대 Active/Standby 설계에 참여하지 않습니다.

전체 토폴로지



핵심 요청 경로는 다음과 같습니다.

```
클라이언트 / 현장 시스템
    -> Business VIP :8088
    -> 현재 Active Enterprise (D / E / F)
    |-- PostgreSQL VIP :5432 -> D/E HAProxy -> A/B 현재 쓰기 가능한 Primary
    |-- Redis VIP      :6379 -> C 또는 F 독립 Redis
    `-- RustFS VIP     :19000 -> A/B HAProxy -> C/D/E/F RustFS 클러스터
```

4개의 VIP는 비즈니스 설정의 고정 진입점입니다. Enterprise가 노드를 전환하거나 미들웨어 backend 역할이 변경되어도 클라이언트와 Enterprise는 접속 주소를 변경하지 않습니다.

계획 IP 주소

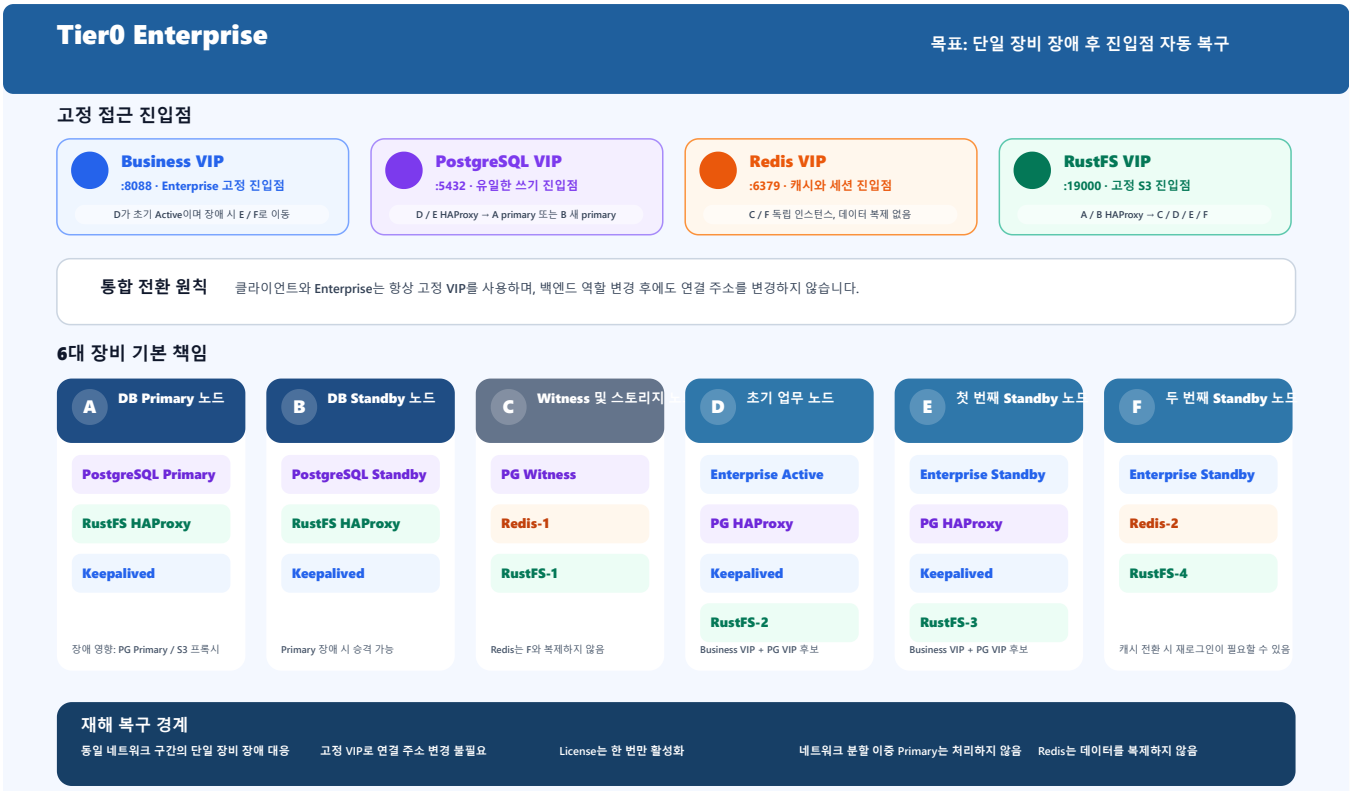
아래 주소는 예시입니다. 구현 전 현장의 동일 IPv4 서브넷에 있는 실제 주소로 교체하십시오. 4개의 VIP는 DHCP, 정적 호스트, 또는 다른 VRRP 인스턴스에서 사용 중이면 안 됩니다.

이름	예시 IP	설명
deployer-controller	`10.60.10.20`	독립 deployer. 6대 비즈니스 머신에는 포함하지 않음

이름	예시 IP	설명
ent-ha-01 / A	`10.60.10.11`	PostgreSQL Primary 및 관련 역할
ent-ha-02 / B	`10.60.10.12`	PostgreSQL Standby 및 관련 역할
ent-ha-03 / C	`10.60.10.13`	Witness, Redis-1, RustFS-1
ent-ha-04 / D	`10.60.10.14`	초기 Active Enterprise 및 관련 역할
ent-ha-05 / E	`10.60.10.15`	Enterprise Standby 및 관련 역할
ent-ha-06 / F	`10.60.10.16`	Enterprise Standby 및 관련 역할
Business VIP	`10.60.10.101`	Enterprise 통합 진입점, 기본값 `8088`
PostgreSQL VIP	`10.60.10.102`	데이터베이스 통합 진입점, 기본값 `5432`
Redis VIP	`10.60.10.103`	Redis 통합 진입점, 기본값 `6379`
RustFS VIP	`10.60.10.104`	S3 통합 진입점, 기본값 `19000`

구현 전에는 비즈니스 NIC 이름, IPv4 prefix, gateway, DNS, NTP, hostname, SSH 사용자, 디스크 mount point, firewall allowlist를 확인하십시오. VIP, node IP, NIC를 스크립트가 추측한 값으로 운영 환경에 직접 사용해서는 안 됩니다.

6대 머신 책임



머신	기본 배포 컴포넌트	핵심 책임
A	PostgreSQL Primary, RustFS HAProxy, Keepalived	초기 데이터베이스 Primary, 첫 번째 RustFS proxy 후보
B	PostgreSQL Standby, RustFS HAProxy, Keepalived	Streaming database Standby, 두 번째 RustFS proxy 후보
C	PostgreSQL Witness, Redis-1, RustFS-1	데이터베이스 Witness/상태 기록, Redis 및 RustFS 데이터 노드
D	PG HAProxy, Keepalived, RustFS-2, Enterprise, Fleet Agent, rsync	초기 비즈니스 Active, 첫 번째 PostgreSQL proxy 후보
E	PG HAProxy, Keepalived, RustFS-3, Enterprise, Fleet Agent, rsync	비즈니스 Standby, 두 번째 PostgreSQL proxy 후보

머신	기본 배포 컴포넌트	핵심 책임
F	Redis-2, RustFS-4, Enterprise, Fleet Agent, rsync	비즈니스 Standby, 두 번째 Redis 인스턴스

이 할당은 단일 머신 장애 허용과 6대 머신 간 리소스 균형을 위한 것입니다. deployer는 hostname/IP 주소의 자연 순서에 따라 머신을 A-F에 매핑하고 기본 역할을 선택하지만, 최종 할당은 현장 디스크 구성과 장애 도메인을 기준으로 다시 검토해야 합니다.

머신 구성 체크리스트

Enterprise 비즈니스 계층은 Single Active/Standby입니다. 어느 시점이든 Business VIP를 보유한 노드만 비즈니스 요청을 처리합니다. 비즈니스 계층은 하나의 비즈니스 노드 CPU, 메모리, 디스크를 늘리는 수직 확장으로 확장하십시오. Enterprise 노드 수를 곱해서 동시 처리량을 추정하지 마십시오.

머신	노드 유형	vCPU	메모리	기본 스토리지	디스크 권장 사항
A	PostgreSQL Primary 노드	8C	16G	1T	`/` 약 80G, 나머지는 PostgreSQL 데이터, WAL, backup staging 용도
B	PostgreSQL Standby 노드	8C	16G	1T	`/` 약 80G, 나머지는 PostgreSQL 데이터, WAL, backup staging 용도

머신	노드 유형	vCPU	메모리	기본 스토리지	디스크 권장 사항
C	Witness / middleware 노드	4C	8G	500G	system, Redis, RustFS data에 고정 mount point 할당
D	초기 Active Enterprise 노드	8C	32G	500G	system, containers, Enterprise 동기화 파일, RustFS data에 고정 mount point 할당
E	Standby Enterprise 노드	8C	32G	500G	D와 동일
F	Standby Enterprise 노드	8C	32G	500G	D와 동일

범주	수량	노드당 사양	vCPU 소계	메모리 소계	스토리지 소계
PostgreSQL 데이터베이스 노드	2	`8C / 16G / 1T`	16C	32G	2T
Witness / middleware 노드	1	`4C / 8G / 500G`	4C	8G	500G
Enterprise 비즈니스	3	`8C / 32G / 500G`	24C	96G	1.5T

범주	수량	노드당 사양	vCPU 소계	메모리 소계	스토리지 소계
노드					
합계	**6**	-	**44C**	**136G**	**4T**

구성 요약

Enterprise 통합 접속 구성

D, E, F는 동일한 Enterprise 구성을 유지합니다.

의존성	구성 원칙
PostgreSQL	A/B 노드 IP가 아니라 PostgreSQL VIP에 연결
Redis	C/F 노드 IP가 아니라 Redis VIP에 연결
File storage	`FILESTORE_DRIVER=s3` 사용, S3 endpoint를 RustFS VIP로 지정
Fleet identity	3개 노드는 하나의 논리 `installationId`를 공유하고, 각 노드는 고유한 `memberId`를 가짐
Fleet Agent	각 Enterprise 노드에서 하나의 process-level Fleet Agent 실행
License	Business VIP를 통해 한 번만 활성화하고, Bundle을 HA fileset에 포함

`installationId`와 `memberId`는 설치 프로세스에서 생성되고 영속화됩니다. 운영자가 이를 수동으로 만들거나 머신 간에 `memberId`를 복사해서는 안 됩니다.

VIP와 후보 노드

VIP	후보 노드	Health 기준	인계 결과
Business VIP	D, E, F	Fleet Agent readiness, 비즈니스 컨테이너 상태, sync generation	Owner만 Enterprise HA 비즈니스 그룹을 시작
PostgreSQL VIP	D, E	HAProxy/Keepalived 상태. HAProxy는 쓰기 가능한 Primary로만 라우팅	Enterprise는 동일한 데이터베이스 주소 유지
Redis VIP	C, F	Redis `PING` 및 로컬 서비스 상태	다른 독립 인스턴스로 전환. cache/session 손실 가능
RustFS VIP	A, B	HAProxy 및 RustFS backend health	S3 주소는 유지되고 proxy entry만 이동

컴포넌트 배포 설계

Enterprise, Fleet Agent, Business VIP

- D, E, F는 동일한 전체 Enterprise offline ZIP을 설치합니다.
- Fleet Agent는 host-level process service이며 비즈니스 컨테이너가 아닙니다.
- Keepalived는 Business VIP를 관리하고 Fleet Agent readiness endpoint를 호출하여 로컬 노드가 VIP를 보유할 수 있는지 판단합니다.
- Business VIP Owner만 비즈니스 그룹을 실행하고, 나머지 2대는 Standby 상태를 유지합니다.
- VIP를 잃은 노드는 비즈니스 그룹을 중지해야 합니다. 복구된 노드는 먼저 Standby로 재참여해야 합니다.
- EMQX는 Single Active 비즈니스 그룹의 일부로 실행됩니다. Agent 제어 밖에서 재시작하거나 3대 머신 사이에 별도 election 메커니즘을 만들지 마십시오.

PostgreSQL Primary/Standby, Witness, 통합 진입점

- A는 Primary, B는 Streaming Standby, C는 repmgr Witness로 시작합니다.
- Witness는 cluster 상태를 기록하고 장애 판단에 참여합니다. 전체 비즈니스 데이터를 저장하지 않으며 읽기나 쓰기를 처리하지 않습니다.
- Primary 장애 시 repmgr는 정상 Standby를 새 Primary로 승격합니다.
- D/E HAProxy는 `5432` 트래픽을 현재 쓰기 가능한 Primary로만 라우팅합니다.
- PostgreSQL VIP는 D/E proxy 노드 사이에서 이동합니다. proxy failover와 database primary/standby failover는 분리되어 있습니다.
- 검수 시에는 "쓰기 가능한 Primary가 하나뿐임"과 "Standby가 streaming을 재개함"을 모두 증명해야 합니다.

Redis 독립 인스턴스

- C와 F는 각각 하나의 standalone Redis 인스턴스를 실행합니다.
- 두 Redis 인스턴스는 Sentinel, Cluster, primary/replica replication을 구성하지 않습니다.
- Redis VIP는 접속 진입점만 전환합니다. 현재 인스턴스에 장애가 발생하면 VIP는 다른 정상 인스턴스로 이동합니다.
- Redis에는 손실되거나 재구성 가능한 데이터만 저장해야 합니다. 전환 후 빈 cache와 사용자 재로그인은 설계 경계에 포함됩니다.
- 향후 Redis가 폐기할 수 없는 상태를 저장한다면 독립 인스턴스 가정을 유지하지 말고 replication/arbitration 설계로 업그레이드해야 합니다.

RustFS 4노드 클러스터

- C, D, E, F는 4노드 distributed RustFS object storage cluster를 구성합니다.
- A와 B의 HAProxy는 모든 정상 RustFS backend로 proxy합니다.
- RustFS VIP는 A와 B 사이에서 이동합니다. Enterprise는 항상 이 VIP를 통해 S3에 접근합니다.
- 단일 RustFS 노드 장애는 cluster mechanism이 처리하고, 단일 proxy 장애는 VIP failover가 처리합니다.
- object data를 rsync, hot copy, RustFS mount directory 직접 편집으로 동기화하지 마십시오.

- RustFS node count, disk count, erasure-coding availability boundary는 배포된 버전의 cluster check 결과를 따라야 합니다.

App, Flow, Marimo, License 파일 동기화

Fleet Agent는 PostgreSQL/RustFS에 직접 둘 수 없지만 새 Active 노드에 반드시 있어야 하는 로컬 비즈니스 파일만 동기화합니다.

- App runtime directory 및 `node_modules`
- Flow, Marimo, Notebook 및 관련 local runtime file
- License Bundle
- 현재 구현에 명시적으로 등록된 기타 HA fileset

동기화는 deployer가 관리하는 비즈니스 노드 간 SSH trust 위에서 system `rsync`를 사용합니다. 각 release는 완전한 generation/manifest를 생성합니다. Standby 노드는 generation이 완전함을 확인한 뒤에만 인계할 수 있습니다. PostgreSQL data directory, Redis data directory, RustFS data directory는 이 동기화 범위에 포함되지 않습니다.

접속 방식과 네트워크 기준

접근 관계

Source	Target	Purpose
사용자 / 현장 시스템	Business VIP	Web/API 비즈니스 접근
D/E/F Enterprise	PostgreSQL VIP	비즈니스 데이터베이스 읽기/쓰기
D/E/F Enterprise	Redis VIP	cache 및 session
D/E/F Enterprise	RustFS VIP	S3 object 읽기/쓰기
Deployer controller	A-F	SSH, 배포, 상태 수집, 운영
D/E/F	D/E/F	rsync/SSH 파일 동기화
A/B/C	A/B/C	PostgreSQL streaming replication, repmgr, Witness communication

Source	Target	Purpose
RustFS members/proxies	C/D/E/F	RustFS cluster 및 S3 backend communication
동일 VIP를 공유하는 노드	해당 후보 노드	VRRP advertisement 및 VIP 이동

포트와 프로토콜

포트는 필요한 최소 source-to-target 관계에 따라 허용하십시오. 모든 관리 포트를 사용자 네트워크에 직접 노출하지 마십시오.

Port / protocol	권장 allowlist	Purpose
`22/TCP`	Deployer -> A-F; D/E/F 상호 sync	SSH, deployment, rsync
`8088/TCP`	사용자 네트워크 -> Business VIP	Enterprise 비즈니스 진입점
`5432/TCP`	D/E/F -> PostgreSQL VIP; PG/proxy internal access	PostgreSQL, streaming replication, proxy
`6379/TCP`	D/E/F -> Redis VIP; health-check nodes -> C/F	Redis access 및 health check
`19000/TCP`	D/E/F -> RustFS VIP	Enterprise fixed S3 entry
`9000/TCP`	A/B -> C/D/E/F; RustFS members	RustFS S3 backend/cluster communication. 버전별 확인 필요
`9001/TCP`	통제된 운영 네트워크 -> RustFS admin entry	선택적 RustFS admin endpoint. 사용자 네트워크에 노출 금지
`19731/TCP`	Localhost / Keepalived / 통제된 운영 네트워크	Fleet Agent HTTP health/control interface

Port / protocol	권장 allowlist	Purpose
`18080/TCP`	운영 네트워크 -> deployer controller	Deployer HTTPS UI
`112/VRRP`	각 VIP 후보 노드	Keepalived VIP movement. TCP/UDP가 아니라 IP protocol number
`1883/8883`	현장 장치 네트워크 -> 비즈니스 진입점, 제품 설정 기준	MQTT/MQTTS

Failover와 기능 경계

Failure scenario	Expected behavior	Boundary
D Active 비즈니스 노드 장애	Business VIP가 E 또는 F로 이동하고, 새 Owner는 readiness 통과 후 비즈니스 그룹을 시작	Standby에 최신 HA fileset generation이 완료되어 있어야 함
PostgreSQL A Primary 장애	B가 Primary로 승격되고 PostgreSQL VIP는 D/E proxy를 통해 쓰기 가능한 Primary로 계속 라우팅	데이터 손실은 streaming replication 상태에 따라 달라짐
현재 Redis 노드 장애	Redis VIP가 다른 독립 Redis 인스턴스로 이동	cache/session 손실 가능
RustFS proxy A 또는 B 장애	RustFS VIP가 다른 proxy로 이동	RustFS data는 여전히 4노드 cluster health에 의존
단일 RustFS data node 장애	redundancy 조건이 충족되면 RustFS cluster가 서비스를 계속함	실제 경계는 disk count 및 erasure-coding configuration에 따라 달라짐

네트워크 분리, 동시 다중 노드 장애, 잘못된 VRRP priority, 디스크 가득 참, clock drift, deployer 외부의 수동 변경은 일반 자동 복구 약속 범위 밖이며 운영 절차로 다뤄야 합니다.

백업, 모니터링, 점검

- PostgreSQL은 독립적인 logical 또는 physical backup과 restore 검증이 필요합니다.
- RustFS는 capacity, disk, cluster health monitoring이 필요합니다.
- Redis는 향후 replication 설계가 도입되기 전까지 cache/session으로 취급해야 합니다.
- D/E/F의 Fleet Agent와 Keepalived 상태를 모니터링해야 합니다.
- rsync generation lag를 모니터링해야 합니다. 오래된 generation을 가진 Standby는 비즈니스 트래픽을 인계해서는 안 됩니다.
- 정기 failover drill에는 Business VIP, PostgreSQL promotion, Redis VIP switch, RustFS proxy switch, backup restore가 포함되어야 합니다.

Go-live 전 점검

- deployer controller에서 6대 머신 모두 SSH로 접근할 수 있습니다.
- hostname, IP address, NIC name, NTP, DNS, firewall rule이 고정되고 기록되어 있습니다.
- 4개의 VIP는 사용 중이 아니며 후보 노드 사이에서 이동할 수 있습니다.
- disk mount point가 고정되어 있고 reboot 후에도 유지됩니다.
- Enterprise, PostgreSQL, Redis, RustFS, Keepalived, HAProxy, repmgr, Fleet Agent, rsync 구성이 deployer에 의해 생성되고 검토되었습니다.
- Cluster License는 Business VIP를 통해 한 번만 활성화되었습니다.
- production traffic 투입 전에 backup 및 restore 절차를 rehearsal했습니다.

구현 순서

1. deployer controller와 6대 머신을 준비합니다.
2. IP, VIP, NIC, disk, DNS, NTP, SSH, firewall 정보를 확인합니다.
3. Enterprise offline package와 configuration material을 업로드합니다.
4. PostgreSQL primary/standby 및 Witness를 배포합니다.
5. Redis independent instance와 Redis VIP를 배포합니다.
6. RustFS 4노드 cluster와 RustFS VIP를 배포합니다.
7. D/E/F에 Enterprise, Fleet Agent, rsync fileset, Business VIP를 배포합니다.
8. Business VIP를 통해 License를 활성화합니다.

9. 기능 검증과 failover drill을 실행합니다.
10. 최종 topology, password/key, backup policy, monitoring target, recovery procedure를 기록합니다.

검수 기준

기능 검수

- 사용자는 Business VIP를 통해 Enterprise에 접근할 수 있습니다.
- Enterprise는 PostgreSQL VIP를 통해서만 PostgreSQL을 읽고 씁니다.
- Enterprise는 Redis VIP를 통해서만 Redis를 사용합니다.
- Enterprise는 RustFS VIP를 통해서만 RustFS를 사용합니다.
- failover 후 App, Flow, Marimo, Notebook, License 파일이 Active 노드에 존재합니다.
- Branch Enterprise는 계속 독립적으로 설치할 수 있으며 이 6대 HA group에 참여하지 않습니다.

Failover drill 검수

- Active 비즈니스 노드 중지: Business VIP가 정상 Standby로 이동하고 비즈니스 그룹은 그곳에서만 시작됩니다.
- PostgreSQL Primary 중지: Standby가 승격되고 PostgreSQL VIP는 쓰기 가능한 Primary를 계속 가리킵니다.
- Redis VIP owner 중지: Redis VIP가 다른 인스턴스로 이동하며 cache/session 손실을 허용합니다.
- RustFS proxy owner 중지: RustFS VIP가 다른 proxy로 이동합니다.
- RustFS data node 하나 중지: object access는 실제 RustFS redundancy boundary 안에서 유지됩니다.
- 장애 노드 복구: Standby 또는 정상 member로 돌아오며 더 새로운 data나 file을 덮어쓰지 않습니다.

현장 확인 리스크와 항목

- 네트워크가 VRRP protocol `112`를 허용하는지.
- 4개의 VIP 주소가 실제로 미사용 상태인지.

- disk mount layout과 capacity가 실제 file retention 및 RustFS redundancy requirement를 충족하는지.
- Redis가 disposable data만 저장하는지.
- PostgreSQL replication lag와 backup policy가 비즈니스 RPO를 충족하는지.
- 운영 담당자가 VIP failover, file synchronization, database replication, object storage redundancy의 차이를 이해하는지.

결론

현재 계획은 6대 머신, 4개의 고정 VIP, Single Active Enterprise를 중심으로 합니다. 비즈니스 진입점, 데이터베이스 진입점, 캐시 진입점, 객체 스토리지 진입점을 분리합니다. 동일 네트워크 세그먼트 안에서 일반적인 단일 머신 장애를 커버할 수 있고, failover 후 connection configuration 변경, License 재활성화, App 파일 수동 복사 작업을 크게 줄입니다.

동시에 세 가지 경계를 받아들여야 합니다. Redis는 데이터를 복제하지 않고, 비즈니스 fileset은 비동기로 동기화되며, 네트워크 partition에는 consensus arbitration이 없습니다. 따라서 이 계획은 **Fleet Center 6대 Single Active Active/Standby 재해 복구 계획**으로 정의하는 것이 적합합니다. 외부에 zero interruption, zero data loss, 또는 cross-data-center strict high availability를 약속해서는 안 됩니다.

참조

SLA 및 High Availability 경계

표준 Tier0 Enterprise deployment를 위한 SLA 권장 사항, high availability 경계, acceptance gate, drill, hardening 항목입니다.

이 문서는 six-VM fully offline deployment에 적용됩니다. 구성은 application / Swarm node 3대, PostgreSQL HA node 2대, witness node 1대입니다.

Architecture Baseline

Layer	Current Design	High Availability Boundary
Application runtime	Enterprise New compose는 고정된 `tier0_app_active_node` 에서 실행됩니다.	`backend` , `redis` , `EMQX` , `SourceFlow` , `EventFlow` , `Marimo` , `OPC UA Server` 는 아직 automatic multi-replica service가 아닙니다.
HTTP entry	3개의 application node가 keepalived APP VIP + HAProxy를 실행합니다.	VIP는 float할 수 있습니다. HAProxy는 `8088/TCP` 의 active node로 forward하고 `/healthz` 를 check합니다.
MQTT / OPC UA	EMQX는 `1883/8883/8083/8084` 를 사용하고 OPC UA는 `4840` 을 사용합니다.	이들은 active node에 직접 연결됩니다. TCP VIP proxying에 포함되지 않으므로 protocol-entry HA는 약속하지 않습니다.
Database	PostgreSQL / TimescaleDB streaming replication + repmgr + witness + DB VIP.	primary 장애 시 standby를 자동 promote할 수 있습니다. asynchronous replication은 strict zero RPO를 약속할 수 없습니다.

Layer	Current Design	High Availability Boundary
Backup and restore	`backup-db.sh` / `restore-db.sh`.	recoverability를 제공하지만 online HA와 동일하지는 않습니다.

권장 SLA

Service Object	Recommended Statement	Constraints
Platform HTTP entry	monthly availability `99.5%`부터 시작합니다.	VIP failover는 active application node의 automatic migration과 같지 않습니다.
Database access	운영 목표: `RTO <= 2 minutes`.	RPO는 replication lag에 따라 달라집니다. 같은 architecture의 과거 failover test는 약 59-64초였습니다.
MQTT / OPC UA	현재 설계에서는 HA SLA를 약속하지 않습니다.	HAProxy TCP proxying, 별도 VIP 또는 service clustering을 먼저 추가해야 합니다.
Backup and restore	Daily backup, major change 전 mandatory backup.	restore time은 data volume과 on-site I/O에 따라 달라집니다.

`99.9%` 이상을 주장하기 전에 application automatic migration 또는 multi-replica deployment, protocol-entry HA, monitoring alert, recovery drill을 완료하세요. 계획된 capability를 이미 제공된 capability처럼 설명하지 마세요.

Acceptance Gates

각 deployment 또는 upgrade는 최소한 다음 조건을 만족해야 합니다.

- `verify-materials.sh`가 통과하고 delivery package SHA256 check가 통과합니다.
- 3개의 Swarm node가 `Ready`입니다.

- `APP VIP /healthz`와 `/readyz`가 `200`을 반환하며 home page와 `/uns`에 접근할 수 있습니다.
- active node의 7개 compose service가 실행 중입니다.
- `DB VIP:5432`에 접근할 수 있으며 primary, standby, witness topology가 정상입니다.
- default account가 실제로 login할 수 있습니다. IAM initialization 성공 여부를 port, container, HTTP 200 response만으로 판단하지 마세요.
- HTTP, EMQX port, WebSocket/WSS, OPC UA TCP smoke check가 통과합니다.

Inspection 및 Alert

매일 아래 command를 실행합니다.

```
./bin/verify.sh --site config/site.conf
./bin/smoke-business-emqx.sh --site config/site.conf
```

최소한 다음 alert를 설정합니다.

- APP VIP `/healthz` 또는 `/readyz` failure.
- DB VIP unreachable, no primary, split-brain risk, replication interruption 또는 threshold 초과 replication lag.
- keepalived, HAProxy, PostgreSQL, repmgrd abnormal status.
- `backend`, `redis`, `EMQX` 같은 key container가 exit하거나 자주 restart됩니다.
- data disk usage가 `80% / 90%`를 초과합니다.
- 최신 backup이 실패했거나 생성되지 않았습니다.

Drill 요구사항

Drill	Frequency	Verification Criteria
APP VIP failover	납품 전 및 분기별	현재 VIP node에서 keepalived를 중지한 뒤 `/healthz`와 `/readyz`가 복구됩니다.

Drill	Frequency	Verification Criteria
DB primary failure	납품 전 및 6개월마다	standby가 promote되고 DB VIP가 복구되며 old primary가 standby로 rebuild됩니다.
Database restore	monthly sample 및 major change 전	backup이 restore 가능합니다. restore 후 `verify`, smoke check, login verification이 통과합니다.
Material verification	package build마다	source material, package SHA, unpacking verification이 통과합니다.

Incident Handling Principles

1. 먼저 impact scope를 확인합니다: HTTP, MQTT, OPC UA, database, single-node impact.
2. VIP, HAProxy, active application, database primary 복구를 우선합니다.
3. log, data directory, backup을 보존합니다. incident site를 직접 clean하거나 overwrite하지 마세요.
4. database incident에서는 현재 primary를 먼저 확인한 뒤 failover, rejoin, restore 중 무엇을 할지 결정합니다.
5. recovery 후 `verify`, smoke check, 실제 login verification을 실행합니다.

High-SLA Hardening Items

- application을 multi-replica로 만들거나 active node의 검증 가능한 automatic migration process를 구축합니다.
- EMQX를 cluster화하거나 dedicated VIP가 있는 HAProxy TCP / TLS를 통해 routing합니다.
- OPC UA에 대해 HAProxy TCP, dedicated VIP 또는 service clustering 기반 high-availability entry를 제공합니다.
- PostgreSQL synchronous replication을 사용하거나 허용 가능한 replication lag와 RPO를 명확히 정의합니다.

- backup scheduling, off-site replication, 정기 restore drill을 자동화합니다.
- container, host, database, log, black-box probe를 unified alerting에 연결합니다.

관련 절차: 운영 Runbook 및 표준 Port 목록.

참조

표준 Port 목록

표준 six-VM offline Tier0 Enterprise deployment를 위한 firewall port list입니다.

아래 table은 Enterprise New six-VM offline deployment를 기준으로 합니다. customer firewall은 최소한으로 필요한 source range에서 오는 traffic만 허용해야 합니다. 이 port들을 public internet에 직접 노출하지 마세요.

Source	Target	Port	Protocol	Purpose
Control host	Six target VMs	22	TCP	SSH / Ansible
3개 application node 상호 간	3개 application node	2377	TCP	Docker Swarm management plane
3개 application node 상호 간	3개 application node	7946	TCP/UDP	Swarm node discovery
3개 application node 상호 간	3개 application node	4789	UDP	Swarm overlay
3개 application node	Active application node	8088	TCP	HAProxy to Enterprise New backend; application network segment only
Business network segment	APP VIP	80	TCP	Platform HTTP entry, `/healthz`, `/readyz`
Business network segment	APP VIP / active	1883	TCP	MQTT

Source	Target	Port	Protocol	Purpose
	application node			
Business network segment	APP VIP / active application node	8883	TCP	MQTT TLS
Business network segment	APP VIP / active application node	8083	TCP	MQTT WebSocket
Business network segment	APP VIP / active application node	8084	TCP	MQTT WebSocket TLS
Business network segment	APP VIP / active application node	4840	TCP	OPC UA Server
Controlled application network segment	Active application node	8089	TCP	Launchpad
Dynamic app access network segment	Active application node	30000-30199	TCP	App Gateway / dynamic application host ports
Application nodes, DB nodes, witness	DB VIP / DB nodes	5432	TCP	PostgreSQL / TimescaleDB

Source	Target	Port	Protocol	Purpose
DB1, DB2, witness	DB1, DB2, witness	5432	TCP	Streaming replication / repmgr
Application nodes	NFS server	2049	TCP/UDP	`NFS_ENABLED=true` 일 때 사용

현재 compose deployment는 Redis, SourceFlow, EventFlow, Marimo, EMQX Dashboard를 외부에 노출하지 않습니다. 이러한 service는 Docker runtime network 안에서 backend에 의해서만 호출됩니다.

참조

운영 Runbook

표준 Tier0 Enterprise deployment의 daily inspection, failover, backup, restore, cleanup, troubleshooting 절차.

이 document는 Enterprise New offline application을 기준으로 합니다.

일일 점검

1. deployment package の root directory で다음 commands를 실행합니다.

```
./bin/verify.sh --site config/site.conf
./bin/smoke-business-emqx.sh --site config/site.conf
```

2. 다음 항목을 확인합니다.
   - 3 台の Swarm nodes が `Ready` である。
   - `APP VIP /healthz` と `/readyz` が `200` を返す。
   - `DB VIP:5432` に接속할 수 있다。
   - PostgreSQL primary、standby、witness topology が정상이다。
   - `backend`、`redis`、`emqx`、`sourceflow`、`eventflow`、`marimo`、`opcua-server` が active application node で모두 running 상태이다。

Application Checks

1. active application node で다음 commands를 실행합니다.

```
cd /opt/tier0-enterprise/tier0-deploy
bash bin/compose.sh ps
bash bin/compose.sh logs --tail=200 backend
curl -fsS http://127.0.0.1:8088/healthz
curl -fsS http://127.0.0.1:8088/readyz
```

> ⓘ 참고

`bin/compose.sh` を迂回して raw `docker compose` commands を直接実行하지 마세요.この wrapper script は installer が生成した runtime environment を読み込みます。

2. Health-check の見方:
   - `healthz` failure: まず backend process、image、container logs 를 확인합니다.
   - `readyz` failure: DB VIP、Redis、database migration logs も確認します。
   - Home page に到達できるが login できない場合: `.env` の `ADMIN_INITIAL_PASSWORD` を確認し、その後 backend IAM migration logs と user tables 를 확인합니다.HTTP port だけで deployment success を判断하지 마세요.

Protocol Entry Checks

```
nc -vz <APP_VIP> 1883
nc -vz <APP_VIP> 8883
nc -vz <APP_VIP> 8083
nc -vz <APP_VIP> 8084
nc -vz <APP_VIP> 4840
```

> ⓘ 참고
> EMQX と OPC UA Server は active application node に固定されています。APP VIP failover が保証するのは `APP VIP:80` の HTTP entry のみです。Protocol ports は HAProxy TCP proxying 経由で route されないため、この design は protocol entries の high availability を保証しません。

Database Checks

Database nodes で次を실행합니다.

```
systemctl status tier0-postgresql
systemctl status tier0-repmgrd
sudo -iu postgres psql -Atqc 'select pg_is_in_recovery();'
```

```
sudo -iu postgres /usr/local/bin/repmgr -f /etc/repmgr.conf cluster show
```

- `pg_is_in_recovery=false` は primary を의미합니다.
- `pg_is_in_recovery=true` は standby を의미합니다.
- DB VIP は current primary のみに配置されている필요합니다.
- standby replication が interrupted した場合、または witness が upstream errors を報告した場合は、まず site を保全してください。full deployment を直接再実行하지 마세요.

APP VIP Failover

1. test 前に、現在 VIP を保持している node を記録し、次를 확인합니다.

```
curl -fsS http://<APP_VIP>/healthz
curl -fsS http://<APP_VIP>/readyz
```

2. 現在の VIP node で `keepalived` を停止した後、別の application node への VIP takeover を確認し、両方の URLs を再テストします。HAProxy backend は active application node の `8088/TCP` を指すため、3 台すべての application nodes が active node の port 8088 に到達できる필요합니다.

3. `keepalived` を復旧した後、`verify.sh` と smoke checks を再실행합니다.drill 中に active application containers を停止하지 마세요.

DB Primary Failure Switchover

Failure drill の前に backup を作成し、replication が healthy であること를 확인합니다.

```
./bin/backup-db.sh --site config/site.conf
```

現在の primary で `tier0-postgresql` を停止した後、`repmgr` が standby を promote し、DB VIP takeover すること를 확인합니다.target RTO は 2 分以内です。同じ architecture での過去の tests は約 59-64 秒でした。

> ⚠ 주의

- old primary を production topology に直接戻して起動してはいけません。まず PostgreSQL、repmgrd、keepalived を停止し、残っている DB VIP を削除します。その後 current new primary から basebackup を実行し、node を standby として register し、最後に witness を再接続します。
- failover 後に full installer を直接再実行하지 마세요.inventory 内の static primary/standby definitions が current topology を乱す可能性があります。

Backup and Restore

- Backup:

```
./bin/backup-db.sh --site config/site.conf
```

- Restore:

```
./bin/restore-db.sh --site config/site.conf --restore-file /backup/tier0-db/postgres-YYYYmmddTHHMMSS.dump
```

> ⓘ 참고
> restore 前に current primary を確認してください。restore 後は `verify.sh`、smoke checks、default-user path の実際の platform login verification を실행합니다.

Cleanup and Retest

```
./full-cleanup.sh --env install.env --yes
./install.sh --auto --yes
```

> ⚠ 주의
> Full cleanup は 6 nodes 上の Tier0 runtime data、Docker/Swarm data、database backups を削除します。また control container が終了した後、control node の final cleanup も실

행합니다.この entry は disposable であることが確認済みの test environments でのみ사용하세요.customer data または incident site を保持する必要がある場合は、通常の `--cleanup` を使用し、先に backup を完了してください。

Common Issues

Backend Startup Failure

```
cd /opt/tier0-enterprise/tier0-deploy
bash bin/compose.sh logs --tail=300 backend
bash bin/compose.sh ps
```

> ⓘ 참고
> DB URL、DB VIP、Redis、migration、mounted directory permissions に중점적으로 확인하세요.

EMQX Startup Failure

```
cd /opt/tier0-enterprise/tier0-deploy
bash bin/compose.sh logs --tail=200 emqx
```

> ⓘ 참고
> certificates、configuration files、volume permissions、および ports `1883/8883/8083/8084` が占有されていないか確認してください。

repmgrd Repeated Restarts

```
systemctl status tier0-repmgrd
tail -100 /var/log/repmgr/repmgr.log
sudo -iu postgres psql -Atqc 'show shared_preload_libraries;'
```

ⓘ 참고
`shared_preload_libraries` に `repmgr` が含まれていること、および systemd unit が `repmgrd --daemonize=false` を使用していることを確認してください。

Material Verification Failure

```
./bin/verify-materials.sh
```

⚠ 주의
SHA256 verification を skip して deployment を続行하지 마세요.delivery package を再取得するか、traceable process に従って materials を rebuild してください。

참조

OpenAPI 레퍼런스 부록

Tier0 OpenAPI v1 · 작업: 36 · 스키마: 85

작업

API 필드 이름, 예제 및 원본 설명은 포함된 영어 OpenAPI 명세를 따릅니다.

UNS

Unified Namespace 데이터 읽기, 쓰기 및 관리.

POST /uns/attachments

Upload a UNS attachment
Uploads a multipart attachment for a UNS node. Supply a file and at least one of unsId or topic; a valid unsId takes precedence.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/attachments' \
  --header 'x-api-key: <api-key>'
```

multipart/form-data

필드	유형	필수	설명
unsId	integer (int64)	아니요	Numeric owner node ID. Supply unsId or topic; a valid unsId takes precedence.
topic	string	아니요	Full owner node path, used when a valid unsId is not supplied.
fileName	string	아니요	Original file name.
sha256	string	아니요	SHA-256 checksum of the file contents.
file	string (binary)	예	Binary file contents. The multipart field name must be file.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "unsId": 0,
    "fileName": "string",
    "filePath": "string",
    "fileUrl": "string",
    "contentType": "string"
  }
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
msg	string	아니요	Response message
data	object	아니요	UnsAttachmentUploadResp. 스키마 절에서 참조된 스키마를 확인하세요.

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/attachments/list

List UNS attachments
Lists attachments for a UNS node or topic. Temporary file URLs can expire and must then be requested again.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/attachments/list' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"unsId":0,"topic":"string","page":0,"size":0,"includeFileUrl":false}'
```

application/json

필드	유형	필수	설명
unsId	integer (int64)	아니요	Numeric UNS node ID.
topic	string	아니요	Full UNS topic path.
page	integer (int64)	아니요	Page number, starting at 1.
size	integer (int64)	아니요	Number of attachments per page.
includeFileUrl	boolean (boolean)	아니요	Whether to return temporary file URLs. Defaults to true.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "unsId": 0,
    "list": [
      "value"
    ],
    "total": 0,
    "page": 0,
    "size": 0
  }
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
msg	string	아니요	Response message
data	object	아니요	UnsAttachmentListResp. 스키마 절에서 참조된 스키마를 확인하세요.

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/browse

Browse the UNS tree
Returns a UNS subtree. Use an empty path for the workspace root.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/browse' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"path":"string","max_depth":0,"include_metadata":false,"include_leaf_value":false}'
```

application/json

필드	유형	필수	설명
path	string	아니요	Root UNS path. Empty means the workspace root.

필드	유형	필수	설명
max_depth	integer (int64)	아니요	Maximum tree depth. Omit or use 0 for the server default.
include_metadata	boolean (boolean)	아니요	Whether returned nodes should include metadata.
include_leaf_value	boolean (boolean)	아니요	Whether returned leaf nodes should include their latest payload.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "tree": [
      "value"
    ]
  }
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
msg	string	아니요	Response message
data	object	아니요	BrowseResp. 스키마 절에서 참조된 스키마를 확인하세요.

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain

상태	의미	미디어 유형
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/create

Create UNS nodes

Creates UNS nodes and nested subtrees. Check each item result and use the returned topic in subsequent calls.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/create' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"namespace":
[{"name":"value","type":"value","children":"value","topicType":"value","fields":"value"}]}'
```

application/json

필드	유형	필수	설명
namespace	array<object>	예	UNS nodes to create. Nest children to create a subtree.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ]
  }
}
```


성공 응답 필드

`200 · application/json`

필드	유형	필수	설명
`code`	`integer`	아니요	Response code
`msg`	`string`	아니요	Response message
`data`	`object`	아니요	CreateResp. 스키마 절에서 참조된 스키마를 확인하세요.

응답 상태 의미

상태	의미	미디어 유형
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/uns/delete`

Delete UNS nodes
Deletes the listed nodes sequentially. By default this is a soft delete; the batch is not rolled back when a later item fails.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"topics":["string"],"hard_delete":false}'
```

application/json

필드	유형	필수	설명
topics	array<string>	예	UNS topic or path node paths to delete.
hard_delete	boolean (boolean)	아니요	Permanently delete instead of moving nodes to the recycle state.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false
  }
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
msg	string	아니요	Response message
data	object	아니요	OpenapiEmptyResp. 스키마 절에서 참조된 스키마를 확인하세요.

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json

상태	의미	미디어 유형
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/history

Query UNS history

Queries historical topic values over an inclusive RFC 3339 time range. Results can be aggregated or sampled.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/history' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"topics":["string"],"start_time":"string","end_time":"string","aggregation":
{"interval":"string","function":"string","field":"string","fields":["value"]},"page":0}'
```

application/json

필드	유형	필수	설명
topics	array<string>	예	UNS topic paths whose history should be queried.
start_time	string	예	Inclusive query start in RFC 3339 format.
end_time	string	예	Inclusive query end in RFC 3339 format.
aggregation	object	아니요	HistoryAggregation. 스키마 절에서 참조된 스키마를 확인하세요.
page	integer (int64)	아니요	Page number, starting from 1.
size	integer (int64)	아니요	Page size; values above the server maximum are capped.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json


```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ],
    "page": 0,
    "size": 0,
    "total": 0
  }
}
```


성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
msg	string	아니요	Response message
data	object	아니요	HistoryResp. 스키마 절에서 참조된 스키마를 확인하세요.

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/read

Read UNS values
Returns value, quality, and timeStamp (VQT) for each requested topic. A failed topic does not discard successful items in the same batch.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/read' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"topics":["string"],"include_metadata":false,"include_leaf_value":false}'
```

application/json

필드	유형	필수	설명
topics	array<string>	예	UNS topic paths to read.
include_metadata	boolean (boolean)	아니요	Whether each result should include UNS node metadata.
include_leaf_value	boolean (boolean)	아니요	Whether leaf nodes should include their latest payload.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ]
  }
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
msg	string	아니요	Response message

필드	유형	필수	설명
data	object	아니요	ReadResp. 스키마 절에서 참조된 스키마를 확인하세요.

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/restore

Restore a deleted UNS node

Restores a soft-deleted UNS node. Permanently deleted nodes cannot be restored.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/restore' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"path":"string"}'
```

application/json

필드	유형	필수	설명
path	string	예	Full UNS path of the node to restore. Use its path before deletion, not a node ID or HTTP URL.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false
  }
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
msg	string	아니요	Response message
data	object	아니요	OpenapiEmptyResp. 스키마 절에서 참조된 스키마를 확인하세요.

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/search

Search UNS nodes

Searches UNS node names, aliases, and paths, with optional type and path filters.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/search' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data
'{"keyword":"string","topicType":"string","path_prefix":"string","include_metadata":false,"include_
leaf_value":false}'
```

application/json

필드	유형	필수	설명
keyword	string	아니요	Keyword matched against UNS node names, aliases, and paths.
topicType	string	아니요	Topic data category filter: STATE, ACTION, or METRIC.
path_prefix	string	아니요	Only return nodes under this UNS path prefix.
include_metadata	boolean (boolean)	아니요	Whether returned nodes should include metadata.
include_leaf_value	boolean (boolean)	아니요	Whether returned leaf nodes should include their latest payload.
page	integer (int64)	아니요	Page number, starting from 1.
size	integer (int64)	아니요	Page size.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "objects": [
      "value"
    ],
    "page": 0,
    "size": 0,
    "total": 0
  }
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
msg	string	아니요	Response message
data	object	아니요	SearchResp. 스키마 절에서 참조된 스키마를 확인하세요.

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/unsBindFlow

Bind a flow to a UNS node
Binds a flow ID to a UNS node ID.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/unsBindFlow' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"unsId":0,"flowId":0}'
```

application/json

필드	유형	필수	설명
unsId	integer (int64)	예	UNS node ID to bind.
flowId	integer (int64)	예	Flow ID to bind.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "unsId": 0,
    "flowId": 0
  }
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
msg	string	아니요	Response message
data	object	아니요	UnsBindFlowResp. 스키마 절에서 참조된 스키마를 확인하세요.

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–

상태	의미	미디어 유형
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/update

Update a UNS node

Updates an existing UNS node. Empty strings, an empty fields array, and an empty extendProperties object retain existing values.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/update' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data
'{"path":"string","name":"string","description":"string","displayName":"string","extendProperties":
{}}'
```

application/json

필드	유형	필수	설명
path	string	예	Existing UNS topic or path node to update.
name	string	아니요	New node name.
description	string	아니요	Updated node description.
displayName	string	아니요	Updated display name.
extendProperties	object	아니요	Updated custom metadata key-value pairs.
fields	array<object>	아니요	Updated payload schema fields.
alias	string	아니요	Updated topic alias.
enableHistory	boolean (boolean)	아니요	Supply true or false to change history storage. Omit to keep the existing setting.
updateMask	array<string>	아니요	Compatibility field ignored by the current Enterprise implementation. It cannot be used to clear fields.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false
  }
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
msg	string	아니요	Response message
data	object	아니요	OpenapiEmptyResp. 스키마 절에서 참조된 스키마를 확인하세요.

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/write

Write UNS values

Writes values to existing TOPIC nodes; wildcards are not supported. `value` must match the node payload schema. **Batch results:** check each `data.results` item, not only HTTP 200. **Retry:** confirm the outcome before retrying a write.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/write' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"writes":
[{"topic":"value","value":"value","timeStamp":"value"}],"qos":0,"retain":false}'
```

application/json

필드	유형	필수	설명
writes	array<object>	예	Write operations to apply.
qos	integer (int64)	아니요	MQTT QoS: 0, 1, or 2.
retain	boolean (boolean)	아니요	Whether the MQTT message should be retained.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ]
  }
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code

필드	유형	필수	설명
`msg`	`string`	아니요	Response message
`data`	`object`	아니요	WriteResp. 스키마 절에서 참조된 스키마를 확인하세요.

응답 상태 의미

상태	의미	미디어 유형
`200`	Inspect the success flag of each `data.results` item. HTTP 200 alone does not confirm that every write succeeded.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

파일 리소스

저장된 파일 업로드, 다운로드 및 관리.

`POST` `/assets/files`

Create a direct-upload form

Issues an object-store upload form; it does not transfer file contents. Submit every returned post field unchanged with the file.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data
'{"fileName":"string","contentType":"string","size":0,"business":"string","useBy":"string"}'
```

`application/json`

필드	유형	필수	설명
fileName	string	예	Original file name.
contentType	string	아니요	MIME content type.
size	integer (int64)	예	File size in bytes. Business validation requires a value greater than 0 and at most 1 GiB.
business	string	아니요	Business domain or feature associated with the file.
useBy	string	아니요	Optional usage category: user, workspace, or platform.
visibility	string	아니요	File visibility, such as private or public.
appInstanceId	string	아니요	Associated application instance ID.
sessionId	string	아니요	Caller session ID.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "data": {
    "fileId": 0,
    "filePath": "string",
    "fileUrl": "string",
    "uploadUrl": "string",
    "expiresAt": 0
  },
  "msg": "string"
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	AssetFileUploadResp. 스키마 절에서 참조된 스키마를 확인하세요.

필드	유형	필수	설명
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/delete

Delete a file

Deletes a file identified by its server-issued storage path.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"filePath":"string"}'
```

application/json

필드	유형	필수	설명
filePath	string	예	Server-issued storage path returned by an upload API.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "data": {
    "deleted": false
  },
  "msg": "string"
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	AssetFileDeleteResp. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

GET /assets/files/download

Download a private file

Returns file bytes directly. Do not forward x-api-key or other credentials when a client follows a redirect to a cross-origin storage URL.

요청 매개변수

이름	위치	유형	필수	설명
`filePath`	query	`string`	예	Server-issued storage path returned by an upload API.
`responseContentDisposition`	query	`string`	아니요	Optional Content-Disposition value for the download response.

응답

`302` `400` `401` `403` `404` `409` `413` `500`

응답 상태 의미

상태	의미	미디어 유형
`302`	Redirect to the final download URL.	—
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	—
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	—
`413`	Request too large. Reduce the request before retrying.	—
`500`	Service error. Investigate the service and confirm the outcome before retrying.	—

`POST` `/assets/files/multipart/abort`

Abort a multipart upload

Cancels a multipart upload session.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/abort' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileKey":"string","uploadId":"string"}'
```

`application/json`

필드	유형	필수	설명
fileKey	string	예	Object-store key used for the multipart upload.
uploadId	string	예	Object-store multipart upload session ID.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "data": {
    "aborted": 1
  },
  "msg": "string"
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	AssetFileMultipartAbortResp. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–

상태	의미	미디어 유형
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/multipart/complete

Complete a multipart upload
Completes a multipart upload after every uploaded part number and ETag have been supplied.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/complete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileKey":"string","uploadId":"string","parts":[{"partNumber":"value","etag":"value"}]}'
```

application/json

필드	유형	필수	설명
fileKey	string	예	Object-store key used for the multipart upload.
uploadId	string	예	Object-store multipart upload session ID.
parts	array<object>	예	Uploaded part numbers and their ETags.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "data": {
    "filePath": "string",
    "fileUrl": "string",
    "sizeBytes": 0,
    "expiresAt": 0
  },
  "msg": "string"
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	AssetFileMultipartCompleteResp. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/multipart/init

Initialize a multipart upload
Creates a multipart upload session and returns its ID, part size, and part count.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/init' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileName":"string","contentType":"string","size":0,"partSize":0,"business":"string"}'
```

application/json

필드	유형	필수	설명
fileName	string	예	Original file name.
contentType	string	아니요	File MIME type, for example application/octet-stream.

필드	유형	필수	설명
size	integer (int64)	예	Total file size in bytes, greater than 0 and at most 1 GiB.
partSize	integer (int64)	아니요	Preferred part size in bytes. The server may normalize it.
business	string	아니요	Business domain or feature associated with the file.
useBy	string	아니요	Optional usage category: user, workspace, or platform.
visibility	string	아니요	File visibility: private or public.
appInstanceId	string	아니요	Associated application instance ID.
sessionId	string	아니요	Caller session ID.
bizId	string	아니요	Associated business identifier.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "data": {
    "fileKey": "string",
    "filePath": "string",
    "uploadId": "string",
    "partSize": 0,
    "partCount": 0
  },
  "msg": "string"
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	AssetFileMultipartInitResp. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/multipart/part-urls

Create multipart upload URLs
Returns presigned PUT URLs for the requested one-based part numbers.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/part-urls' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileKey":"string","uploadId":"string","partNumbers":[0]}'
```

application/json

필드	유형	필수	설명
fileKey	string	예	Object-store key used for the multipart upload.
uploadId	string	예	Object-store multipart upload session ID.
partNumbers	array<integer (int32)>	예	One-based part numbers to sign.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "data": {
    "partUrls": [
      "value"
    ]
  },
  "msg": "string"
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	AssetFileMultipartPartUrlsResp. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

GET /assets/files/public

Download a public file

Downloads bytes for a file explicitly marked public. An unguessable path is not authorization; do not mark sensitive files public.

요청 매개변수

이름	위치	유형	필수	설명
`filePath`	query	`string`	예	Server-issued storage path returned by an upload API.
`responseContentDisposition`	query	`string`	아니요	Optional Content-Disposition value for the download response.

응답

`200` `400` `404` `409` `413` `500`

응답 상태 의미

상태	의미	미디어 유형
`200`	Returns file bytes directly. Content-Type depends on the file type; no JSON envelope is used.	–
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

`GET` `/assets/files/url`

Create a temporary file URL
Creates a temporary URL for a file.

요청 매개변수

이름	위치	유형	필수	설명
`filePath`	query	`string`	예	Server-issued storage path returned by an upload API.
`expiredSec`	query	`integer (int64)`	아니요	Requested temporary URL lifetime in seconds.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "data": {
    "fileUrl": "string",
    "expiresAt": 0
  },
  "msg": "string"
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	AssetFileUrlResp. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/upload

Create a legacy asset upload URL
Legacy compatibility endpoint. The current Enterprise implementation returns an empty uploadUrl; use /assets/files for new integrations.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/upload' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileName":"string","sizeBytes":0,"contentType":"string","size":0,"sha256":"string"}'
```

application/json

필드	유형	필수	설명
fileName	string	예	Original file name.
sizeBytes	integer (int64)	예	File size in bytes.
contentType	string	아니요	File MIME type, for example application/octet-stream.
size	integer (int64)	아니요	File size in bytes.
sha256	string	아니요	Optional SHA-256 checksum of the file content.
storageKey	string	아니요	Optional object-store key. Omit it to let the server generate one.
isPublic	boolean (boolean)	아니요	Whether the file should be accessible through the public download endpoint.
business	string	아니요	Business domain or feature that owns the uploaded file.
useBy	string	아니요	Usage scenario or owner category for the uploaded file.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "uploadUrl": "string",
    "filePath": "string",
    "fileUrl": "string"
  }
}
```


성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
msg	string	아니요	Response message
data	object	아니요	AssetUploadResp. 스키마 절에서 참조된 스키마를 확인하세요.

응답 상태 의미

상태	의미	미디어 유형
200	Legacy upload response. The current Enterprise implementation returns an empty uploadUrl; use /assets/files for new integrations.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

인증

현재 OpenAPI ID 및 권한 확인.

POST `/auth/whoami`

Inspect the current API key identity

Returns the current API-key identity and its effective permissions. Call this before integration work to verify access.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json


```
{
  "code": 0,
  "data": {
    "userID": 0,
    "userName": "string",
    "email": "string",
    "workspaceID": 0,
    "workspaceName": "string"
  },
  "msg": "string"
}
```


성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	AuthWhoamiResp. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain

상태	의미	미디어 유형
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

Flow

Source Flow 및 Event Flow 정의 관리.

POST `/flow/create`

Create a flow
Creates a SourceFlow or EventFlow and returns its numeric resource ID.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/create' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"flowName":"string","flowType":"string","description":"string","template":"string"}'
```

application/json

필드	유형	필수	설명
flowName	string	예	Flow name, up to 63 Unicode characters.
flowType	string	예	Flow category: SourceFlow or EventFlow.
description	string	아니요	Flow description.
template	string	아니요	Optional flow template JSON.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "data": {
    "id": 0,
    "brokerID": "string"
  },
  "msg": "string"
}
```


성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	FlowCreateResp. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/delete

Delete flows

Deletes the listed flows sequentially; earlier deletions are not rolled back when a later item fails.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"ids":[0]}'
```

application/json

필드	유형	필수	설명
ids	array<integer (int64)>	예	Flow IDs to delete.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "data": {
    "success": false
  },
  "msg": "string"
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	FlowEmptyResp. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain

상태	의미	미디어 유형
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/deploy

Deploy a flow

Deploys a complete Node-RED canvas JSON string to a flow.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/deploy' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0,"flowsJson":"string"}'
```

application/json

필드	유형	필수	설명
id	integer (int64)	예	Flow ID to deploy.
flowsJson	string	예	Node-RED canvas JSON to deploy.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "data": {
    "flowId": "string"
  },
  "msg": "string"
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	FlowDeployResp. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/flowdata

Get flow canvas data

Returns the Node-RED canvas revision and node objects for a flow.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/flowdata' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0}'
```

application/json

필드	유형	필수	설명
id	integer (int64)	예	Flow ID.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json


```
{
  "code": 0,
  "data": {
    "rev": "string",
    "flows": [
      "value"
    ]
  },
  "msg": "string"
}
```


성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	FlowDataResp. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–

상태	의미	미디어 유형
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/get

Get a flow
Returns flow metadata for a numeric flow ID.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/get' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0}'
```

application/json

필드	유형	필수	설명
id	integer (int64)	예	Flow ID.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "data": {
    "id": 0,
    "flowId": "string",
    "flowName": "string",
    "flowType": "string",
    "flowStatus": "string"
  },
  "msg": "string"
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	FlowInfo. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/list

List flows
Lists flows, optionally filtered by a keyword or flow category.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/list' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"keyword":"string","flowType":"string"}'
```

application/json

필드	유형	필수	설명
keyword	string	아니요	Keyword matched against flow names and descriptions.
flowType	string	아니요	Flow category filter: SourceFlow or EventFlow.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json


```
{
  "code": 200,
  "msg": "success",
  "data": {
    "list": [
      {
        "id": 101,
        "flowId": "flow-101",
        "flowName": "Line 1 collector",
        "flowType": "source",
        "flowStatus": "draft",
        "template": "[]",
        "description": "Collects line telemetry",
        "isFavorite": 2,
        "currentVersionId": 0,
        "currentVersionName": "",
        "currentVersionType": "",
        "createdTime": 1788134400000,
        "updatedTime": 1788134400000
      }
    ]
  }
}
```


성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	FlowListResp. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain

상태	의미	미디어 유형
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/nodes

List available flow node types

Returns the Node-RED node sets available for a flow category.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/nodes' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"flowType":"string"}'
```

application/json

필드	유형	필수	설명
flowType	string	예	Flow category whose Node-RED node catalog should be returned.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "data": {
    "nodes": [
      "value"
    ]
  },
  "msg": "string"
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	FlowNodesResp. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/update

Update a flow
Updates flow metadata and settings for a numeric flow ID.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/update' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0,"flowName":"string","description":"string","template":"string","isFavorite":0}'
```

application/json

필드	유형	필수	설명
id	integer (int64)	예	Flow ID to update.
flowName	string	아니요	Updated flow name.
description	string	아니요	Updated flow description.
template	string	아니요	Updated flow template JSON.
isFavorite	integer (int64)	아니요	Favorite state: 1 for favorite, 2 for not favorite.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "data": {
    "success": false
  },
  "msg": "string"
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	FlowEmptyResp. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain

상태	의미	미디어 유형
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

플랫폼 및 프로젝트

플랫폼 정보 읽기 및 프로젝트 관리.

POST /info

Get service information
Returns the service name, version, reported capabilities, and configured MQTT broker.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/info' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{}'
```

application/json
OpenAPI에 이 요청 본문의 객체 필드가 선언되어 있지 않습니다.

응답

200 400 401 403 404 409 413 500

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	–
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain

상태	의미	미디어 유형
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /launchpad/{projectName}/getMembers

List project members
Lists Launchpad project members. URL-encode projectName when it contains special characters.

요청 매개변수

이름	위치	유형	필수	설명
projectName	path	string	예	Project name in the Launchpad URL. URL-encode special characters.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/launchpad/{projectName}/getMembers' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"roleKey":"string","roles":
["string"],"updatedAtStart":"string","updatedAtEnd":"string","page":0}'
```

application/json

필드	유형	필수	설명
roleKey	string	아니요	Role key filter.
roles	array<string>	아니요	Role filters or the returned role list, depending on the model.
updatedAtStart	string	아니요	Inclusive update-time lower bound in RFC 3339 format.
updatedAtEnd	string	아니요	Inclusive update-time upper bound in RFC 3339 format.
page	integer (int64)	아니요	Page number, starting at 1.
size	integer (int64)	아니요	Number of members per page.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "data": {
    "list": [
      "value"
    ],
    "total": 0,
    "page": 0,
    "size": 0
  },
  "msg": "string"
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	LaunchpadMembersResp. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /platform/getMembers

List platform members
Lists platform members with optional keyword, role, status, and update-time filters.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/platform/getMembers' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"keyword":"string","roleKey":"string","roles":["string"],"statuses":
["string"],"updatedAtStart":"string"}'
```

application/json

필드	유형	필수	설명
keyword	string	아니요	Search keyword.
roleKey	string	아니요	Role key filter.
roles	array<string>	아니요	Role filters or the returned role list, depending on the model.
statuses	array<string>	아니요	Member status filters: active or disabled.
updatedAtStart	string	아니요	Inclusive update-time lower bound in RFC 3339 format.
updatedAtEnd	string	아니요	Inclusive update-time upper bound in RFC 3339 format.
page	integer (int64)	아니요	Page number, starting at 1.
size	integer (int64)	아니요	Number of members per page.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "data": {
    "list": [
      "value"
    ],
    "total": 0,
    "page": 0,
    "size": 0
  },
  "msg": "string"
}
```


성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	PlatformMembersResp. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

MQTT 인증

MQTT 자격 증명 생성 및 삭제.

POST /mqtt-auth/create

Create an MQTT credential
Creates a normal MQTT credential. The returned password is available only once and must be stored securely immediately.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/mqtt-auth/create' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"name":"string","description":"string","clientIDRandomSuffixEnabled":false}'
```

application/json

필드	유형	필수	설명
name	string	예	MQTT credential name.
description	string	아니요	Description of the MQTT credential.
clientIDRandomSuffixEnabled	boolean	아니요	Whether to append a random suffix to the MQTT client ID.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "data": {
    "id": 0,
    "name": "string",
    "clientID": "string",
    "username": "string",
    "password": "string"
  },
  "msg": "string"
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code

필드	유형	필수	설명
data	object	아니요	MqttAuthCreateResp. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /mqtt-auth/delete

Delete an MQTT credential

Idempotently deletes a normal MQTT credential. System credentials are not deleted.

요청 본문

```
curl --request POST 'https://<tier0-host>/openapi/v1/mqtt-auth/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0}'
```

application/json

필드	유형	필수	설명
id	integer (int64)	예	Resource ID.

응답

200 400 401 403 404 409 413 500

성공 응답 예제

200 · application/json

```
{
  "code": 0,
  "data": {
    "success": false
  },
  "msg": "string"
}
```

성공 응답 필드

200 · application/json

필드	유형	필수	설명
code	integer	아니요	Response code
data	object	아니요	MqttAuthDeleteResp. 스키마 절에서 참조된 스키마를 확인하세요.
msg	string	아니요	Response message

응답 상태 의미

상태	의미	미디어 유형
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

스키마

API 필드 이름, 예제 및 원본 설명은 포함된 영어 OpenAPI 명세를 따릅니다.

AssetFileDeleteReq

필드	유형	필수	설명
`filePath`	`string`	예	Server-issued storage path returned by an upload API.

AssetFileDeleteResp

필드	유형	필수	설명
`deleted`	`boolean`	예	Whether the file was deleted.

AssetFileMultipartAbortReq

필드	유형	필수	설명
`fileKey`	`string`	예	Object-store key used for the multipart upload.
`uploadId`	`string`	예	Object-store multipart upload session ID.

AssetFileMultipartAbortResp

필드	유형	필수	설명
`aborted`	`integer (int32)`	예	Abort result: 1 = aborted (including idempotent replay success), 2 = not aborted.

AssetFileMultipartCompleteReq

필드	유형	필수	설명
`fileKey`	`string`	예	Object-store key used for the multipart upload.
`uploadId`	`string`	예	Object-store multipart upload session ID.
`parts`	`array<object>`	예	Uploaded part numbers and their ETags.

AssetFileMultipartCompleteResp

필드	유형	필수	설명
`filePath`	`string`	예	Server-issued storage path returned by an upload API.
`fileUrl`	`string`	아니요	File access URL.
`sizeBytes`	`integer (int64)`	예	File size in bytes.
`expiresAt`	`integer (int64)`	아니요	Expiration time, in Unix milliseconds.

AssetFileMultipartInitReq

필드	유형	필수	설명
`fileName`	`string`	예	Original file name.
`contentType`	`string`	아니요	File MIME type, for example application/octet-stream.
`size`	`integer (int64)`	예	Total file size in bytes, greater than 0 and at most 1 GiB.
`partSize`	`integer (int64)`	아니요	Preferred part size in bytes. The server may normalize it.
`business`	`string`	아니요	Business domain or feature associated with the file.
`useBy`	`string`	아니요	Optional usage category: user, workspace, or platform.
`visibility`	`string`	아니요	File visibility: private or public.
`appInstanceId`	`string`	아니요	Associated application instance ID.
`sessionId`	`string`	아니요	Caller session ID.
`bizId`	`string`	아니요	Associated business identifier.

AssetFileMultipartInitResp

필드	유형	필수	설명
`fileKey`	`string`	예	Object-store key used for the multipart upload.
`filePath`	`string`	예	Server-issued storage path returned by an upload API.
`uploadId`	`string`	예	Object-store multipart upload session ID.
`partSize`	`integer (int64)`	예	Part size in bytes; the server may normalize it.
`partCount`	`integer (int64)`	예	Total number of upload parts.
`expiresAt`	`integer (int64)`	예	Expiration time, in Unix milliseconds.

AssetFileMultipartPart

필드	유형	필수	설명
`partNumber`	`integer (int32)`	예	Part number, starting at 1.
`etag`	`string`	예	ETag returned after uploading this part.

AssetFileMultipartPartUrl

필드	유형	필수	설명
partNumber	integer (int32)	예	Part number, starting at 1.
url	string	예	Presigned PUT URL for this part.
expiresAt	integer (int64)	예	Expiration time, in Unix milliseconds.

AssetFileMultipartPartUrlsReq

필드	유형	필수	설명
fileKey	string	예	Object-store key used for the multipart upload.
uploadId	string	예	Object-store multipart upload session ID.
partNumbers	array<integer (int32)>	예	One-based part numbers to sign.

AssetFileMultipartPartUrlsResp

필드	유형	필수	설명
partUrls	array<object>	예	Presigned URLs for the requested parts.

AssetFileUploadReq

필드	유형	필수	설명
fileName	string	예	Original file name.
contentType	string	아니요	MIME content type.
size	integer (int64)	예	File size in bytes. Business validation requires a value greater than 0 and at most 1 GiB.
business	string	아니요	Business domain or feature associated with the file.
useBy	string	아니요	Optional usage category: user, workspace, or platform.
visibility	string	아니요	File visibility, such as private or public.
appInstanceId	string	아니요	Associated application instance ID.

필드	유형	필수	설명
sessionId	string	아니요	Caller session ID.

AssetFileUploadResp

필드	유형	필수	설명
fileId	integer (int64)	아니요	File ID.
filePath	string	예	Server-issued storage path returned by an upload API.
fileUrl	string	아니요	File access URL.
uploadUrl	string	예	Upload URL; the legacy Enterprise endpoint currently returns an empty string.
expiresAt	integer (int64)	아니요	Expiration time, in Unix milliseconds.

AssetFileUrlResp

필드	유형	필수	설명
fileUrl	string	예	File access URL.
expiresAt	integer (int64)	아니요	Expiration time, in Unix milliseconds.

AssetUploadReq

AssetUploadReq

필드	유형	필수	설명
fileName	string	예	Original file name.
sizeBytes	integer (int64)	예	File size in bytes.
contentType	string	아니요	File MIME type, for example application/octet-stream.
size	integer (int64)	아니요	File size in bytes.
sha256	string	아니요	Optional SHA-256 checksum of the file content.
storageKey	string	아니요	Optional object-store key. Omit it to let the server generate one.

필드	유형	필수	설명
isPublic	boolean (boolean)	아니요	Whether the file should be accessible through the public download endpoint.
business	string	아니요	Business domain or feature that owns the uploaded file.
useBy	string	아니요	Usage scenario or owner category for the uploaded file.

AssetUploadResp

AssetUploadResp

필드	유형	필수	설명
uploadUrl	string	예	Upload URL; the legacy Enterprise endpoint currently returns an empty string.
filePath	string	예	Server-issued storage path returned by an upload API.
fileUrl	string	아니요	File access URL.

AuthWhoamiResp

필드	유형	필수	설명
userID	integer (int64)	예	Numeric ID of the API key owner.
userName	string	예	User name.
email	string	예	Email address.
workspaceID	integer (int64)	예	Platform workspace ID.
workspaceName	string	예	Workspace name; may be empty in Enterprise.
apiKeyName	string	예	Name of the current API key.
keyPrefix	string	예	API key prefix.
permissions	array<string>	예	Effective permissions of the current identity.
roles	array<string>	예	Role filters or the returned role list, depending on the model.
keyType	string	예	API key ownership type, for example personal.

BrowseReq

BrowseReq

필드	유형	필수	설명
path	string	아니요	Root UNS path. Empty means the workspace root.
max_depth	integer (int64)	아니요	Maximum tree depth. Omit or use 0 for the server default.
include_metadata	boolean (boolean)	아니요	Whether returned nodes should include metadata.
include_leaf_value	boolean (boolean)	아니요	Whether returned leaf nodes should include their latest payload.

BrowseResp

BrowseResp

필드	유형	필수	설명
tree	array<object>	예	UNS node tree.

CreateResp

CreateResp

필드	유형	필수	설명
success	boolean (boolean)	예	Whether the operation succeeded.
results	array<object>	예	Per-item results. Check success and error for each item.

FlowCreateReq

필드	유형	필수	설명
flowName	string	예	Flow name, up to 63 Unicode characters.
flowType	string	예	Flow category: SourceFlow or EventFlow.
description	string	아니요	Flow description.
template	string	아니요	Optional flow template JSON.

FlowCreateResp

필드	유형	필수	설명
id	integer (int64)	예	Resource ID.
brokerID	string	예	ID of the MQTT broker configuration node injected when the flow is created.

FlowDataResp

필드	유형	필수	설명
rev	string	예	Node-RED canvas revision; may be an empty string.
flows	array<object>	예	Array of Node-RED canvas node objects.

FlowDeleteReq

필드	유형	필수	설명
ids	array<integer (int64)>	예	Flow IDs to delete.

FlowDeployReq

필드	유형	필수	설명
id	integer (int64)	예	Flow ID to deploy.
flowsJson	string	예	Node-RED canvas JSON to deploy.

FlowDeployResp

필드	유형	필수	설명
flowId	string	예	Flow identifier.

FlowEmptyResp

필드	유형	필수	설명
success	boolean	예	Whether the operation succeeded.

FlowGetReq

필드	유형	필수	설명
id	integer (int64)	예	Flow ID.

FlowInfo

필드	유형	필수	설명
id	integer (int64)	예	Resource ID.
flowId	string	예	Flow identifier.
flowName	string	예	Flow name.
flowType	string	예	Response flow type: source or event.
flowStatus	string	예	Current flow status, for example draft.
template	string	예	Flow template JSON, encoded as a string.
description	string	예	Description.
isFavorite	integer (int64)	예	Favorite state: 1 for favorite, 2 for not favorite.
currentVersionId	integer (int64)	예	Current version ID; 0 if unset.
currentVersionName	string	예	Current version name; empty if unset.
currentVersionType	string	예	Current version type; empty if unset.
createdTime	integer (int64)	예	Creation time, in Unix milliseconds.
updatedTime	integer (int64)	예	Last update time, in Unix milliseconds.

FlowListReq

필드	유형	필수	설명
keyword	string	아니요	Keyword matched against flow names and descriptions.
flowType	string	아니요	Flow category filter: SourceFlow or EventFlow.

FlowListResp

필드	유형	필수	설명
list	array<object>	예	Result list.

FlowNodesReq

필드	유형	필수	설명
flowType	string	예	Flow category whose Node-RED node catalog should be returned.

FlowNodesResp

필드	유형	필수	설명
nodes	array<object>	예	Available Node-RED node sets.

FlowUpdateReq

필드	유형	필수	설명
id	integer (int64)	예	Flow ID to update.
flowName	string	아니요	Updated flow name.
description	string	아니요	Updated flow description.
template	string	아니요	Updated flow template JSON.
isFavorite	integer (int64)	아니요	Favorite state: 1 for favorite, 2 for not favorite.

HistoryAggregation

HistoryAggregation

필드	유형	필수	설명
interval	string	예	Positive aggregation window, such as 1m, 5m, 1h, or 1d.
function	string	아니요	Aggregation function: avg, min, max, sum, count, first, or last.
field	string	아니요	First-level payload field to aggregate.
fields	array<object>	아니요	Per-field aggregation settings.

HistoryAggregationField

HistoryAggregationField

필드	유형	필수	설명
name	string	예	Payload field to aggregate.
function	string	아니요	Function for this field; defaults to the outer aggregation function.

HistoryQueryMeta

HistoryQueryMeta

필드	유형	필수	설명
aggregated	boolean (boolean)	예	Whether the result was aggregated.
sampled	boolean (boolean)	예	Whether the result was sampled.
rawTotal	integer (int64)	예	Total number of raw records.
returnedTotal	integer (int64)	예	Number of records returned in this response.
requestedIntervalMs	integer (int64)	아니요	Requested aggregation interval in milliseconds.
actualIntervalMs	integer (int64)	아니요	Actual aggregation interval in milliseconds.
strategy	string	아니요	Processing strategy used for the query.

HistoryReq

HistoryReq

필드	유형	필수	설명
topics	array<string>	예	UNS topic paths whose history should be queried.
start_time	string	예	Inclusive query start in RFC 3339 format.
end_time	string	예	Inclusive query end in RFC 3339 format.
aggregation	object	아니요	HistoryAggregation. 스키마 절에서 참조된 스키마를 확인하세요.
page	integer (int64)	아니요	Page number, starting from 1.
size	integer (int64)	아니요	Page size; values above the server maximum are capped.

HistoryResp

HistoryResp

필드	유형	필수	설명
success	boolean (boolean)	예	Whether the operation succeeded.
results	array<object>	예	Per-item results. Check success and error for each item.
page	integer (int64)	예	Page number, starting at 1.
size	integer (int64)	예	File size in bytes.
total	integer (int64)	예	Total number of matching records.

InfoReq

```
{
  "type": "object",
  "properties": {}
}
```

LaunchpadGetMembersReq

필드	유형	필수	설명
roleKey	string	아니요	Role key filter.
roles	array<string>	아니요	Role filters or the returned role list, depending on the model.
updatedAtStart	string	아니요	Inclusive update-time lower bound in RFC 3339 format.
updatedAtEnd	string	아니요	Inclusive update-time upper bound in RFC 3339 format.
page	integer (int64)	아니요	Page number, starting at 1.
size	integer (int64)	아니요	Number of members per page.

LaunchpadMemberResp

필드	유형	필수	설명
memberId	string	예	Member identifier.
userId	string	예	User ID.
userName	string	아니요	User name.
email	string	아니요	Email address.
roles	array<object>	예	Role filters or the returned role list, depending on the model.
updatedAt	string	아니요	Last update time.

LaunchpadMemberRoleAppResp

필드	유형	필수	설명
appId	string	예	Application ID, represented as a string.
appName	string	예	Application name.

LaunchpadMemberRoleResp

필드	유형	필수	설명
roleId	string	예	Role ID, represented as a string.
roleKey	string	예	Role key filter.
roleName	string	아니요	Role display name.
description	string	아니요	Description.
memberCount	integer (int64)	예	Number of members assigned to the role.
apps	array<object>	아니요	Applications associated with the role.

LaunchpadMembersResp

필드	유형	필수	설명
list	array<object>	예	Result list.
total	integer (int64)	예	Total number of matching records.
page	integer (int64)	예	Page number, starting at 1.
size	integer (int64)	예	File size in bytes.

MqttAuthCreateReq

필드	유형	필수	설명
name	string	예	MQTT credential name.
description	string	아니요	Description of the MQTT credential.
clientIDRandomSuffixEnabled	boolean	아니요	Whether to append a random suffix to the MQTT client ID.

MqttAuthCreateResp

필드	유형	필수	설명
id	integer (int64)	예	Resource ID.
name	string	예	Name.
clientID	string	예	MQTT client ID.
username	string	예	MQTT user name.

필드	유형	필수	설명
password	string	예	Plaintext password. Save it now; it cannot be retrieved again.
clientIDRandomSuffixEnabled	boolean	예	Whether to append a random suffix to the MQTT client ID.

MqttAuthDeleteReq

필드	유형	필수	설명
id	integer (int64)	예	Resource ID.

MqttAuthDeleteResp

필드	유형	필수	설명
success	boolean	예	Whether the operation succeeded.

NamespaceNode

NamespaceNode

필드	유형	필수	설명
name	string	예	Node name, up to 63 Unicode characters. In a nested request, this is the name under its parent.
type	string	아니요	Node type: PATH or TOPIC.
children	array<object>	아니요	Child nodes to create under this node.
topicType	string	아니요	Topic data category: STATE, ACTION, or METRIC.
fields	array<object>	아니요	Payload schema fields for TOPIC nodes.
description	string	아니요	Node description, up to 512 Unicode characters.
displayName	string	아니요	Display name shown in clients.
extendProperties	object	아니요	Custom metadata key-value pairs.

필드	유형	필수	설명
alias	string	아니요	Optional topic alias.
enableHistory	boolean (boolean)	아니요	Whether history storage is enabled for this topic.

NodeCreateReq

NodeCreateReq

필드	유형	필수	설명
namespace	array<object>	예	UNS nodes to create. Nest children to create a subtree.

NodeDeleteReq

NodeDeleteReq

필드	유형	필수	설명
topics	array<string>	예	UNS topic or path node paths to delete.
hard_delete	boolean (boolean)	아니요	Permanently delete instead of moving nodes to the recycle state.

NodeRedNodeSet

필드	유형	필수	설명
id	string	예	Resource ID.
name	string	예	Name.
types	array<string>	예	Node-RED node types provided by the node set.
enabled	boolean	예	Whether the node set is enabled.
module	string	예	Node-RED module name.
version	string	예	Service or node module version.

NodeRestoreReq

NodeRestoreReq

필드	유형	필수	설명
path	string	예	Full UNS path of the node to restore. Use its path before deletion, not a node ID or HTTP URL.

NodeUpdateReq

NodeUpdateReq

필드	유형	필수	설명
path	string	예	Existing UNS topic or path node to update.
name	string	아니요	New node name.
description	string	아니요	Updated node description.
displayName	string	아니요	Updated display name.
extendProperties	object	아니요	Updated custom metadata key-value pairs.
fields	array<object>	아니요	Updated payload schema fields.
alias	string	아니요	Updated topic alias.
enableHistory	boolean (boolean)	아니요	Supply true or false to change history storage. Omit to keep the existing setting.
updateMask	array<string>	아니요	Compatibility field ignored by the current Enterprise implementation. It cannot be used to clear fields.

OpenapiCreateResult

OpenapiCreateResult

필드	유형	필수	설명
success	boolean (boolean)	예	Whether the operation succeeded.
topic	string	예	Full UNS topic path.
error	object	아니요	OpenapiItemError. 스키마 절에서 참조된 스키마를 확인하세요.

OpenapiEmptyResp

OpenapiEmptyResp

필드	유형	필수	설명
success	boolean (boolean)	예	Whether the operation succeeded.

OpenapiHistoryItem

OpenapiHistoryItem

필드	유형	필수	설명
success	boolean (boolean)	예	Whether the operation succeeded.
topic	string	예	Full UNS topic path.
result	object	아니요	OpenapiHistoryResult. 스키마 절에서 참조된 스키마를 확인하세요.
error	object	아니요	OpenapiItemError. 스키마 절에서 참조된 스키마를 확인하세요.

OpenapiHistoryResult

OpenapiHistoryResult

필드	유형	필수	설명
values	array<object>	예	Historical VQT records.
meta	object	아니요	HistoryQueryMeta. 스키마 절에서 참조된 스키마를 확인하세요.

OpenapiHistoryValue

OpenapiHistoryValue

필드	유형	필수	설명
value	object	아니요	Payload value.
quality	string	예	Data quality: Good, GoodNoData, Uncertain, or Bad.
timeStamp	integer (int64)	예	Unix timestamp in milliseconds. Writes use server time when omitted.

OpenapiItemError

OpenapiItemError

필드	유형	필수	설명
code	integer (int64)	예	Status code; in an item error, identifies the failure.
message	string	예	Item-level error message.

OpenapiNodeInfo

OpenapiNodeInfo

필드	유형	필수	설명
id	integer (int64)	예	Resource ID.
path	string	예	Full UNS node path.
name	string	예	Name.
type	string	예	Node type: PATH or TOPIC.
topicType	string	예	Topic category: STATE, ACTION, or METRIC.
description	string	아니요	Description.
displayName	string	아니요	Display name.
extendProperties	object	아니요	Custom node metadata as key-value pairs.
fields	array<object>	아니요	Payload field definitions.
children	array<object>	아니요	Child nodes under this node.
alias	string	아니요	Node alias.
enableHistory	boolean (boolean)	아니요	Whether history storage is enabled.
bindFlowID	integer (int64)	아니요	Bound Source Flow ID. 0 means no Source Flow is bound.
bindEventFlowID	integer (int64)	아니요	Bound Event Flow ID. 0 means no Event Flow is bound.
payload	object	아니요	OpenapiVQT. 스키마 절에서 참조된 스키마를 확인하세요.

OpenapiReadItem

OpenapiReadItem

필드	유형	필수	설명
success	boolean (boolean)	예	Whether the operation succeeded.
topic	string	예	Full UNS topic path.
result	object	아니요	OpenapiVQT. 스키마 절에서 참조된 스키마를 확인하세요.

필드	유형	필수	설명
error	object	아니요	OpenapiItemError. 스키마 절에서 참조된 스키마를 확인하세요.
metadata	object	아니요	OpenapiNodeInfo. 스키마 절에서 참조된 스키마를 확인하세요.

OpenapiVQT

OpenapiVQT

필드	유형	필수	설명
value	object	아니요	Payload value.
quality	string	예	Data quality: Good, GoodNoData, Uncertain, or Bad.
timeStamp	integer (int64)	예	Unix timestamp in milliseconds. Writes use server time when omitted.

OpenapiWriteResult

OpenapiWriteResult

필드	유형	필수	설명
success	boolean (boolean)	예	Whether the operation succeeded.
topic	string	예	Full UNS topic path.
error	object	아니요	OpenapiItemError. 스키마 절에서 참조된 스키마를 확인하세요.

PlatformGetMembersReq

필드	유형	필수	설명
keyword	string	아니요	Search keyword.
roleKey	string	아니요	Role key filter.
roles	array<string>	아니요	Role filters or the returned role list, depending on the model.
statuses	array<string>	아니요	Member status filters: active or disabled.
updatedAtStart	string	아니요	Inclusive update-time lower bound in RFC 3339 format.
updatedAtEnd	string	아니요	Inclusive update-time upper bound in RFC 3339 format.
page	integer (int64)	아니요	Page number, starting at 1.
size	integer (int64)	아니요	Number of members per page.

PlatformMemberResp

필드	유형	필수	설명
userId	string	예	User ID.
userName	string	아니요	User name.
nickName	string	아니요	User nickname.
email	string	아니요	Email address.
status	string	예	Platform member status: active or disabled.
roles	array<object>	예	Role filters or the returned role list, depending on the model.
updatedAt	string	예	Last update time.

PlatformMemberRoleResp

필드	유형	필수	설명
roleId	string	예	Role ID, represented as a string.
roleKey	string	예	Role key filter.
roleName	string	아니요	Role display name.
description	string	아니요	Description.

PlatformMembersResp

필드	유형	필수	설명
list	array<object>	예	Result list.
total	integer (int64)	예	Total number of matching records.
page	integer (int64)	예	Page number, starting at 1.
size	integer (int64)	예	File size in bytes.

ReadReq

ReadReq

필드	유형	필수	설명
topics	array<string>	예	UNS topic paths to read.
include_metadata	boolean (boolean)	아니요	Whether each result should include UNS node metadata.

필드	유형	필수	설명
include_leaf_value	boolean (boolean)	아니요	Whether leaf nodes should include their latest payload.

ReadResp

ReadResp

필드	유형	필수	설명
success	boolean (boolean)	예	Whether the operation succeeded.
results	array<object>	예	Per-item results. Check success and error for each item.

SchemaField

SchemaField

필드	유형	필수	설명
name	string	예	Payload field name.
type	string	예	Payload field type: INTEGER, LONG, DOUBLE, FLOAT, STRING, BOOLEAN, or DATETIME.
unit	string	아니요	Optional engineering unit.

SearchReq

SearchReq

필드	유형	필수	설명
keyword	string	아니요	Keyword matched against UNS node names, aliases, and paths.
topicType	string	아니요	Topic data category filter: STATE, ACTION, or METRIC.
path_prefix	string	아니요	Only return nodes under this UNS path prefix.
include_metadata	boolean (boolean)	아니요	Whether returned nodes should include metadata.
include_leaf_value	boolean (boolean)	아니요	Whether returned leaf nodes should include their latest payload.
page	integer (int64)	아니요	Page number, starting from 1.
size	integer (int64)	아니요	Page size.

SearchResp

SearchResp

필드	유형	필수	설명
objects	array<object>	예	Matching UNS nodes.
page	integer (int64)	예	Page number, starting at 1.
size	integer (int64)	예	File size in bytes.
total	integer (int64)	예	Total number of matching records.

UnsAttachmentInfo

UnsAttachmentInfo

필드	유형	필수	설명
unsId	integer (int64)	예	Numeric UNS node ID.
fileName	string	아니요	Original file name.
filePath	string	예	Server-issued storage path returned by an upload API.
fileUrl	string	아니요	File access URL.
contentType	string	아니요	File MIME type, for example application/octet-stream.
size	integer (int64)	아니요	File size in bytes.
sha256	string	아니요	SHA-256 checksum of the file contents.
createdAt	integer (int64)	아니요	Creation time, in Unix milliseconds.
fileUrlExpiresAt	integer (int64)	아니요	File URL expiration time, in Unix milliseconds.

UnsAttachmentListReq

UnsAttachmentListReq

필드	유형	필수	설명
unsId	integer (int64)	아니요	Numeric UNS node ID.
topic	string	아니요	Full UNS topic path.
page	integer (int64)	아니요	Page number, starting at 1.
size	integer (int64)	아니요	Number of attachments per page.
includeFileUrl	boolean (boolean)	아니요	Whether to return temporary file URLs. Defaults to true.

UnsAttachmentListResp

UnsAttachmentListResp

필드	유형	필수	설명
unsId	integer (int64)	예	Numeric UNS node ID.
list	array<object>	예	Result list.
total	integer (int64)	예	Total number of matching records.
page	integer (int64)	예	Page number, starting at 1.
size	integer (int64)	예	File size in bytes.

UnsAttachmentUploadReq

UnsAttachmentUploadReq

필드	유형	필수	설명
unsId	integer (int64)	아니요	Numeric owner node ID. Supply unsId or topic; a valid unsId takes precedence.
topic	string	아니요	Full owner node path, used when a valid unsId is not supplied.
fileName	string	아니요	Original file name.
sha256	string	아니요	SHA-256 checksum of the file contents.
file	string (binary)	예	Binary file contents. The multipart field name must be file.

UnsAttachmentUploadResp

UnsAttachmentUploadResp

필드	유형	필수	설명
unsId	integer (int64)	예	Numeric UNS node ID.
fileName	string	아니요	Original file name.
filePath	string	예	Server-issued storage path returned by an upload API.
fileUrl	string	아니요	File access URL.
contentType	string	아니요	File MIME type, for example application/octet-stream.

필드	유형	필수	설명
size	integer (int64)	아니요	File size in bytes.
sha256	string	아니요	SHA-256 checksum of the file contents.
createdAt	integer (int64)	아니요	Creation time, in Unix milliseconds.
fileUrlExpiresAt	integer (int64)	아니요	File URL expiration time, in Unix milliseconds.

UnsBindFlowReq

UnsBindFlowReq

필드	유형	필수	설명
unsId	integer (int64)	예	UNS node ID to bind.
flowId	integer (int64)	예	Flow ID to bind.

UnsBindFlowResp

UnsBindFlowResp

필드	유형	필수	설명
success	boolean (boolean)	예	Whether the operation succeeded.
unsId	integer (int64)	예	Numeric UNS node ID.
flowId	integer (int64)	예	Flow identifier.

WriteItem

WriteItem

필드	유형	필수	설명
topic	string	예	UNS topic path to write.
value	object	아니요	Payload value to write.
timeStamp	integer (int64)	아니요	Optional Unix timestamp in milliseconds. Omit it to use server time.

WriteReq

WriteReq

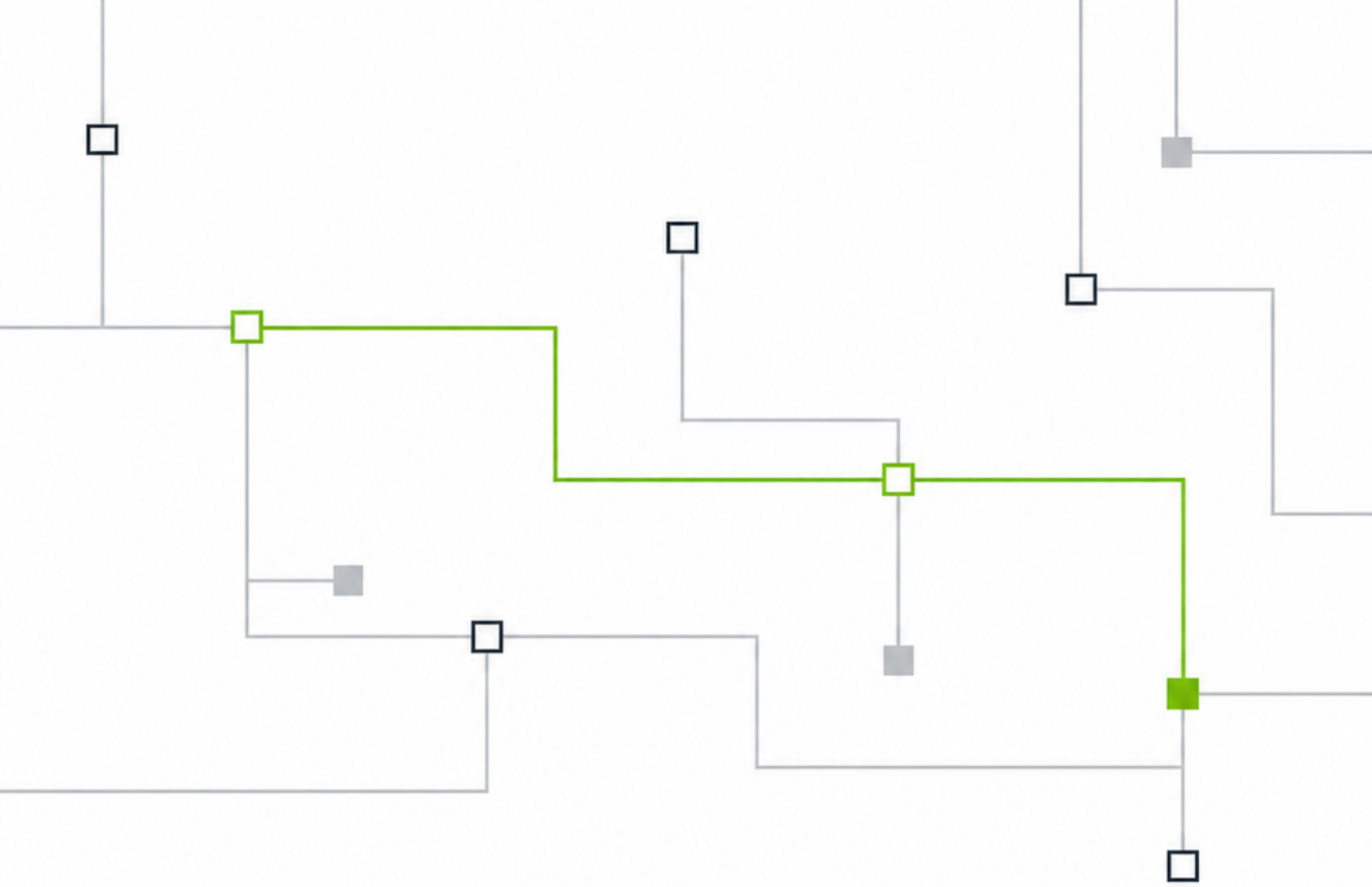
필드	유형	필수	설명
writes	array<object>	예	Write operations to apply.

필드	유형	필수	설명
qos	integer (int64)	아니요	MQTT QoS: 0, 1, or 2.
retain	boolean (boolean)	아니요	Whether the MQTT message should be retained.

WriteResp

WriteResp

필드	유형	필수	설명
success	boolean (boolean)	예	Whether the operation succeeded.
results	array<object>	예	Per-item results. Check success and error for each item.



TIER 0

tier0.app